

Lattice-Based Accumulator and Application to Anonymous Credential Revocation

Victor Youdom Kemmoe [†], Anna Lysyanskaya [†], Ngoc Khanh Nguyen [‡]

[†]Brown University, US
{vyoudomk, anna}@cs.brown.edu
[‡]King's College London, UK
ngoc_khanh.nguyen@kcl.ac.uk

Abstract

An accumulator is a cryptographic system for compactly representing a set of elements such that every element in the set has a short membership witness. A dynamic accumulator, furthermore, allows elements to be added to and deleted from the accumulator. Camenisch and Lysyanskaya (CRYPTO'02) constructed the first dynamic accumulator under the strong-RSA assumption and showed how it can be used to enable revocation of anonymous credentials. In this paper, we give a lattice-based dynamic accumulator tailor-made for enabling revocation of post-quantum anonymous credential systems. As a concrete example, we instantiate our dynamic accumulator on top of the anonymous credential system implemented in the LaZer library (ACM CCS 2024).

1 Introduction

A cryptographic accumulator [BdM93] allows one to represent a set $\mathcal{S}$ by a *short* digest $\mathcal{A}_{\mathcal{S}}$. The important feature of an accumulator is that it supports compact proofs of membership: corresponding to each element $x \in \mathcal{S}$, there is a short witness $w_{x, \mathcal{A}_{\mathcal{S}}}$ attesting to the fact that x is in fact in the set represented by $\mathcal{A}_{\mathcal{S}}$.¹ The security properties of a cryptographic accumulator guarantee that (informally) an adversary cannot find a value $x \notin \mathcal{S}$ and a membership witness $w_{x, \mathcal{A}_{\mathcal{S}}}$.

Further, an accumulator is *dynamic* [CL02a] if elements can be added to or deleted from the set such that membership witnesses can be quickly updated. In other words, if $x \neq y \in \mathcal{S}$ and $\mathcal{S}' = \mathcal{S} - \{y\}$, then $\mathcal{A}_{\mathcal{S}'}$ can be efficiently computed from $\mathcal{A}_{\mathcal{S}}$ and y (rather than the entire $\mathcal{S}'$) and $w_{x, \mathcal{A}_{\mathcal{S}'}}$ can be efficiently computed from $w_{x, \mathcal{A}_{\mathcal{S}}}$, $\mathcal{A}_{\mathcal{S}}$, $\mathcal{A}_{\mathcal{S}'}$ and a short (independent of the size of $\mathcal{S}$) update message. A dynamic accumulator with free addition [BCD⁺17], [KL24] allows an accumulator manager in possession of a trapdoor to add elements to the set $\mathcal{S}$ without modifying the value $\mathcal{A}_{\mathcal{S}}$: the trapdoor allows the manager to compute $w_{x, \mathcal{A}_{\mathcal{S}}}$ for any x . As a result, the value $\mathcal{A}$ of the accumulator would need to change only when an element is deleted. The security properties of a dynamic accumulator guarantee that (informally) an adversary cannot find a value $x \notin \mathcal{S}$ and a membership witness $w_{x, \mathcal{A}_{\mathcal{S}}}$ even when $\mathcal{S}$ and $\mathcal{A}_{\mathcal{S}}$ are the result of a game between the adversary and the challenger where the adversary directs the challenger to add and remove elements and provide witnesses for added elements.

¹Sometimes this flavor of accumulator is called a *positive* accumulator to tell it apart from the *negative* accumulator that admits witnesses of non-membership attesting to $z \notin \mathcal{S}$; and from the *universal* accumulator where for any x there is a witness, either of membership if $x \in \mathcal{S}$ or of non-membership if $x \notin \mathcal{S}$ [LLX07, BCD⁺17]. In this paper we don't consider negative and universal accumulators and thus do not discuss them further.

A dynamic accumulator with free addition is a convenient way to handle revocation of credentials [BCD⁺17]. Together with a public key for verifying a credential, the credential issuing and revocation authority publishes an initial value for an accumulator $\mathcal{A}_0 = \mathcal{A}_\emptyset$ for which it has a trapdoor; this value will be updated when credentials are revoked. To issue a credential corresponding to x , the authority computes the witness $w_{x, \mathcal{A}_t}$ where $\mathcal{A}_t$ is the current value of the accumulator at step t ; $w_{x, \mathcal{A}_t}$ is then given to the credential holder; the set $\mathcal{S}$ is updated to include x so that $\mathcal{S}_{t+1} = \mathcal{S}_t \cup \{x\}$, but the accumulator value does not change, i.e. $\mathcal{A}_{t+1} = \mathcal{A}_t$. To revoke a credential corresponding to $y \in \mathcal{S}_t$, the authority updates the accumulator value from $\mathcal{A}_t$ to $\mathcal{A}_{t+1}$ that corresponds to the set $\mathcal{S}' = \mathcal{S} - \{y\}$; the dynamic nature of the accumulator allows this operation to be done efficiently. The authority then publishes $\mathcal{A}_{t+1}$ and the short update message that allows each unrevoked credential holder to update its witness from $w_{x, \mathcal{A}_t}$ to $w_{x, \mathcal{A}_{t+1}}$.

Application to revocation of anonymous credentials. Recall [BBC⁺24] that in an attribute-based credential system, users obtain credentials from issuing organizations; on a high level, a credential is the organization's signature on a vector of values $(a_1, \dots, a_\ell)$ corresponding to ℓ attributes of the user's identity, for example, the user's ID number, expiration date, age, and other application-specific information; these are obtained in a privacy-preserving manner in which the issuer may not learn the values of all of the attributes, but just the fact that they satisfy its credential-granting policy. When showing an anonymous credential to a verifier with a particular access control policy, a user does not show its attributes and the issuer's signature in the clear, but instead carries out a zero-knowledge proof of knowledge of a credential that satisfies the verifier's policy.

Camenisch and Lysyanskaya [CL02a] demonstrated that dynamic accumulators were a useful primitive for constructing anonymous credential systems with revocation. Let one of the credential attributes be associated with a revocation handle: the issuer samples a fresh handle x every time it issues a credential, and sets the value of the user's revocation attribute (say the i^{th} attribute is the revocation attribute) to it: $a_i := x$; then it issues the credential with attributes $(a_1, \dots, a_\ell)$ to the user, and also adds x to the accumulator set $\mathcal{S}$ represented by $\mathcal{A}_\mathcal{S}$ and sends $(x, w_{x, \mathcal{A}_\mathcal{S}})$ to the user.

When showing an anonymous credential, a user Alice would not reveal its revocation handle x and the witness $w_{x, \mathcal{A}}$ in the clear. Instead, Alice would compute a (non-interactive) cryptographic commitment to these values, and use a zero-knowledge proof system to prove that (1) she is in possession of the credential with revocation attribute x ; and (2) she is in possession of the witness w that x belongs to the set represented by $\mathcal{A}$. Thus, it is important that a cryptographic accumulator used for the purposes of anonymous credential revocation support a commit-and-prove paradigm for elements of the accumulated sets and their witnesses.

When a credential with revocation attribute y needs to be revoked, the credential issuer's accumulator $\mathcal{A}_\mathcal{S}$ is updated to $\mathcal{A}_{\mathcal{S}'} = \mathcal{A}_{\mathcal{S} - \{y\}}$. Alice whose revocation attribute $x \neq y$ is still in the accumulator can update her witness $w_{x, \mathcal{A}_\mathcal{S}}$ to $w_{x, \mathcal{A}_{\mathcal{S}'}}$ and will still be able to show her credential. However, the security properties of a dynamic accumulator ensure that Bob whose revocation attribute is y will no longer be able to do so.

Post-quantum-secure cryptographic accumulators. State-of-the-art constructions of positive dynamic accumulators are based on the strong RSA assumption [CL02a, LLX07, KL24] or the q -strong Diffie-Hellman assumption [Ngu05, ATSM09, CKS09] that do not hold in the post-quantum setting. Their free-addition counterparts allow to add elements without modifying the accumulator's value [BCD⁺17, KL24, KB21, JML24]. Another approach — Merkle-tree-based constructions [RY16] — only yields an accumulator in the special case where the size of the universe is polynomial. Then the accumulator value $\mathcal{A}_\mathcal{S}$ is the root of the Merkle tree where leaf i contains 1 iff $i \in \mathcal{S}$. A lattice-based construction of Papamanthou et al. [PSTY13] uses Ajtai's Module-SIS-based hash function [Ajt96] to instantiate a Merkle tree, which therefore yields a bounded-size accumulator. Building on this work, Yu et al. [YAY⁺18] adapted Papaman-

thou et al.’s construction to support non-membership witnesses, and Libert et al. [LLNW23] presented a Stern-type protocol [Ste96] to prove knowledge of an accumulated element. Zhao et al. [ZYH24] adapted the Module-SIS based vector commitment of Peikert et al. [PPS21] by introducing a trapdoor to obtain a dynamic accumulator of bounded size (the number of items that can be added are explicitly bounded by the public parameters) with short update information. De Castro and Peikert [dCP23] constructed an updatable functional commitment scheme based on the Module-SIS assumption. They hinted at how their functional commitment could be used to instantiate a dynamic accumulator with witnesses (evaluation openings) that have size linear in the bit length of the input. Wee and Wu [WW23a] improved the evaluation opening’s size in De Castro and Peikert’s construction by relying on the ℓ -succinct Module-SIS assumption [Wee24].

However, none of these prior lattice-based constructions have free addition. I.e., every time an element is added to the accumulator, the accumulator’s value must be updated, and some update information must be released. We present a comparison between those schemes and ours in Table 1. Note that for a fixed polynomial-size set $\mathcal{S}$, any positive dynamic accumulator can be transformed into a non-adaptive accumulator with free addition using preprocessing: during the preprocessing phase, the accumulator manager precomputes the accumulator’s value by adding all elements of $\mathcal{S}$ to the accumulator and storing membership witnesses for each of them. Then, during the online phase, when an Add query is sent for an element $x \in \mathcal{S}$, the accumulator manager retrieves its precomputed membership witness without changing the accumulator’s value.

In the literature, we also have updatable vector commitments [PPS21], [WW23b] based on the Module-SIS assumption that can be used to instantiate a bounded-size accumulator. However, the communication complexity of the resulting accumulator is worse than the previous one listed since after an addition or a deletion, a unique update message has to be issued for each element in the accumulator.

Scheme	Assumption	Security	$ w $	$ \text{upmsg} _{\text{Add}}$	$ \text{upmsg} _{\text{Del}}$	$ \text{pp} $
[CL02a] [LLX07] [KL24]	Strong RSA	Adaptive	$\ell \cdot \text{poly}(\lambda)$	ℓ^*	ℓ	$\text{poly}(\lambda)$
[BCD ⁺ 17], [KL24]	Strong RSA	Non-adaptive [†]	$\ell \cdot \text{poly}(\lambda)$	—	ℓ	$\text{poly}(\lambda)$
[Ngu05], [ATSM09], [CKS09]	q -Strong DH	Adaptive	$\text{poly}(\lambda)$	ℓ^*	ℓ	$s \cdot \text{poly}(\lambda)$
[KB21, JML24]	q -Strong DH	Non-adaptive [‡]	$\text{poly}(\lambda)$	—	ℓ	$\text{poly}(\lambda)$
[PSTY13], [YAY ⁺ 18], [LLNW23]	M-SIS	Adaptive	$\text{poly}(\lambda) \cdot \log s$	$\text{poly}(\lambda) \cdot \log s^*$	$\text{poly}(\lambda) \cdot \log s$	$\text{poly}(\lambda)$
[ZYH24]	M-SIS	Adaptive	$\text{poly}(\lambda)$	$\text{poly}(\lambda)^*$	$\text{poly}(\lambda)$	$\text{poly}(\lambda) \cdot s \log s$
[dCP23]	M-SIS	Selective	$\ell \cdot \text{poly}(\lambda)$	ℓ^*	ℓ	$\ell \cdot \text{poly}(\lambda)$
[dCP23]+[WW23a]	ℓ -Succinct M-SIS	Selective	$\text{poly}(\lambda)$	ℓ^*	ℓ	$\ell^2 \cdot \text{poly}(\lambda)$
Our work	M-SIS	Selective	$\ell \cdot \text{poly}(\lambda)$	—	ℓ	$\ell \cdot \text{poly}(\lambda)$
	ℓ -Succinct M-SIS	Selective	$\text{poly}(\lambda)$	—	ℓ	$\ell^2 \cdot \text{poly}(\lambda)$

Table 1: Lattice-based positive dynamic accumulators. $|w|$ is the size of the witness, $|\text{upmsg}|_{\text{Op}}$ is the size the information needed to update a witness after executing $\text{Op} \in \{\text{Add}, \text{Del}\}$, $|\text{pp}|$ is the size of the public parameters, ℓ is the length of an input, s represents the cardinality of the accumulated set. By adding $*$ to $|\text{upmsg}|_{\text{Add}}$ column, we remind the reader that pre-processing would allow reducing $|\text{upmsg}|_{\text{Add}}$ to 0. For Security, Non-adaptive[†] means adaptivity can be achieved under the same assumption in the random oracle model (ROM), while Non-adaptive[‡] means adaptivity can be achieved under the same assumption in the ROM or under the q -Inversion Symmetric Diffie-Hellman assumption [JML24] in the plain model.

Our results. In this paper, we give the first post-quantum dynamic accumulator with free addition that works in the commit-and-prove paradigm and is directly applicable to revocation of anonymous credentials in existing lattice-based constructions [BLNS23b], [JRS23], [LCL⁺23] [AGJ⁺24]. Our first, simplest con-

structions (based on the Agrawal, Boneh and Boyen signature scheme [ABB10]) comes with caveats: we relax the definition of security so that it is still sufficient for the desired application in two ways: (1) we require selective security, i.e. the value $x^* \notin \mathcal{S}$ whose membership in the accumulator the adversary aims to forge is given to the challenger in advance; and (2) the security game is replacement-free, i.e. once some element y is removed from the accumulator it can never be added back in. (We note that the same caveats apply to the security of the De Castro and Peikert accumulator for the case where only deletions and not additions cause the accumulator value to change.) This is good enough for our application: To see why (1) is a reasonable relaxation, recall that it is the credential issuer who chooses the revocation handles that it then authenticates as part of the anonymous credential, so the adversary does not have the freedom to choose any value for x^* in order to use a revoked credential. As for (2), we may simply require that, once a credential is revoked, it cannot be reinstated with the same revocation handle; the user must obtain a new credential with a new revocation handle. For settings where (1) and (2) could be problematic, we give an upgraded version of our construction: an adaptively secure positive dynamic accumulator obtained by combining our first construction with an adaptively secure digital signature. As we explain in Section 5.3 (see Table 2), when we set the security parameters in our construction appropriately to achieve security on the 128-bit level, we get accumulator and witness sizes that are promising — and perhaps can be further optimized to become truly practical.

1.1 Technical Overview

In this subsection, we present an overview of our Module Short Integer Solution (M-SIS) based positive replacement-free accumulator (PRFA), i.e. a positive dynamic accumulator in which deleted elements cannot be re-added. Our accumulator can support polynomially many deletions for optimal parameters and admits an efficient zero-knowledge commit-and-prove argument to prove membership of an element.

Homomorphic Operations. First, we recall a key equation from lattice-based cryptography that was first laid out by Gentry, Sahai and Waters [GSW13] and Boneh et al. [BGG⁺14]. Let $R \supseteq \mathbb{Z}$ be a ring, $R_q = R/qR$, with $q \in \mathbb{Z}$, and $\mathcal{F} = \{f : \{0, 1\}^\ell \rightarrow \{0, 1\}\}$ be a family of Boolean circuits. For any $f \in \mathcal{F}$, $\mathbf{A} = [\mathbf{A}_0 \mid \cdots \mid \mathbf{A}_{\ell-1}] \in R_q^{n \times \ell m}$, $\mathbf{x} \in \{0, 1\}^\ell$, there exist matrices $\mathbf{A}_f \in R_q^{n \times m}$ and $\mathbf{H}_{f,\mathbf{x}}$ such that

$$(\mathbf{A} - \mathbf{x}^\top \otimes \mathbf{G}) \cdot \mathbf{H}_{f,\mathbf{x}} = \mathbf{A}_f - f(\mathbf{x}) \cdot \mathbf{G} \quad (1.1)$$

where $\mathbf{G} \in R_q^{n \times m}$ is a gadget matrix [MP12]. The matrix $\mathbf{A}_f$ can be efficiently computed from the matrix $\mathbf{A}$ and the function f , and the matrix $\mathbf{H}_{f,\mathbf{x}}$ can be efficiently computed from the matrix $\mathbf{A}$, the function f , and the vector $\mathbf{x}$. Moreover, if $\mathcal{F}$ is a family of indicator functions, i.e., each $f \in \mathcal{F}$ is defined with respect to $\tilde{\mathbf{x}} \in \{0, 1\}^\ell$ such that $f_{\tilde{\mathbf{x}}}(\mathbf{x}) = 1$ if $\mathbf{x} = \tilde{\mathbf{x}}$, then $\|\mathbf{H}_{f,\mathbf{x}}\|_\infty = 1$ when $\mathbf{G}$ is the base 2 gadget matrix (cf. Corollary 2.2.1). In addition, given a vector $\mathbf{u} \in R_q^n$, we can compress $\mathbf{H}_{f,\mathbf{x}}$ and $\mathbf{A}_f$ into vectors $\mathbf{h}_{f,\mathbf{x}} = \mathbf{H}_{f,\mathbf{x}} \cdot \mathbf{G}^{-1}(\mathbf{u})$ and $\mathbf{a}_f = \mathbf{A}_f \cdot \mathbf{G}^{-1}(\mathbf{u})$, respectively, such that $(\mathbf{A} - \mathbf{x}^\top \otimes \mathbf{G}) \cdot \mathbf{h}_{f,\mathbf{x}} = \mathbf{a}_f - f(\mathbf{x}) \cdot \mathbf{u}$ where $\mathbf{G}^{-1}(\mathbf{u}) \in R_q^m$ is a vector such that $\mathbf{G} \cdot \mathbf{G}^{-1}(\mathbf{u}) = \mathbf{u}$.

Updatable Signature. We recall the M-SIS based selectively secure signature of Agrawal, Boneh and Boyen [ABB10] that we will call ABB. In ABB, the public key is $\text{pp} = (\mathbf{A}, \mathbf{B}, \mathbf{C})$ where $\mathbf{A} \in R_q^{n \times m}$, $\mathbf{B} \in R_q^{n \times \ell m'}$ and $\mathbf{C} \in R_q^{n \times m'}$, and the secret key is $\text{sk} = \mathbf{T}_\mathbf{A} \in R^{m \times m'}$. $\mathbf{T}_\mathbf{A}$ is a trapdoor that allows to efficiently sample a low-norm matrix $\mathbf{V} \in R^{(m+\ell m') \times m'}$ such that $[\mathbf{A} \mid \bar{\mathbf{B}}] \cdot \mathbf{V} = \mathbf{U}$ for any $\mathbf{U} \in R_q^{n \times m'}$ and $\bar{\mathbf{B}} \in R_q^{n \times \ell m'}$. To sign a message $\mathbf{x} \in \{0, 1\}^\ell$, one uses $\mathbf{T}_\mathbf{A}$ to sample a low-norm matrix $\mathbf{S}_\mathbf{x}$ such that $[\mathbf{A} \mid \mathbf{B} - \mathbf{x}^\top \otimes \mathbf{G}] \cdot \mathbf{S}_\mathbf{x} = \mathbf{C}$. Now, we observe that ABB's public key can be made updatable by using equation (1.1). More specifically, given a boolean function $f : \{0, 1\}^\ell \rightarrow \{0, 1\}$, $\mathbf{C}$ can be updated

to $C' = C + B_f$, and for any $y \in \{0, 1\}^\ell$ such that $f(y) = 0$, a signature S_y on y can be updated to $S'_y = S_y + [0_{m' \times m} | H_{f,y}^\top]^\top$:

$$\begin{aligned} [A | B - y^\top \otimes G] \cdot S'_y &= C + (B - y^\top \otimes G) \cdot H_{f,y} \\ &= C + B_f \quad (\text{Since } f(y) = 0) \end{aligned}$$

Hence, seemingly f can be used to *revoke* ABB signatures on any x' such that $f(x') = 1$. Below, we expand on this intuition.

Replacement-free accumulator. Now we are ready to give an overview of our PRFA (cf. Section 4). Similarly to the ABB signature, our public parameters are $pp = (A, B)$; as for $C \in R_q^{n \times m'}$, it is our initial accumulator value: $\mathcal{A}_0 = C$. Also following ABB, the secret key is a $sk = T_A$. To add an element $x \in \{0, 1\}^\ell$ to the accumulator, the accumulator manager *signs* x by using T_A to sample $w_x = S_x \in R^{(m+\ell m') \times m'}$ such that $[A | B - x^\top \otimes G] \cdot S_x = C$.

To delete an element $y \in \{0, 1\}^\ell$ from the accumulator, the accumulator manager updates the accumulator value from $\mathcal{A} = C$ to $\mathcal{A}' = C + B_{\mathbb{1}_y}$, where $\mathbb{1}_y : \{0, 1\}^\ell \rightarrow \{0, 1\}$ represents the indicator function for y . The accumulator manager publishes the new accumulator value $\mathcal{A}'$ along with the value y . Any user that holds an element $z \neq y$ and its witness $w_z = S_z$ can update w_z from the tuple $upmsg = (\mathcal{A}', y)$ by computing $w'_z = S_z + [0_{m' \times m} | H_{\mathbb{1}_y, z}^\top]^\top$. Note that $\|H_{\mathbb{1}_y, z}\|_\infty = 1$ (cf. Corollary 2.2.1), so $\|w'_z\|_\infty = \|w_z\|_\infty + 1$. Therefore, our PRFA can support polynomially many deletions and unbounded additions under M-SIS with a polynomial bound.

Unfortunately, for this construction, the replacement-free property of the security game is necessary for it: Suppose an element $x' \in \{0, 1\}^\ell$ was added, deleted and then re-added. This means that there exist $S_{x'}, H_{\mathbb{1}_{x'}, x'}, \tilde{S}_{x'}$ such that $[A | B - (x')^\top \otimes G] \cdot (S_{x'} + [0_{m' \times m} | H_{\mathbb{1}_{x'}, x'}^\top]^\top) = C + B_{\mathbb{1}_{x'}} - G$ and $[A | B - (x')^\top \otimes G] \cdot \tilde{S}_{x'} = C + B_{\mathbb{1}_{x'}}$. Therefore, $T = (S_{x'} + [0_{m' \times m} | H_{\mathbb{1}_{x'}, x'}^\top]^\top - \tilde{S}_{x'})$ is a G -trapdoor [MP12] for the matrix $[A | B - (x')^\top \otimes G]$, i.e., using T , we can always sample a valid membership witness for x' . Note that by setting $\mathcal{A} = c \in R_q^n$ instead of a matrix and by adding a vector $u \in R_q^n$ to the public paradigm, i.e., $pp = (A, B, u)$, we can compress membership witnesses as follows: when adding an element x , instead of sampling a matrix S_x , we sample a vector $w_x = s_x \in R^{m+\ell m'}$ such that $[A | B - x^\top \otimes G] \cdot s_x = c$, and after deleting an element y , $\mathcal{A}$ is updated to $\mathcal{A}' = \mathcal{A} + B_{\mathbb{1}_y} \cdot G^{-1}(u)$ and w_x is updated to $w'_x = w_x + [0_{m' \times m} | H_{\mathbb{1}_y, x}^\top]^\top \cdot G^{-1}(u)$. In Section 4.1, we show that by using vectors for $\mathcal{A}$ and w_x instead of matrices, one will need to breach the replacement-free condition polynomially many time to obtain a trapdoor, thereby reinforcing the replacement-free condition. We also show how, by increasing the sizes of public parameters and witnesses, the construction can be adapted to completely prevent the leakage of trapdoors in case the replacement-free condition is not enforced. In Section 4.2, we show how to adapt Baldimsti et al.'s compiler [BCD⁺17] to transform our PRFA into an adaptively secure positive dynamic accumulator.

Commit-and-Prove Argument. We show how to efficiently prove knowledge of the revocation attribute x and its membership witness s_x using the recently developed succinct zero-knowledge frameworks from lattices. In short, the task boils down to proving knowledge of $x \in \{0, 1\}^\ell$ and a short vector s_x that satisfy the following relation:

$$[A | B - x^\top \otimes G] \cdot s_x = c. \quad (1.2)$$

We first describe how to prove this type of statement in zero-knowledge using the LNP framework [LNP22a]. Other than the prover's last message z (which is linear in the witness length), this proof is succinct. To achieve succinctness, we replace z with a proof of knowledge of z satisfying the verifier of the LNP protocol; this step uses the LaBRADOR proof system by Beullens and Seiler [BS23], which provides concretely

attractive proof sizes. Note that, due the LNP layer above, we do not need to apply the zero-knowledge variant of the LaBRADOR protocol. Although this general strategy has already been described in [BS23], we provide a more explicit description suited to the statement in Equation 1.2 (see Appendix D for more details). Finally, in Section 5 we describe how to upgrade the existing lattice-based anonymous credentials [BLNS23b], [JRS23], [LCL⁺23], [AGJ⁺24], that follow the “signatures with efficient protocols” paradigm [CL02b], to support credential revocation using our developed commit-and-prove system.

We provide a concrete example of appending the revocation mechanism on top of the anonymous credential system implemented in the LaZer library [LSS24]. For the revocation attribute of length $\ell = 32$, we obtain the accumulator value $\mathbf{c}$ of size 50KB, while a membership witness $\mathbf{s}_x$ has size 14.7MB (see Table 2). By applying the composition of LNP and LaBRADOR protocols, our commit-and-prove argument produces proofs of size at most 125KB. Although not yet fully ready for practical deployment, we believe this is a stepping stone towards concretely efficient post-quantum positive accumulators with free addition.

2 Preliminaries

Notations. A function $f : \mathbb{N} \rightarrow [0, 1]$ is negligible if $f(x) = o(x^{-c})$ for all $c \in \mathbb{N}$. We use $\text{negl}(\cdot)$ to denote a negligible function. We use $\lambda \in \mathbb{N}$ to denote the security parameter, and for $n \in \mathbb{N}$, $[n] = \{0, \dots, n-1\}$. Given a set $\mathcal{S}$ and $a \in \mathcal{S}$, $\mathbb{1}_a : \mathcal{S} \rightarrow \{0, 1\}$ denotes the indicator function of the singleton $\{a\}$, $x \leftarrow \mathcal{S}$ denotes sampling x uniformly at random from $\mathcal{S}$, and if $\mathcal{S}$ is finite, $U(\mathcal{S})$ denotes the uniform distribution over $\mathcal{S}$. We use bold lowercase letters to denote vectors/tuples (e.g., $\mathbf{a}$) and bold uppercase letters to denote matrices (e.g., $\mathbf{A}$). By default, all vectors are column vectors. Given a vector $\mathbf{a}$, we either use a_i or $\mathbf{a}[i]$ to denote its i -th component. Let $q, d \in \mathbb{N}$ such that d is a power of two. We use $\mathbb{Z}_q$ to denote the ring of integers modulo q , $R = \mathbb{Z}[X]/(X^d + 1)$ to denote the ring of integers of the $2d$ -th cyclotomic field, and $R_q = R/qR$. For any $f(X) = \sum_{i=0}^{d-1} f_i X^i \in R$, we define the ℓ_∞ and ℓ_p norms as follows:

$$\|f\|_\infty = \max_{0 \leq i < d} |f_i| \quad , \quad \|f\|_p = \sqrt[p]{|f_0|^p + \dots + |f_{d-1}|^p}$$

For any $\mathbf{f} \in R^k$, with $k \in \mathbb{N}_{>1}$, we extend the ℓ_∞ and ℓ_p norms as follows:

$$\|\mathbf{f}\|_\infty = \max_{0 \leq i < k} \|\mathbf{f}[i]\|_\infty \quad , \quad \|\mathbf{f}\|_p = \sqrt[p]{\|\mathbf{f}[0]\|_p^p + \dots + \|\mathbf{f}[k-1]\|_p^p}$$

By default, $\|\mathbf{f}\| = \|\mathbf{f}\|_2$. For $1 \leq p \leq \infty$, let $S_\beta^p = \{x \in R_q : \|x\|_p \leq \beta\}$. Additionally, we define $\mathcal{B}_1$ to be the set of all polynomials in R with coefficients in $\{0, 1\}$.

Let $\mathcal{R}$ be a ring such that any free $\mathcal{R}$ -module $\mathcal{R}^n$ is equipped with the ℓ_2 -norm. We use $\mathbf{I}_k \in \mathcal{R}^{k \times k}$ to denote the identity matrix with entries in $\mathcal{R}$, and for any matrix $\mathbf{A} \in \mathcal{R}^{m \times n}$, $s_1(\mathbf{A}) = \sup_{\mathbf{x}} \frac{\|\mathbf{A}\mathbf{x}\|}{\|\mathbf{x}\|}$ denotes its second operator norm and $\|\mathbf{A}\|_\infty = \max_{i,j} \|\mathbf{A}_{i,j}\|_\infty$.

Tensor product Given two matrices $\mathbf{A}$ and $\mathbf{B}$ with entries in a ring $\mathcal{R}$, we use $\mathbf{A} \otimes \mathbf{B}$ to denote the tensor/Kronecker product between $\mathbf{A}$ and $\mathbf{B}$. In addition, for any matrices $\mathbf{A}, \mathbf{B}, \mathbf{C}, \mathbf{D}$ with entries in $\mathcal{R}$ such that the products $\mathbf{AB}$ and $\mathbf{CD}$ are well-defined, the Kronecker product satisfies the mixed-product property: $(\mathbf{A} \otimes \mathbf{B})(\mathbf{C} \otimes \mathbf{D}) = (\mathbf{AC} \otimes \mathbf{BD})$.

Definition 2.1 (Samplable Distributions [AB09, CHW25]). *A family of distributions $\mathcal{D} = \{D_\lambda\}_{\lambda \in \mathbb{N}}$ with associated auxilliary information $\{\text{aux}_\lambda\}_{\lambda \in \mathbb{N}}$ is efficiently samplable if there exist a ppt algorithm Samp and a polynomial $\mathfrak{r}$ such that the following distributions are statistically close:*

$$\left\{ x : \begin{array}{l} r \leftarrow \{0, 1\}^{\mathfrak{r}(\lambda)} \\ x \leftarrow \text{Samp}(1^\lambda, \text{aux}_\lambda; r) \end{array} \right\} \stackrel{s}{\approx} \{x : x \leftarrow D_\lambda\}$$

When the randomness of Samp is implicit, we just write $\text{Samp}(1^\lambda, \text{aux}_\lambda)$.

Definition 2.2 (Pseudo-random function). A function $F : \mathcal{K} \times \mathcal{S} \rightarrow \mathcal{T}$ is a pseudo-random function (PRF) if for any ppt adversary $\mathcal{A}$, $|\Pr[1 \leftarrow \mathcal{A}^{F_k(\cdot)}(1^\lambda)] - \Pr[1 \leftarrow \mathcal{A}^{f(\cdot)}(1^\lambda)]| \leq \text{negl}(\lambda)$, where $k \leftarrow \mathcal{K}$ and $f \leftarrow \text{Funcs}[\mathcal{S}, \mathcal{T}]$ (the set of all functions from $\mathcal{S}$ to $\mathcal{T}$).

Definition 2.3 (Commitment Scheme). A commitment scheme for a domain $\mathcal{M}$ is a tuple of ppt algorithms $\Pi_{\text{Com}} = (\text{Setup}, \text{Commit}, \text{Verify})$ with the following syntax:

- $\text{Setup}(1^\lambda) \rightarrow \text{crs}_{\text{Com}}$: Given a security parameter $\lambda \in \mathbb{N}$ in unary, the Setup algorithm outputs a common reference string crs_{Com} .
- $\text{Commit}(\text{crs}_{\text{Com}}, m) \rightarrow (c, o)$: Given a common reference string crs_{Com} and a value $m \in \mathcal{M}$, the Commit algorithm outputs a commitment c and opening o .
- $\text{Verify}(\text{crs}_{\text{Com}}, c, m, o) \rightarrow 1/0$: Given a common reference string crs_{Com} , a commitment c , a value $m \in \mathcal{M}$, and an opening o , the deterministic Verify algorithm outputs 1 if o is a valid opening of the value m with respect to c . Else, it outputs 0.

In addition, Π_{Com} should satisfy the following properties:

- **Correctness**: For any $\lambda \in \mathbb{N}$, $m \in \mathcal{M}$, $\text{crs}_{\text{Com}} \leftarrow \text{Setup}(1^\lambda)$, $(c, o) \leftarrow \text{Commit}(\text{crs}_{\text{Com}}, m)$,

$$\Pr[\text{Verify}(\text{crs}_{\text{Com}}, c, m, o) = 1] \geq 1 - \text{negl}(\lambda).$$

- **Hiding**: For any ppt adversary $\mathcal{A}$, there exists a negligible function $\text{negl}(\lambda)$ such that for any $\lambda \in \mathbb{N}$, $\text{crs}_{\text{Com}} \leftarrow \text{Setup}(1^\lambda)$, $b \leftarrow \{0, 1\}$,

$$\Pr \left[\begin{array}{l} (m_0, m_1, \text{st}_{\mathcal{A}}) \leftarrow \mathcal{A}(1^\lambda, \text{crs}_{\text{Com}}), \\ (c_b, o_b) \leftarrow \text{Commit}(\text{crs}_{\text{Com}}, m_b) \\ b' \leftarrow \mathcal{A}(\text{st}_{\mathcal{A}}, c_b) : \\ b' = b \end{array} \right] \leq \frac{1}{2} + \text{negl}(\lambda)$$

- **Binding**: For any ppt adversary $\mathcal{A}$, there exists a negligible function $\text{negl}(\lambda)$ such that for any $\lambda \in \mathbb{N}$, $\text{crs}_{\text{Com}} \leftarrow \text{Setup}(1^\lambda)$

$$\Pr \left[\begin{array}{l} (c, m_0, o_0, m_1, o_1) \leftarrow \mathcal{A}(1^\lambda, \text{crs}_{\text{Com}}) : \\ \text{Verify}(\text{crs}_{\text{Com}}, c, m_0, o_0) = 1 \wedge \\ \text{Verify}(\text{crs}_{\text{Com}}, c, m_1, o_1) = 1 \wedge \\ m_0 \neq m_1 \end{array} \right] \leq \text{negl}(\lambda)$$

Definition 2.4 (Digital Signature). A digital signature for a domain $\mathcal{M}$ is a tuple of ppt algorithms $\Sigma = (\text{Gen}, \text{Sign}, \text{Verify})$ with the following syntax:

- $\text{Gen}(1^\lambda) \rightarrow (\text{pk}, \text{sk})$: Given a security parameter $\lambda \in \mathbb{N}$ in unary, the Gen algorithm outputs a public key pk and a secret key sk .
- $\text{Sign}(\text{sk}, m) \rightarrow \sigma$: Given a secret key sk , a message $m \in \mathcal{M}$, the Sign algorithm outputs a signature σ .

- **Verify**(pk, m, σ) $\rightarrow$ 0/1: Given a public key pk, a message $m \in \mathcal{M}$, and a signature σ , the deterministic Verify algorithm outputs 1 if σ is a signature on m generated by the secret key associated to pk. Else, it outputs 0.

In addition, Σ should satisfy the following properties:

- **Correctness**: For any $\lambda \in \mathbb{N}$, $m \in \mathcal{M}$, $(\text{pk}, \text{sk}) \leftarrow \text{Gen}(1^\lambda)$, $\sigma \leftarrow \text{Sign}(\text{sk}, m)$,

$$\Pr[\text{Verify}(\text{pk}, m, \sigma) = 1] \geq 1 - \text{negl}(\lambda)$$

- **Existential Unforgeability against Chosen Message Attack (EUF-CMA)**: For any ppt adversary $\mathcal{A}$ with oracle access to Sign, there exists a negligible function $\text{negl}(\lambda)$ such that for any $\lambda \in \mathbb{N}$,

$$\Pr \left[\begin{array}{l} (\text{pk}, \text{sk}) \leftarrow \text{Gen}(1^\lambda), \\ (m^*, \sigma^*) \leftarrow \mathcal{A}^{\text{Sign}(\text{sk}, \cdot)}(1^\lambda, \text{pk}) : \\ \text{Verify}(\text{pk}, m^*, \sigma^*) = 1 \wedge m^* \notin Q_{\text{Sign}} \end{array} \right] \leq \text{negl}(\lambda)$$

where Q_{Sign} represents the set of signature queries that $\mathcal{A}$ sent to its oracle.

2.1 Lattice Primer

In this subsection, we recall some important lemmas and theorems of lattice-based cryptography.

Definition 2.5 (Gaussian function). Given $\mathbf{c} \in \mathbb{R}^k$ and a Gaussian width parameter $\mathfrak{s} > 0$, the Gaussian function $\rho_{\mathfrak{s}, \mathbf{c}} : \mathbb{R}^k \rightarrow \mathbb{R}$ centered at $\mathbf{c}$ is defined by $\rho_{\mathfrak{s}, \mathbf{c}}(\mathbf{x}) = e^{-\pi \|\mathbf{x} - \mathbf{c}\|_2^2 / \mathfrak{s}^2}$ for all $\mathbf{x} \in \mathbb{R}^k$. For any discrete set $\mathcal{X} \subset \mathbb{R}^k$, $\rho_{\mathfrak{s}, \mathbf{c}}(\mathcal{X}) = \sum_{x \in \mathcal{X}} \rho_{\mathfrak{s}, \mathbf{c}}(\mathbf{x})$.

Definition 2.6 (Discrete Gaussian Distribution). For any discrete set $\mathcal{X}$ admitting an efficient embedding ϕ to a subset of $\mathbb{R}^n$, the discrete Gaussian distribution over $\mathcal{X}$ centered at $\mathbf{c} \in \mathbb{R}^n$ with Gaussian width parameter $\mathfrak{s} > 0$ is defined as $D_{\mathcal{X}, \mathfrak{s}, \mathbf{c}}(x) = \rho_{\mathfrak{s}, \mathbf{c}}(\phi(x)) / \rho_{\mathfrak{s}, \mathbf{c}}(\phi(\mathcal{X}))$ for all $x \in \mathcal{X}$. If $\mathbf{c} = \mathbf{0}$, we use $D_{\mathcal{X}, \mathfrak{s}} = D_{\mathcal{X}, \mathfrak{s}, \mathbf{0}}$.

Lattices. A k -dimensional lattice Λ is a discrete additive subgroup of $\mathbb{R}^k$, and its dual is defined by the set $\Lambda^* = \{\mathbf{x} \in \mathbb{R}^k : \forall \mathbf{y} \in \Lambda, \langle \mathbf{x}, \mathbf{y} \rangle \in \mathbb{Z}\}$. For any $\mathbf{A} \in R_q^{n \times m}$, the R -modules $\Lambda^\perp(\mathbf{A}) = \{\mathbf{s} \in R^m : \mathbf{A}\mathbf{s} = \mathbf{0} \bmod q\}$ and $\Lambda(\mathbf{A}^\top) = \{\mathbf{y} \in R^m : \exists \mathbf{s} \in R_q^n \wedge \mathbf{s}^\top \mathbf{A} \equiv \mathbf{y}^\top \bmod q\}$ are md -dimensional (module) lattices [LS15], and their respective duals satisfy the following equality: $(\Lambda^\perp(\mathbf{A}))^* = 1/q \cdot \Lambda(\mathbf{A}^\top)$. For any $\mathbf{c} \in R_q^m$, we use $\Lambda_{\mathbf{c}}^\perp(\mathbf{A}) = \{\mathbf{s} \in R^m : \mathbf{A}\mathbf{s} = \mathbf{c} \bmod q\}$. In general, for any $\lambda \in \mathbb{N}$ and polynomials $n(\cdot), m(\cdot), q(\cdot), d(\cdot)$, we use $n = n(\lambda)$, $m = m(\lambda)$, $q = q(\lambda)$, $d = d(\lambda)$ to denote (module) lattice parameters that determine the ring R_q and the matrix $\mathbf{A}$.

Definition 2.7 (Smoothing Parameter [MR07]). Given a k -dimensional lattice Λ and a positive real $\varepsilon > 0$, the smoothing parameter $\eta_\varepsilon(\Lambda)$ is the smallest real $\mathfrak{s} > 0$ such that $\rho_{1/\mathfrak{s}, \mathbf{0}}(\Lambda^* \setminus \{\mathbf{0}\}) \leq \varepsilon$.

Lemma 2.1 (Smoothing Parameter Bounds [Pei07]). For any k -dimensional lattice Λ , let $\lambda_1^\infty(\Lambda^*)$ be the ℓ_∞ -norm of a shortest vector in Λ^* . Then, for any $\varepsilon > 0$, $\eta_\varepsilon(\Lambda) \leq \frac{1}{\lambda_1^\infty(\Lambda^*)} \cdot \sqrt{\log(2k(1 + 1/\varepsilon))} / \pi$.

Lemma 2.2 (Lattice Tail Bounds). Let Λ be a full rank n -dimensional lattice. For any $\mathbf{c} \in \mathbb{R}^n$, $\varepsilon > 0$ and sufficiently large Gaussian width parameter $\mathfrak{s} \geq \eta_\varepsilon(\Lambda)$, we have

$$a. ([\text{MR07, Lemma 4.4}]) \quad \Pr_{\mathbf{x} \sim D_{\Lambda, \mathfrak{s}, \mathbf{c}}} [\|\mathbf{x} - \mathbf{c}\|_2 > \mathfrak{s}\sqrt{n}] \leq \frac{1+\varepsilon}{1-\varepsilon} \cdot 2^{-n}$$

$$b. ([\text{Pei07, Corollary 5.3}]) \Pr_{\mathbf{x} \sim D_{\Lambda, \mathfrak{s}, \mathbf{c}}} [\|\mathbf{x} - \mathbf{c}\|_{\infty} \geq \mathfrak{s} \cdot \log \sqrt{n}] \leq 2ne^{1-\pi \log^2 \sqrt{n}}$$

Lemma 2.3. *The second operator norm of a matrix satisfy the following bound:*

- Given $\mathfrak{s} > 0$ and $\mathbf{A} \leftarrow (D_{R, \mathfrak{s}})^{n \times m}$, $s_1(\mathbf{A}) \leq \mathfrak{s} \sqrt{d} \cdot (\sqrt{n} + \sqrt{m} + \log \sqrt{d})$ with overwhelming probability [DM14, Fact 6].
- For any matrix $\mathbf{A} \in R^{n \times m}$, $s_1(\mathbf{A}) \leq d \sqrt{nm} \|\mathbf{A}\|_{\infty}$.

We recall additional lemmas regarding the properties of discrete Gaussian samples with respect to lattices.

Lemma 2.4. *Let $q = p_1 p_2 \dots p_t$ where $p_i \in \mathbb{Z}$ is a prime with inertia degree r_i in R and $p_i \neq p_j$ for $i \neq j$. For a matrix $\mathbf{A} \leftarrow R_q^{n \times m}$, let E be the event that $\mathbf{A}$ contains an invertible $n \times n$ submatrix. Then,*

$$\Pr[\neg E] \leq d \sum_i p_i^{-r_i(m-n)}.$$

Proof. $\mathbf{A}$ has an invertible $n \times n$ submatrix if $\text{rank}(\mathbf{A}) = n$. By the Rank-Nullity theorem, $\text{rank}(\mathbf{A}) \neq n$ if and only if the left-kernel of $\mathbf{A}$ is non-trivial, i.e., there exists $\mathbf{v} \in R_q^n \setminus \{\mathbf{0}\}$ such that $\mathbf{v}^\top \mathbf{A} = \mathbf{0}$, which is exactly $\neg E$. By the Chinese remainder theorem [Neu99], we have $R_q \cong (\mathbb{F}_{p_1^{r_1}})^{d/r_1} \times \dots \times (\mathbb{F}_{p_t^{r_t}})^{d/r_t}$. Given that $\mathbf{A} \leftarrow R_q^{n \times m}$, by restricting ourselves to $\mathbb{F}_{p_i^{r_i}}$, it follows that the probability that $\neg E$ happens in $\mathbb{F}_{p_i^{r_i}}$ is upper bounded by $p_i^{-r_i(m-n)}$ via union bound. By applying union bound over all the p_i , it follows that $\Pr[\neg E] \leq \sum_i \frac{d}{r_i} p_i^{-r_i(m-n)} \leq d \sum_i p_i^{-r_i(m-n)}$. $\square$

Lemma 2.5 (Regularity Lemma [LPR13], adapted). *Let n, m, q be lattice parameters with q satisfying Lemma 2.4. For any $\mathbf{A} \leftarrow R_q^{n \times m}$ and $\mathfrak{s} > 2d \cdot q^{n/m+2/(dm)}$, the following distributions are statistically close:*

$$\{(\mathbf{A}, \mathbf{Ax}) : x \leftarrow D_{R^m, \mathfrak{s}}\} \stackrel{\mathfrak{s}}{\approx} \{(\mathbf{A}, \mathbf{u}) : u \leftarrow R_q^n\}$$

Remark 2.1. Lyubashevsky, Peikert, and Regev [LPR13] considered matrices of the form $\mathbf{A} = [\mathbf{I}_n \mid \bar{\mathbf{A}}] \in R_q^{n \times m}$, where $\bar{\mathbf{A}} \leftarrow R_q^{n \times (m-n)}$. However, for any q satisfying Lemma 2.4, any matrix $\mathbf{B} \leftarrow R_q^{n \times m}$ can be rewritten as $[\mathbf{I}_n \mid \bar{\mathbf{B}}]$ with overwhelming probability.

Gadget Matrix [MP12]. Given $n, q \in \mathbb{N}$, $\mathfrak{b} \geq 2$, let $\tilde{q} = \lceil \log_{\mathfrak{b}} q \rceil$, $m = n\tilde{q}$ and

$$\mathbf{g}^\top = (1, \mathfrak{b}, \dots, \mathfrak{b}^{\tilde{q}-1}) \in R_q^{1 \times \tilde{q}}$$

The gadget matrix of base $\mathfrak{b}$ is defined by $\mathbf{G} = \mathbf{I}_n \otimes \mathbf{g}^\top \in R_q^{n \times m}$. In addition, there exists an efficient function $\mathbf{G}^{-1} : R_q^{n \times t} \rightarrow R_q^{m \times t}$ such that for any $\mathbf{A} \in R_q^{n \times t}$, we have $\mathbf{G} \cdot \mathbf{G}^{-1}(\mathbf{A}) = \mathbf{A}$. In brief, if $\mathfrak{b} = 2$, then $\mathbf{G}^{-1}$ decomposes each component of $\mathbf{A}$ into polynomials with binary coefficients (if $R_q = \mathbb{Z}_q$, this is simply binary decomposition).

Theorem 2.1 (Lattice Trapdoor [MP12], [GPV08], adapted). *Given a security parameter $\lambda \in \mathbb{N}$, $\mathfrak{b} \geq 2$, let $n = n(\lambda)$, $d = d(\lambda)$, $m = m(\lambda)$, $q = q(\lambda)$, with q satisfying Lemma 2.4, be lattice parameters, and let $m' = n \lceil \log_{\mathfrak{b}} q \rceil$, with $m > m'$. There exist ppt algorithms (TrapGen, SamplePre) with the following properties:*

- $\text{TrapGen}(1^n, 1^m, q) \rightarrow (\mathbf{A}, \mathbf{T}_\mathbf{A})$: Given n, m, q as inputs, the TrapGen algorithm outputs a matrix $\mathbf{A} \in R_q^{n \times m}$ with an associated trapdoor $\mathbf{T}_\mathbf{A} \in R^{(m-m') \times m'}$. Moreover, $\mathbf{A}$ is statistically close to uniform and $s_1(\mathbf{T}_\mathbf{A}) \leq \varsigma \sqrt{d}(\sqrt{m} + \sqrt{m'} + \log \sqrt{d})$, with $\varsigma > 2d \cdot q^{n/(m-m')+2/(d(m-m'))}$.

- $\text{SamplePre}(\mathbf{A}, \mathbf{T}_\mathbf{A}, \mathbf{u}, \mathfrak{s}) \rightarrow \mathbf{v}$: Given a matrix $\mathbf{A} \in R_q^{n \times m}$ with its trapdoor $\mathbf{T}_\mathbf{A} \in R^{(m-m') \times m'}$, a target $\mathbf{u} \in R_q^n$, and a sufficiently large Gaussian width parameter $\mathfrak{s} \geq s_1(\mathbf{T}_\mathbf{A}) \cdot \mathfrak{b} \cdot \omega\left(\sqrt{\log(nd)}\right)$, the SamplePre algorithm outputs a vector $\mathbf{v} \in R^m$ such that $\mathbf{A}\mathbf{v} = \mathbf{u}$. Also,

$$\{\mathbf{v} \leftarrow \text{SamplePre}(\mathbf{A}, \mathbf{T}_\mathbf{A}, \mathbf{u}, \mathfrak{s})\} \stackrel{\mathfrak{s}}{\approx} \{\mathbf{v} \leftarrow D_{\Lambda_\mathbf{u}^\perp(\mathbf{A}), \mathfrak{s}}\}.$$

Lemma 2.6 ([CHKP10], [ABB10], [BGG⁺14], adapted). For any $\lambda \in \mathbb{N}$ and $\mathfrak{b} \geq 2$, let $n = n(\lambda)$, $d = d(\lambda)$, $m = m(\lambda)$ and $q = q(\lambda)$, with q satisfying Lemma 2.4, be lattice parameters, and let $m' = n \lceil \log_{\mathfrak{b}} q \rceil$, with $m \geq m'$. Let $(\mathbf{A}, \mathbf{T}_\mathbf{A}) \leftarrow \text{TrapGen}(1^n, 1^m, q)$, $\mathbf{B} \in R_q^{n \times t}$ such that there exists a matrix $\mathbf{R} \in R^{(m+t) \times m'}$ and $[\mathbf{A} \mid \mathbf{B}] \cdot \mathbf{R} = \mathbf{G}$. Let $\mathbf{H} = [\mathbf{A} \mid \mathbf{B}]$. Then, for any $\mathbf{u} \in R_q^n$ and Gaussian width parameter $\mathfrak{s} \geq \max\{s_1(\mathbf{T}_\mathbf{A}), s_1(\mathbf{R})\} \cdot \mathfrak{b} \cdot \omega(\sqrt{\log(nd)})$, there exist ppt algorithms SampleLeft and SampleRight with the following properties:

- $\text{SampleLeft}(\mathbf{H}, \mathbf{T}_\mathbf{A}, \mathbf{u}, \mathfrak{s}) \rightarrow \mathbf{v}_1$: Given $\mathbf{H}, \mathbf{T}_\mathbf{A}$, the target vector $\mathbf{u}$, and the Gaussian width parameter $\mathfrak{s}$, the SampleLeft algorithm outputs a vector $\mathbf{v}_1 \in R^{m+t}$.
- $\text{SampleRight}(\mathbf{H}, \mathbf{R}, \mathbf{u}, \mathfrak{s}) \rightarrow \mathbf{v}_2$: Given $\mathbf{H}, \mathbf{R}$, the target vector $\mathbf{u}$, and the Gaussian width parameter $\mathfrak{s}$, the SampleRight algorithm outputs a vector $\mathbf{v}_2 \in R^{m+t}$.

In addition, $\mathbf{H}\mathbf{v}_1 = \mathbf{H}\mathbf{v}_2 = \mathbf{u}$, and the following distributions are statistically close:

$$\{\mathbf{v} \leftarrow D_{\Lambda_\mathbf{u}^\perp(\mathbf{H}), \mathfrak{s}}\} \stackrel{\mathfrak{s}}{\approx} \{\mathbf{v}_1 \leftarrow \text{SampleLeft}(\mathbf{H}, \mathbf{T}_\mathbf{A}, \mathbf{u}, \mathfrak{s})\} \stackrel{\mathfrak{s}}{\approx} \{\mathbf{v}_2 \leftarrow \text{SampleRight}(\mathbf{H}, \mathbf{R}, \mathbf{u}, \mathfrak{s})\}$$

Given a matrix $\mathbf{U}$, we use $\text{SampleLeft}(\mathbf{H}, \mathbf{T}_\mathbf{A}, \mathbf{U}, \mathfrak{s})$ (resp. $\text{SampleRight}(\mathbf{H}, \mathbf{R}, \mathbf{U}, \mathfrak{s})$) to denote the execution of SampleLeft (resp. SampleRight) on each column of $\mathbf{U}$.

Next, we present a theorem derived from a long line of work on lattice homomorphic operations over matrices [GSW13], [BGG⁺14], [GVW15], [BCTW16], [dCP23], [WW23b]. Our presentation is based on Wee and Wu's work [WW23b].

Theorem 2.2 (Homomorphic Operation on Matrices). For any $\lambda \in \mathbb{N}, \mathfrak{b} \geq 2$, let $n = n(\lambda)$, $d = d(\lambda)$, $q = q(\lambda)$ be lattice parameters. Let $m = n \lceil \log_{\mathfrak{b}} q \rceil$, $\mathbf{G} \in R_q^{n \times m}$ be the base $\mathfrak{b}$ gadget matrix, and $\mathcal{F} = \{\mathcal{F}_\lambda : \{0, 1\}^{\ell(\lambda)} \rightarrow \{0, 1\}\}$ be a family of Boolean circuits. There exist a pair of efficient (in the size of f) and deterministic algorithms ($\text{EvalF}, \text{EvalFx}$) with the following properties:

- $\text{EvalF}(f, \mathbf{A}) \rightarrow \mathbf{A}_f$: Given a Boolean circuit $f \in \mathcal{F}$ with input length ℓ and a matrix $\mathbf{A} \in R_q^{n \times m\ell}$, the EvalF algorithm outputs a matrix $\mathbf{A}_f \in R_q^{n \times m}$.
- $\text{EvalFx}(f, \mathbf{A}, \mathbf{x}) \rightarrow \mathbf{H}_{f, \mathbf{x}}$: Given a Boolean circuit $f \in \mathcal{F}$ with input length ℓ , a matrix $\mathbf{A} \in R_q^{n \times m\ell}$, and a vector $\mathbf{x} \in \{0, 1\}^\ell$ (with entries interpreted as constant polynomials), the EvalFx algorithm outputs a matrix $\mathbf{H}_{f, \mathbf{x}} \in R^{m\ell \times m}$.

In addition, for a given tuple $(f, \mathbf{A}, \mathbf{x}) \in \mathcal{F} \times R_q^{n \times m\ell} \times \{0, 1\}^\ell$, the outputs of EvalF and EvalFx satisfy the following conditions:

$$(\mathbf{A} - \mathbf{x}^\top \otimes \mathbf{G}) \cdot \mathbf{H}_{f, \mathbf{x}} = \mathbf{A}_f - f(\mathbf{x}) \cdot \mathbf{G} \quad (2.1)$$

$$\|\mathbf{H}_{f, \mathbf{x}}\|_\infty \leq \mathfrak{b} - 1 \quad (2.2)$$

Proof Sketch. Suppose $\lambda \in \mathbb{N}, \mathfrak{b} \geq 2$ and let $n = n(\lambda)$, $d = d(\lambda)$, $q = q(\lambda)$ and $m = n\tilde{q}$, where $\tilde{q} = \lceil \log_{\mathfrak{b}} q \rceil$. Without loss of generality, we assume that Boolean circuits in $\mathcal{F}$ are only composed of NAND gates, where each NAND gate is implemented using the algebraic relation $\text{NAND}(v_0, v_1) = 1 - v_0 v_1$. To

prove equations (2.1) and (2.2), we will consider the case of $f(\mu_0, \mu_1, \mu_2) = \text{NAND}(\text{NAND}(\mu_0, \mu_1), \mu_2)$, where μ_i is the i -th input wire, and this will give us enough intuition on how to handle any bounded depth Boolean circuits. Suppose $\mathbf{A} = [\mathbf{A}_0 \mid \mathbf{A}_1 \mid \mathbf{A}_2] \in R_q^{n \times 3m}$ and $\mathbf{x} \in \{0, 1\}^3$. From the work of Gorbunov, Vaikuntanathan, and Wichs [GVW15], EvalF and EvalFx for the NAND function on input (x_i, x_j) are implemented as follows:

$$\begin{aligned} \text{EvalF}(\text{NAND}, [\mathbf{A}_i \mid \mathbf{A}_j]) &= \mathbf{G} - \mathbf{A}_i \mathbf{G}^{-1}(\mathbf{A}_j) \quad , \text{ and} \\ \text{EvalFx}(\text{NAND}, [\mathbf{A}_i \mid \mathbf{A}_j], (x_i, x_j)) &= \begin{bmatrix} -\mathbf{G}^{-1}(\mathbf{A}_j) \\ -x_i \mathbf{I}_m \end{bmatrix} \end{aligned}$$

and it follows that

$$\begin{aligned} \left([\mathbf{A}_i \mid \mathbf{A}_j] - (x_i, x_j)^\top \otimes \mathbf{G} \right) \begin{bmatrix} -\mathbf{G}^{-1}(\mathbf{A}_j) \\ -x_i \mathbf{I}_m \end{bmatrix} &= -\mathbf{A}_i \mathbf{G}^{-1}(\mathbf{A}_j) + x_i x_j \mathbf{G} \\ &= \mathbf{G} - \mathbf{A}_i \mathbf{G}^{-1}(\mathbf{A}_j) - (1 - x_i x_j) \mathbf{G} \end{aligned}$$

By recursively applying these operations on the circuit f , the matrix $\mathbf{A}$, and the input $\mathbf{x}$, we have

$$\begin{aligned} \text{EvalF}(f, \mathbf{A}) &= \mathbf{G} - (\mathbf{G} - \mathbf{A}_0 \mathbf{G}^{-1}(\mathbf{A}_1)) \cdot \mathbf{G}^{-1}(\mathbf{A}_2) \\ &= \mathbf{G} - \mathbf{A}_2 + \mathbf{A}_0 \mathbf{G}^{-1}(\mathbf{A}_1) \cdot \mathbf{G}^{-1}(\mathbf{A}_2) = \mathbf{A}_f \quad , \text{ and} \end{aligned}$$

$$\text{EvalFx}(f, \mathbf{A}, \mathbf{x}) = \begin{bmatrix} \mathbf{G}^{-1}(\mathbf{A}_1) \cdot \mathbf{G}^{-1}(\mathbf{A}_2) \\ x_0 \mathbf{G}^{-1}(\mathbf{A}_2) \\ -(1 - x_0 x_1) \mathbf{I}_m \end{bmatrix} = \mathbf{H}_{f, \mathbf{x}} \quad (2.3)$$

It follows by inspection that $\mathbf{A}_f$ and $\mathbf{H}_{f, \mathbf{x}}$ satisfy equation (2.1). However, $\|\mathbf{H}_{f, \mathbf{x}}\|_\infty \leq (\mathfrak{b} - 1)^2 md$, where d is the degree of the $2d$ -th cyclotomic polynomial $X^d + 1$, and for any circuit $f' \in \mathcal{F}$ of depth $h > 2$ and input $\mathbf{x}' \in \{0, 1\}^\ell$, it follows that $\|\mathbf{H}_{f', \mathbf{x}'}\|_\infty \leq (\mathfrak{b} - 1)^h (md)^{h-1}$. For the matrix $\mathbf{H}_{f, \mathbf{x}}$ (cf. equation (2.3)), notice that the norm growth is generated by the product of *decomposed* matrices $\mathbf{G}^{-1}(\mathbf{A}_1) \cdot \mathbf{G}^{-1}(\mathbf{A}_2)$. To overcome this issue, De Castro and Peikert [dCP23] proposed to multiply the matrix $\mathbf{A}_1$ with the *decomposition* of the matrix $\mathbf{A}_2$ and then apply $\mathbf{G}^{-1}$ on the result, i.e., use $\mathbf{G}^{-1}(\mathbf{A}_1 \cdot \mathbf{G}^{-1}(\mathbf{A}_2))$ instead of $\mathbf{G}^{-1}(\mathbf{A}_1) \cdot \mathbf{G}^{-1}(\mathbf{A}_2)$. By applying their approach, we get

$$\mathbf{A}_f = \mathbf{G} - \mathbf{A}_2 + \mathbf{A}_0 \mathbf{G}^{-1}(\mathbf{A}_1 \cdot \mathbf{G}^{-1}(\mathbf{A}_2)), \text{ and } \mathbf{H}_{f, \mathbf{x}} = \begin{bmatrix} \mathbf{G}^{-1}(\mathbf{A}_1 \cdot \mathbf{G}^{-1}(\mathbf{A}_2)) \\ x_0 \mathbf{G}^{-1}(\mathbf{A}_2) \\ -(1 - x_0 x_1) \mathbf{I}_m \end{bmatrix} \quad (2.4)$$

By inspection, one can check that $\mathbf{A}_f$ and $\mathbf{H}_{f, \mathbf{x}}$ in equation (2.4) satisfy equation (2.1). In addition, $\|\mathbf{H}_{f, \mathbf{x}}\|_\infty \leq \mathfrak{b} - 1$ as required by equation (2.2). $\square$

Remark 2.2. Although De Castro and Peikert's approach [dCP23] allows the norm of EvalFx's output to be kept at 1 for $\mathfrak{b} = 2$, it requires the full description of f at initialization and cannot be *streamed*, unlike the approach used in prior works. Nevertheless, this will not be an issue for the class of Boolean functions considered in our work, as highlighted by Corollary 2.2.1.

Corollary 2.2.1 (Indicator Function [dCP23], adapted). *For any $\ell \in \mathbb{N}$, binary vectors $\mathbf{x}, \mathbf{y} \in \{0, 1\}^\ell$, and matrix $\mathbf{A} = [\mathbf{A}_0 \mid \dots \mid \mathbf{A}_{\ell-1}] \in R_q^{n \times m\ell}$, let*

$$\mathbb{1}_{\mathbf{x}}(\mathbf{y}) = \begin{cases} 1 & \text{if } \mathbf{x} = \mathbf{y} \\ 0 & \text{otherwise} \end{cases}, \mathbf{D}_i = \begin{cases} \mathbf{A}_i & \text{if } x_i = 1 \\ \mathbf{G} - \mathbf{A}_i & \text{if } x_i = 0 \end{cases}, \text{ and } \delta_{x,y} = \begin{cases} 1 & \text{if } x=y \\ 0 & \text{otherwise} \end{cases}$$

We have $\text{EvalF}(\mathbb{1}_{\mathbf{x}}, \mathbf{A}) = \mathbf{D}_0 \cdot \mathbf{G}^{-1}(\mathbf{D}_1 \cdot \mathbf{G}^{-1}(\dots \mathbf{G}^{-1}(\mathbf{D}_{\ell-1})))$ and

$$\text{EvalFx}(\mathbb{1}_{\mathbf{x}}, \mathbf{A}, \mathbf{y}) = \begin{bmatrix} (-1)^{(1-x_0)} \cdot \mathbf{G}^{-1}(\mathbf{D}_1 \cdot \mathbf{G}^{-1}(\mathbf{D}_2 \dots \mathbf{G}^{-1}(\mathbf{D}_{\ell-1}))) \\ (-1)^{(1-x_1)} \cdot \delta_{x_0, y_0} \cdot \mathbf{G}^{-1}(\mathbf{D}_2 \cdot \mathbf{G}^{-1}(\dots \mathbf{G}^{-1}(\mathbf{D}_{\ell-1}))) \\ \vdots \\ (-1)^{(1-x_{\ell-2})} \cdot \delta_{x_0, y_0} \delta_{x_1, y_1} \dots \delta_{x_{\ell-3}, y_{\ell-3}} \cdot \mathbf{G}^{-1}(\mathbf{D}_{\ell-1}) \\ (-1)^{(1-x_{\ell-1})} \cdot \delta_{x_0, y_0} \delta_{x_1, y_1} \dots \delta_{x_{\ell-3}, y_{\ell-3}} \delta_{x_{\ell-2}, y_{\ell-2}} \cdot \mathbf{I}_m \end{bmatrix}$$

2.2 Hardness Assumptions

Definition 2.8 (M-SIS $_{n,m,q,\beta}$ [LS15]). *For any $\lambda \in \mathbb{N}$, let $n = n(\lambda)$, $m = m(\lambda)$, $d = d(\lambda)$, $q = q(\lambda)$, and $\beta = \beta(\lambda) < q$ be lattice parameters. The Module Short Interger Solution (M-SIS) holds for λ if for any ppt adversary $\mathcal{A}$, there exists a negligible function negl such that*

$$\Pr \left[(\mathbf{A}\mathbf{v} = \mathbf{0}) \wedge (0 < \|\mathbf{v}\| \leq \beta) \mid \begin{array}{l} \mathbf{A} \leftarrow R_q^{n \times m} \\ \mathbf{v} \leftarrow \mathcal{A}(1^\lambda, \beta, q, \mathbf{A}) \end{array} \right] \leq \text{negl}(\lambda)$$

Definition 2.9 (M-LWE $_{n,m,q,\chi}$ [LS15]). *For any $\lambda \in \mathbb{N}$, let $n = n(\lambda)$, $m = m(\lambda)$, $d = d(\lambda)$, $q = q(\lambda)$ be lattice parameters and χ be a probability distribution over R . The (decision) Module Learning With Errors (M-LWE) holds for λ with respect to χ if*

$$(\mathbf{A}, \mathbf{A}\mathbf{s} + \mathbf{e}) \stackrel{c}{\approx} (\mathbf{A}, \mathbf{u})$$

where $\mathbf{A} \leftarrow R_q^{m \times n}$, $\mathbf{s} \leftarrow \chi^n$, $\mathbf{e} \leftarrow \chi^m$, $\mathbf{u} \leftarrow R_q^m$.

Definition 2.10 (ℓ -succinct M-SIS $_{n,m,q,\beta,\mathfrak{s}}$ [Wee24], adapted). *For any $\lambda \in \mathbb{N}$, let $n = n(\lambda)$, $m = m(\lambda)$, $d = d(\lambda)$, $q = q(\lambda)$, $\beta = \beta(\lambda)$ be lattice parameters and $\mathfrak{s} = \mathfrak{s}(\lambda)$ be a Gaussian width parameter. The ℓ -Succinct M-SIS assumption holds for λ with respect to $\mathfrak{s}$ if for any ppt adversary $\mathcal{A}$,*

$$\Pr \left[(\mathbf{A}\mathbf{v} = \mathbf{0}) \wedge (0 < \|\mathbf{v}\| \leq \beta) \mid \begin{array}{l} \mathbf{A} \leftarrow R_q^{n \times m}, \mathbf{W} \leftarrow R_q^{n\ell \times m} \\ \mathbf{T} \leftarrow D_{\Lambda_{[\mathbf{I}_\ell \otimes \mathbf{G}]}^\perp(\mathbf{I}_\ell \otimes \mathbf{A} | \mathbf{W}), \mathfrak{s}} \\ \mathbf{v} \leftarrow \mathcal{A}(1^\lambda, \beta, q, \mathbf{A}, \mathbf{W}, \mathbf{T}) \end{array} \right] \leq \text{negl}(\lambda)$$

Remark 2.3. Let $(\mathbf{A}, \mathbf{W}, \mathbf{T})$ be an ℓ -succinct M-SIS instance, and parse $[\mathbf{T}_0^\top \mid \mathbf{T}_1^\top]^\top := \mathbf{T}$ with $\mathbf{T}_0 \in R^{\ell m \times \ell m}$ and $\mathbf{T}_1 \in R^{m \times \ell m}$. Wee [Wee24] noted that for any $\mathbf{x} \in R^\ell$, the matrix $\mathbf{x}^\top \otimes \mathbf{G} \in R^{n \times \ell m}$ can be compressed into a matrix $[\mathbf{A} \mid (\mathbf{x}^\top \otimes \mathbf{I}_n)\mathbf{W}] \in R^{n \times 2m}$ as follows:

$$\begin{aligned} \mathbf{x}^\top \otimes \mathbf{G} &= (\mathbf{x}^\top \otimes \mathbf{I}_n)(\mathbf{I}_\ell \otimes \mathbf{G}) \\ &= [(\mathbf{x}^\top \otimes \mathbf{I}_n)(\mathbf{I}_\ell \otimes \mathbf{A}) \mid (\mathbf{x}^\top \otimes \mathbf{I}_n)\mathbf{W}] \cdot \mathbf{T} \\ &= [\mathbf{A} \mid (\mathbf{x}^\top \otimes \mathbf{I}_n)\mathbf{W}] \cdot \underbrace{\left[\left((\mathbf{x}^\top \otimes \mathbf{I}_m)\mathbf{T}_0 \right)^\top \mid \mathbf{T}_1^\top \right]^\top}_{\mathbf{T}_{\mathbf{x}}} \end{aligned}$$

Therefore, any relation of the form $(\mathbf{x}^\top \otimes \mathbf{G}) \cdot \mathbf{P} = \mathbf{Q}$ for some matrices $\mathbf{P} \in R^{\ell m \times t}$ and $\mathbf{Q} \in R_q^{n \times t}$ can be rewritten as $[\mathbf{A} \mid (\mathbf{x}^\top \otimes \mathbf{I}_n)\mathbf{W}] \cdot \mathbf{T}_{\mathbf{x}} \cdot \mathbf{P} = \mathbf{Q}$.

3 Cryptographic Accumulator

We recall the definition of a positive dynamic accumulator.

Definition 3.1 (Positive Dynamic Accumulator [KL24]). *A positive dynamic accumulator for a domain $\mathcal{M}$ is a tuple of ppt algorithms $\Pi_{\text{PACC}} = (\text{Gen}, \text{Add}, \text{Delete}, \text{MemWitCreate}, \text{MemWitUp}, \text{MemVer})$ with the following syntax:*

- $\text{Gen}(1^\lambda, \text{aux}) \rightarrow (\text{pp}, \text{sk}, \mathcal{A}_0, \text{st})$: Given $\lambda \in \mathbb{N}$ in unary and some auxiliary information aux , the algorithm Gen outputs the public parameters pp , the secret key sk , an initial accumulator value $\mathcal{A}_0$, and an initial state st .
- $\text{Add}(\text{pp}, \text{sk}, \text{st}_t, \mathcal{A}_t, x) \rightarrow (\mathcal{A}_{t+1}, w_{x,t+1}, \text{upmsg}_{t+1}, \text{st}_{t+1})$: Given pp and sk , a state st_t , an accumulator's value $\mathcal{A}_t$, and a value $x \in \mathcal{M}$, if x is present in the accumulator, the Add algorithm aborts and returns $\perp$. Otherwise, the Add algorithm adds x to the accumulator and outputs a new accumulator's value $\mathcal{A}_{t+1}$, a witness $w_{x,t+1}$ certifying that x is in the accumulator, update information upmsg_{t+1} that can be used to update the membership witnesses of other elements, and an updated state st_{t+1} .
- $\text{Delete}(\text{pp}, \text{sk}, \text{st}_t, \mathcal{A}_t, x, w_{x,t}) \rightarrow (\mathcal{A}_{t+1}, \text{upmsg}_{t+1}, \text{st}_{t+1})$: Given pp and sk , an accumulator's value $\mathcal{A}_t$, a value $x \in \mathcal{M}$ and its membership witness $w_{x,t}$, the Delete algorithm removes x from the accumulator and outputs a new accumulator's value $\mathcal{A}_{t+1}$, update information upmsg_{t+1} , and an updated state st_{t+1} .
- $\text{MemWitCreate}(\text{pp}, \text{sk}, \text{st}_t, \mathcal{A}_t, x) \rightarrow w_{x,t}$: Given pp and sk , a state st_t , an accumulator value $\mathcal{A}_t$, and a value $x \in \mathcal{M}$ that was accumulated, the MemWitCreate algorithm outputs a membership witness for x at epoch t .
- $\text{MemWitUp}(\text{pp}, x, w_{x,t}, \text{upmsg}_{t+1}) \rightarrow w_{x,t+1}$: Given pp , an element $x \in \mathcal{M}$ with its witness $w_{x,t}$, and an update message upmsg_{t+1} , the MemWitUp algorithm outputs an updated membership witness $w_{x,t+1}$ for x .
- $\text{MemVer}(\text{pp}, \mathcal{A}_t, x, w_{x,t}) \rightarrow 1/0$: Given pp , an accumulator value $\mathcal{A}_t$, an element $x \in \mathcal{M}$ and its membership witness $w_{x,t}$, the deterministic MemVer algorithm outputs 1 if $w_{x,t}$ is a witness for x 's membership in the set represented by $\mathcal{A}_t$, or 0 otherwise.

In addition, Π_{PACC} should satisfy the following properties:

- **Compactness**: For any $\lambda \in \mathbb{N}$, $t = \text{poly}(\lambda)$, and $x \in \mathcal{M}$, $|\mathcal{A}_t| = \text{poly}(\lambda)$ and $|w_{x,t}| = \text{poly}(\lambda, |x|)$.
- **Correctness (Informal)**: For any $\lambda \in \mathbb{N}$, $t = \text{poly}(\lambda)$, accumulator value $\mathcal{A}_t$ with associated public parameter pp , element $x \in \mathcal{S} \subseteq \mathcal{M}$ with up-to-date membership witness $w_{x,t}$, where $\mathcal{S}$ is the set of elements accumulated in $\mathcal{A}_t$,

$$\Pr[\text{MemVer}(\text{pp}, \mathcal{A}_t, x, w_{x,t}) = 1] \geq 1 - \text{negl}(\lambda)$$

We refer the reader to the work of Kemmoe and Lysyanskaya [KL24] for a formal definition.

Definition 3.2 (Communication efficiency). *We say that a positive dynamic accumulator Π_{PACC} is communication efficient if the size of upmsg grows linearly with the number of deletions that have occurred since the last time the accumulator was modified.*

Note that a communication efficient positive dynamic accumulator satisfies free addition.

For our security definition, we consider the notion of (adaptive) collision-freeness under honestly generated accumulator values [BP97, CL02a].

Definition 3.3 (Collision-Freeness). A positive dynamic accumulator Π_{PAcc} for a domain $\mathcal{M}$ is said to be (adaptively) collision-free if for any ppt adversary $\mathcal{A}$ with oracle access to $\mathcal{O}_{\Pi_{\text{PAcc}}}$, there exists a negligible function negl such that for any $\lambda \in \mathbb{N}$,

$$\Pr \left[\begin{array}{l} (\text{pp}, \text{sk}, \mathcal{A}_0, \text{st}) \leftarrow \text{Gen}(1^\lambda, \text{aux}) \\ (x^*, w_{x^*, t}) \leftarrow \mathcal{A}^{\mathcal{O}_{\Pi_{\text{PAcc}}}}(\text{st}_{\mathcal{A}}, \text{aux}, \text{pp}, \mathcal{A}_0) : \\ \text{MemVer}(\text{pp}, \mathcal{A}_t, x^*, w_{x^*, t}) = 1 \wedge \\ x^* \in \mathcal{M} \setminus \mathcal{S} \end{array} \right] \leq \text{negl}(\lambda)$$

where $\mathcal{A}_t$ is output by $\mathcal{O}_{\Pi_{\text{PAcc}}}$ and $\mathcal{S}$ is the set represented by $\mathcal{A}_t$. $\mathcal{O}_{\Pi_{\text{PAcc}}}$ is initialized with the secret key sk and the initial accumulator state st . It answers to Add, Del, membership witness creation requests by executing the Add, Delete, MemWitCreate algorithms and updates the accumulator state st and accumulator value $\mathcal{A}_t$ after each request.

Remark 3.1 (Selective Collision-Freeness). We consider a relaxed version of Definition 3.3 where an adversary first specifies an element $x^* \in \mathcal{M}$ on which it will output a forged membership witness before receiving the public parameters and the initial accumulator value. However, in Section 4.2, we show how to apply Baldimsti et al.’s compiler [BCD⁺17, Theorem 1] to convert any selectively collision-free positive dynamic accumulator into an adaptively collision-free accumulator.

We also consider a relaxed version of a positive dynamic accumulator that we call positive replacement-free accumulator (PRFA), which is a dynamic accumulator in which an element x can only be added once, i.e., after adding x to the accumulator, it cannot be re-added once it is removed. It should be noted that a PRFA can be functionally upgraded into a positive dynamic accumulator by appending a unique tag to each element that is added to the accumulator.

Definition 3.4 (Positive Replacement-free Accumulator). A positive replacement-free accumulator (PRFA) for a domain $\mathcal{M}$ is a tuple of ppt algorithms $\Pi_{\text{PRFA}} = (\text{Gen}, \text{Add}, \text{Delete}, \text{MemWitCreate}, \text{MemWitUp}, \text{MemVer})$ that satisfies the same syntax, compactness, correctness, and (selective) collision-freeness of a positive dynamic accumulator with the following exceptions:

- $\text{Add}(\text{pp}, \text{sk}, \text{st}_t, \mathcal{A}_t, x)$ aborts and outputs $\perp$ if x was ever added to the accumulator.
- During the (selective) collision-freeness game, $\mathcal{O}_{\Pi_{\text{PAcc}}}$ aborts if an adversary $\mathcal{A}$ tries to re-add an element that was previously added and then deleted.

Remark 3.2. The secret parameter sk can be “null”, i.e., $\text{sk} = \perp$. If that is the case, we say that the accumulator has a transparent Gen algorithm. Nevertheless, for the application that we are considering (certificate/credential revocation), the accumulator will be managed by a trusted authority, the issuer, that will be in charge of adding and removing elements from the accumulator. Therefore, whether the accumulator requires a non-null sk will not change the overall behavior of our system.

For our setting, we will focus on positive dynamic accumulators that can be composed with a commitment scheme $\Pi_{\text{Com}} = (\text{Setup}, \text{Commit}, \text{Verify})$ such that we have an *efficient* NIZK argument of knowledge for the relation

$$\mathcal{R}_{\text{acc}}(\text{crs}_{\text{Com}}, \text{pp}) := \left\{ (\mathcal{A}, c; x, w_x, o) : \begin{array}{l} \Pi_{\text{Com}}.\text{Verify}(\text{crs}_{\text{Com}}, c, x, o) = 1 \wedge \\ \Pi_{\text{PAcc}}.\text{MemVer}(\text{pp}, \mathcal{A}, x, w_x) = 1 \end{array} \right\} \quad (3.1)$$

where $(\mathcal{A}, c)$ represents the statement and (x, w_x, o) represents the witness. To this end, we propose the following definition:

Definition 3.5 (Commit-and-Prove Argument for a Positive Dynamic Accumulator). *Given a commitment scheme Π_{Com} and a positive dynamic accumulator Π_{PAcc} , both defined over a domain $\mathcal{M}$, let $\mathcal{R}_{\text{acc}}$ be an NP relation as defined with respect to Π_{Com} and Π_{PAcc} in (3.1) above. Let $\mathfrak{c} : \mathbb{N} \rightarrow [0, 1]$ be a function. A commit-and-prove argument for the relation $\mathcal{R}_{\text{acc}}$ with completeness $\mathfrak{c}$ is a tuple of ppt algorithms $\Pi_{\text{NIZK}, \mathcal{R}_{\text{acc}}} = (\text{Setup}, \text{Prove}, \text{Verify})$ with the following syntax:*

- $\text{Setup}(1^\lambda) \rightarrow \text{crs}$: *Given a security parameter $\lambda \in \mathbb{N}$ in unary, the Setup algorithm produces a common reference string crs' . Then, it computes $\text{crs}_{\text{Com}} \leftarrow \Pi_{\text{Com}}.\text{Setup}(1^\lambda)$. Finally, it sets $\text{crs} = \text{crs}' \parallel \text{crs}_{\text{Com}}$ and outputs crs .*
- $\text{Prove}(\text{crs}, \mathcal{A}, x, c, w_x, o) \rightarrow \pi$: *Given the crs , an accumulator value $\mathcal{A}$, a value $x \in \mathcal{M}$, a commitment c associated with x , a membership witness w_x , and an opening o for c , the Prove algorithm outputs a proof π certifying that $(\mathcal{A}, c; x, w_x, o) \in \mathcal{R}_{\text{acc}}$.*
- $\text{Verify}(\text{crs}, \mathcal{A}, c, \pi) \rightarrow 1/0$: *Given the crs , an accumulator value $\mathcal{A}$, a commitment c to some value $x \in \mathcal{M}$, and a proof π , the deterministic Verify algorithm outputs 1 if π is valid. Else, it outputs 0.*

In addition, $\Pi_{\text{NIZK}, \mathcal{R}_{\text{acc}}}$ should satisfy the following properties:

- **Completeness:** For any $\lambda \in \mathbb{N}$, $(\mathcal{A}, c; x, w_x, o) \in \mathcal{R}_{\text{acc}}$,

$$\Pr \left[\text{Verify}(\text{crs}, \mathcal{A}, c, \pi) = 1 \mid \begin{array}{l} \text{crs} \leftarrow \text{Setup}(1^\lambda) \\ \pi \leftarrow \text{Prove}(\text{crs}, \mathcal{A}, x, c, w_x, o) \end{array} \right] \geq \mathfrak{c}(\lambda)$$

- **Zero-Knowledge:** For any ppt distinguisher D , there exists a ppt simulator $\text{Sim} = (\text{Sim}_1, \text{Sim}_2)$ and a negligible function negl such that

$$\left| \Pr \left[\text{zkGame}_D(1^\lambda, 0) = 1 \right] - \Pr \left[\text{zkGame}_D(1^\lambda, 1) = 1 \right] \right| \leq \text{negl}(\lambda)$$

where $\text{zkGame}_D(1^\lambda, b)$ is defined in Fig. 1.

- **Knowledge soundness:** For every ppt adversary $\mathcal{A}$, there exist a ppt extractor Ext with rewindable black-box access to $\mathcal{A}$ and a negligible function negl such that for any $\lambda \in \mathbb{N}$,

$$\Pr \left[\begin{array}{l} (\text{pp}, \text{sk}, \mathcal{A}_0, \text{st}) \leftarrow \Pi_{\text{PAcc}}.\text{Gen}(1^\lambda, \text{aux}) \\ (c, \pi, t) \leftarrow \mathcal{A}^{\mathcal{O}_{\Pi_{\text{PAcc}}}}(1^\lambda, \text{crs}, \text{pp}, \text{aux}, \mathcal{A}_0) \\ (x, o, w_x) \leftarrow \text{Ext}^{\text{BB}(\mathcal{A})}(1^\lambda, \text{crs}, \mathcal{A}_t, c, \pi) \\ \text{Verify}(\text{crs}, \mathcal{A}_t, c, \pi) = 1 : \\ \Pi_{\text{Com}}.\text{Verify}(\text{crs}|_{\Pi_{\text{Com}}}, c, x, o) = 1 \wedge \\ \Pi_{\text{PAcc}}.\text{MemVer}(\text{pp}, \mathcal{A}_t, x, w_x) = 1 \end{array} \right] \geq 1 - \text{negl}(\lambda)$$

Revocation in Credential Systems. Camenisch et al. [CKL⁺15] described how revocation can be added to any privacy-preserving credential system through the use of a revocation authority that works in tandem with credential issuers to issue revocation handles to users. Anytime a user presents their credentials through a privacy-preserving protocol, it also shows that its revocation handle is still valid, either by showing that it is still part of a whitelist or that it has not been added to a blacklist. Baldimsti et al. [BCD⁺17] showed how a positive cryptographic accumulator that supports a commit-and-prove argument (cf. Definition 3.5) can be used to instantiate the revocation protocol of Camenisch et al. In a nutshell, when a user receives a

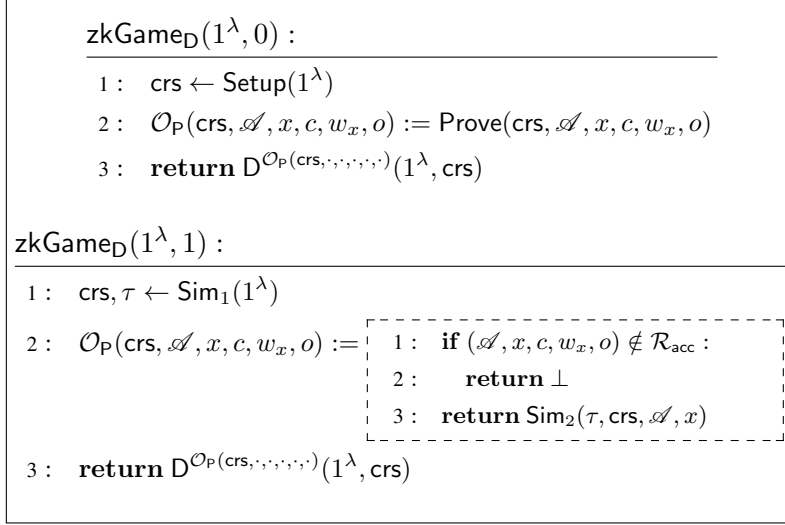


Figure 1: ZK Game

credential from an issuer, it also receives a revocation handle rt and a membership witness w_{rt} attesting that rt was accumulated in a positive accumulator that represents a whitelist. Now, when the user presents its credential, it also issues an argument claiming that its revocation handle rt is still part of the accumulator by using the accumulator's commit-and-prove argument.

4 Positive Replacement-free accumulator from Lattices

In this section, we present a PRFA based on the M-SIS assumption and a secure PRF. We note that the PRF is only needed to reduce the space-complexity of our construction and can be removed in favor of using more space. One of the advantages of our construction is its communication complexity: membership witnesses only need to be updated after a deletion. This construction is syntactically similar to communication-efficient positive dynamic accumulators based on the strong RSA assumption [BCD⁺17], [KL24] and q -strong Diffie-Hellman assumption [KB21], [JML24], in that they start with an unforgeable digital signature scheme and provide a mechanism to revoke issued signatures. Our construction (Construction 4.1) is obtained by combining the selectively secure digital signature of Agrawal, Boneh, and Boyen [ABB10] and Theorem 2.2.

Construction 4.1. Let $(\text{TrapGen}, \text{SampleLeft})$ be algorithms defined in Theorem 2.1 and Lemma 2.6, and $(\text{Eval}, \text{EvalFx})$ be algorithms defined in Theorem 2.2. For any $\lambda \in \mathbb{N}$, let $n = n(\lambda)$, $m = m(\lambda)$, $d = d(\lambda)$, and $q = q(\lambda)$ be lattice parameters. Let $b \geq 2$ and $m' = n \lceil \log_b q \rceil$. Let $s = s(\lambda)$ be a Gaussian width parameter, $\gamma = \gamma(\lambda)$ be a bound on the number of deletions, $\ell = \ell(\lambda)$ be an input length, $\tau = \tau(\lambda)$ be a bound on the randomness used by SampleLeft , and $\beta = \beta(\lambda)$ be a norm bound. Let $F : \{0, 1\}^\lambda \times \{0, 1\}^\ell \times R_q^n \rightarrow \{0, 1\}^\tau$ be a PRF. We construct a PRFA for the domain $\mathcal{M} = \{0, 1\}^\ell$ as follows:

- $\text{Gen}(1^\lambda, \text{aux} = (1^n, 1^m, q))$: Given λ and $\text{aux} = (1^n, 1^m, q)$, do the following:
 1. Compute $(\mathbf{A}, \mathbf{T}_\mathbf{A}) \leftarrow \text{TrapGen}(1^n, 1^m, q)$.
 2. Sample $\mathbf{B} \leftarrow \$ R_q^{n \times \ell m'}$, $(\mathbf{c}_0, \mathbf{u}) \leftarrow \$ R_q^{2n}$, $k \leftarrow \$ \{0, 1\}^\lambda$.

3. Initialize an empty dictionary $DX : \mathcal{M} \rightarrow \{\text{Add}, \text{Del}, \perp\} \times \mathbb{N}$ and a counter $\text{ctr} := 0$.
 4. Set $\text{pp} := (\mathbf{A}, \mathbf{B}, \mathbf{u})$, $\text{sk} := \mathbf{T}_{\mathbf{A}}$, $\mathcal{A}_0 := \mathbf{c}_0$, and $\text{st} := (k, \text{ctr}, \mathbf{c}_0, DX)$.
 5. Return $(\text{pp}, \text{sk}, \mathcal{A}_0, \text{st})$.
- $\text{Add}(\text{pp}, \text{sk}, \text{st}, \mathcal{A}, \mathbf{x})$: Given $(\mathbf{A}, \mathbf{B}, \mathbf{u}) := \text{pp}$, $\mathbf{T}_{\mathbf{A}} := \text{sk}$, $(k, \text{ctr}, \mathbf{c}_0, DX) := \text{st}$, $\mathbf{c} := \mathcal{A}$, $\mathbf{x} \in \mathcal{M}$, do the following:
 1. Parse $(\text{Op}, i) := DX[\mathbf{x}]$, and if $\text{Op} \neq \perp$, abort. ▶ $\mathbf{x}$ was already added or deleted
 2. Compute $r \leftarrow F_k(\mathbf{x}, \mathbf{c})$.
 3. Compute $\mathbf{s}_{\mathbf{x}} \leftarrow \text{SampleLeft}([\mathbf{A} \mid \mathbf{B} - \mathbf{x}^\top \otimes \mathbf{G}], \mathbf{T}_{\mathbf{A}}, \mathbf{c}, \mathfrak{s}; r)$.
 4. Set $DX[\mathbf{x}] := (\text{Add}, \text{ctr})$, $w_{\mathbf{x}} := \mathbf{s}_{\mathbf{x}}$, $\text{upmsg} = (\text{Add}, \perp)$.
 5. Set $\text{st}' := (k, \text{ctr}, \mathbf{c}_0, DX)$.
 6. Return $(\mathcal{A}, w_{\mathbf{x}}, \text{upmsg}, \text{st}')$.
 - $\text{Delete}(\text{pp}, \text{sk}, \text{st}, \mathcal{A}, \mathbf{x}, w_{\mathbf{x}})$: Given $(\mathbf{A}, \mathbf{B}, \mathbf{u}) := \text{pp}$, $\mathbf{T}_{\mathbf{A}} := \text{sk}$, $(k, \text{ctr}, \mathbf{c}_0, DX) := \text{st}$, $\mathbf{c} := \mathcal{A}$, $\mathbf{x} \in \mathcal{M}$, and $\mathbf{S}_{\mathbf{x}} := w_{\mathbf{x}}$, do the following:
 1. If $\text{ctr} > \gamma$, abort. ▶ We have exceeded the allowed number of deletions
 2. Parse $(\text{Op}, i) := DX[\mathbf{x}]$, and if $\text{Op} \neq \text{Add}$, abort. ▶ $\mathbf{x}$ was already deleted or never added
 3. Compute $\mathbf{B}_{\perp_{\mathbf{x}}} \leftarrow \text{EvalF}(\mathbb{1}_{\mathbf{x}}, \mathbf{B})$, and then $\mathbf{c}' \leftarrow \mathbf{c} + \mathbf{B}_{\perp_{\mathbf{x}}} \cdot \mathbf{G}^{-1}(\mathbf{u})$.
 4. Set $\text{ctr}' \leftarrow \text{ctr} + 1$ and $\mathcal{A}' := \mathbf{c}'$.
 5. Set $DX[\mathbf{x}] := (\text{Del}, \text{ctr})$, $\text{upmsg} := (\text{Del}, \mathbf{x})$, and $\text{st}' := (k, \text{ctr}', \mathbf{c}_0, DX)$.
 6. Return $(\mathcal{A}', \text{upmsg}, \text{st}')$.
 - $\text{MemWitUp}(\text{pp}, \mathbf{x}, w_{\mathbf{x}}, \text{upmsg})$: Given $(\mathbf{A}, \mathbf{B}, \mathbf{u}) := \text{pp}$, $\mathbf{x} \in \mathcal{M}$, $\mathbf{S}_{\mathbf{x}} := w_{\mathbf{x}}$, $(\text{Op}, \mathbf{y}) := \text{upmsg}$, do the following:
 1. If $\text{Op} = \text{Add}$: do nothing.
 2. Else, if $\text{Op} = \text{Del}$:
 - Compute $\mathbf{H}_{\perp_{\mathbf{y}}, \mathbf{x}} \leftarrow \text{EvalFx}(\mathbb{1}_{\mathbf{y}}, \mathbf{B}, \mathbf{x})$, and then $\mathbf{s}'_{\mathbf{x}} \leftarrow \mathbf{s}_{\mathbf{x}} + [\mathbf{0}_{m' \times m} \mid \mathbf{H}_{\perp_{\mathbf{y}}, \mathbf{x}}^\top]^\top \cdot \mathbf{G}^{-1}(\mathbf{u})$.
 - Return $\mathbf{s}'_{\mathbf{x}}$ as $w'_{\mathbf{x}}$.
 - $\text{MemVer}(\text{pp}, \mathcal{A}, \mathbf{x}, w_{\mathbf{x}})$: Given $(\mathbf{A}, \mathbf{B}, \mathbf{u}) := \text{pp}$, $\mathbf{c} := \mathcal{A}$, $\mathbf{x} \in \mathcal{M}$, and $\mathbf{s}_{\mathbf{x}} := w_{\mathbf{x}}$, do the following:
 1. If $[\mathbf{A} \mid \mathbf{B} - \mathbf{x}^\top \otimes \mathbf{G}] \cdot \mathbf{s}_{\mathbf{x}} = \mathbf{c}$ and $\|\mathbf{s}_{\mathbf{x}}\| \leq \beta$, return 1.
 2. Otherwise, return 0.
 - $\text{MemWitCreate}(\text{pp}, \text{sk}, \text{st}, \mathcal{A}, \mathbf{x})$: Given $(\mathbf{A}, \mathbf{B}, \mathbf{u}) := \text{pp}$, $\mathbf{T}_{\mathbf{A}} := \text{sk}$, $(k, \text{ctr}, \mathbf{c}_0, DX) := \text{st}$, $\mathbf{c} := \mathcal{A}$, $\mathbf{x} \in \mathcal{M}$, do the following:
 1. Parse $(\text{Op}, i) := DX[\mathbf{x}]$, and if $\text{Op} \neq \text{Add}$, abort. ▶ $\mathbf{x}$ is not in the accumulator
 2. Let $Z_0 = \{\mathbf{y} \in \mathcal{M} : (\text{Op}, j) := DX[\mathbf{y}], \text{Op} = \text{Del}, 0 < j \leq i\}$. ▶ The set of elements deleted before $\mathbf{x}$ was added
 3. Let $Z_1 = \{\mathbf{y} \in \mathcal{M} : (\text{Op}, j) := DX[\mathbf{y}], \text{Op} = \text{Del}, i < j \leq \text{ctr}\}$. ▶ The set of elements deleted after $\mathbf{x}$ was added
 4. Compute $\mathbf{c}_i \leftarrow \mathbf{c}_0 + \sum_{\mathbf{y} \in Z_0} \text{EvalF}(\mathbb{1}_{\mathbf{y}}, \mathbf{B}) \cdot \mathbf{G}^{-1}(\mathbf{u})$. ▶ We recover the accumulator's value from when $\mathbf{x}$ was added
 5. Compute $r \leftarrow F_k(\mathbf{x}, \mathbf{c}_i)$.
 6. Compute $\mathbf{s}_{\mathbf{x}} \leftarrow \text{SampleLeft}([\mathbf{A} \mid \mathbf{B} - \mathbf{x}^\top \otimes \mathbf{G}], \mathbf{T}_{\mathbf{A}}, \mathbf{c}_i, \mathfrak{s}_1; r)$.
 7. For each $\mathbf{y} \in Z_1$, do:
 - Compute $\mathbf{H}_{\perp_{\mathbf{y}}, \mathbf{x}} \leftarrow \text{EvalFx}(\mathbb{1}_{\mathbf{y}}, \mathbf{B}, \mathbf{x})$.

$$- \mathbf{s}_x \leftarrow \mathbf{s}_x + [\mathbf{0}_{m' \times m} \mid \mathbf{H}_{\mathbb{1}_y, x}^\top]^\top \cdot \mathbf{G}^{-1}(\mathbf{u}).$$

8. Return $\mathbf{s}_x$.

Theorem 4.1 (Compactness). *For any $\lambda \in \mathbb{N}$, $b \geq 2$, let $n = \text{poly}(\lambda)$, $q = \text{poly}(\lambda)$, $m' = n \lceil \log_b q \rceil$, $m = O(m')$, $d = \text{poly}(\lambda)$ be lattice parameters, and $\beta = \text{poly}(\lambda)$ be a bound on the membership witness norm. Construction 4.1 is a compact (cf. Definitions 3.1 and 3.4) PRFA.*

Proof. In Construction 4.1, the accumulator value $\mathcal{A}$ is a vector $\mathbf{c} \in R_q^n$. Hence, $|\mathcal{A}| = dn \log q = \text{poly}(\lambda)$. In addition, for any $\mathbf{x} \in \mathcal{M} = \{0, 1\}^\ell$, $w_x = \mathbf{s}_x \in R^{m+\ell m'}$ such that $\|\mathbf{s}_x\| \leq \beta$. Therefore, $|w_x| = d(m + \ell m') \log(2\beta + 1) = \text{poly}(\lambda, \ell)$. This completes the proof. $\square$

Lemma 4.1. *For any $b \geq 2$, $\gamma \geq 0$, let $\tilde{\mathbf{H}} \in R^{\ell m' \times m'}$ be a matrix obtain by summing the outputs of γ evaluations of EvalFx (defined in Corollary 2.2.1). Then, $s_1(\tilde{\mathbf{H}}) \leq \gamma \sqrt{((b-1)dm' \sqrt{\ell-1})^2 + 1}$.*

Proof. Following Corollary 2.2.1, we can parse $\tilde{\mathbf{H}}$ as $[\hat{\mathbf{H}}^\top \mid \xi \mathbf{I}_{m'}]^\top$ where $\hat{\mathbf{H}} \in R^{(\ell-1)m' \times m'}$, $\|\hat{\mathbf{H}}\|_\infty \leq (b-1)\gamma$, and $\xi \leq \gamma$. Therefore,

$$s_1(\tilde{\mathbf{H}}) \leq \sqrt{s_1(\hat{\mathbf{H}})^2 + s_1(\xi \mathbf{I}_{m'})^2} \leq \sqrt{((b-1)\gamma dm' \sqrt{\ell-1})^2 + \gamma^2} \quad (\text{following Lemma 2.3})$$

$\square$

Theorem 4.2 (Correctness). *For $\lambda \in \mathbb{N}$ and $b \geq 2$, let $n = \text{poly}(\lambda)$, $q = \text{poly}(\lambda)$, $m' = n \lceil \log_b q \rceil$, $m = O(m')$, $d = \text{poly}(\lambda)$ be lattice parameters. Let $\gamma = \text{poly}(\lambda)$ be a bound on the number of deletion, $\ell = \text{poly}(\lambda)$ be the input-length, and $T = \gamma(b-1)\sqrt{m'd} \sqrt{1 + ((b-1)dm' \sqrt{\ell-1})^2}$. Let $\mathfrak{s} \geq b\varsigma \sqrt{d}(\sqrt{m} + \sqrt{m'} + \log \sqrt{d}) \cdot \omega(\sqrt{\log(nd)})$ be a Gaussian width parameter, with $\varsigma > 2d \cdot q^{n/m+2/(dm)}$. Let $\beta = \mathfrak{s} \sqrt{d(m + \ell m')} + T$ be a norm bound on membership witnesses, and $\mathfrak{r} = \mathfrak{r}(\lambda)$ be a bound on the randomness used by SampleLeft. Then, Construction 4.1 is correct with probability $1 - \text{negl}(\lambda)$.*

Proof. Let $\text{pp} = (\mathbf{A}, \mathbf{B}, \mathbf{u})$, $\text{sk} = \mathbf{T}_\mathbf{A}$, $\mathcal{A}_0 = \mathbf{c}_0$, and $\text{st} = (k, \text{ctr}, \mathbf{c}_0, \text{DX})$ be the output of Gen. Suppose $\mathbf{x}_1 \in \mathcal{M}$ is added to the accumulator and let $w_{\mathbf{x}_1} = \mathbf{s}_{\mathbf{x}_1}$ be the witness returned by the Add algorithm. Given that the randomness $r \in \{0, 1\}^{\mathfrak{r}}$ used by SampleLeft is the output of a secure PRF and $\mathfrak{s}$ is large enough, Lemma 2.6 still applies and $[\mathbf{A} \mid \mathbf{B} - \mathbf{x}_1^\top \otimes \mathbf{G}] \cdot \mathbf{s}_{\mathbf{x}_1} = \mathbf{c}_0$. In addition, by Lemma 2.2, $\|\mathbf{s}_{\mathbf{x}_1}\| \leq \mathfrak{s} \sqrt{d(m + \ell m')} < \beta$ with $1 - \text{negl}(\lambda)$ probability.

Now, suppose $\mathbf{x}_2 \in \mathcal{M} \setminus \{\mathbf{x}_1\}$ is added and then remove by executing the Delete algorithm. The new accumulator value is $\mathcal{A}_1 = \mathcal{A}_0 + \mathbf{B}_{\mathbb{1}_{\mathbf{x}_2}} \cdot \mathbf{G}^{-1}(\mathbf{u})$. After executing MemWitUp on $w_{\mathbf{x}_1}$, we have $w'_{\mathbf{x}_1} = w_{\mathbf{x}_1} + [\mathbf{0}_{m' \times m} \mid \mathbf{H}_{\mathbb{1}_{\mathbf{x}_2}, \mathbf{x}_1}^\top]^\top \cdot \mathbf{G}^{-1}(\mathbf{u})$ and

$$\begin{aligned} [\mathbf{A} \mid \mathbf{B} - \mathbf{x}_1^\top \otimes \mathbf{G}] \cdot w'_{\mathbf{x}_1} &= \mathbf{c}_0 + (\mathbf{B} - \mathbf{x}_1^\top \otimes \mathbf{G}) \cdot \mathbf{H}_{\mathbb{1}_{\mathbf{x}_2}, \mathbf{x}_1} \cdot \mathbf{G}^{-1}(\mathbf{u}) \\ &= \mathbf{c}_0 + \mathbf{B}_{\mathbb{1}_{\mathbf{x}_2}} \cdot \mathbf{G}^{-1}(\mathbf{u}) = \mathcal{A}_1 \quad (\text{By Corollary 2.2.1}) \end{aligned}$$

Also, $\|[\mathbf{0}_{m' \times m} \mid \mathbf{H}_{\mathbb{1}_{\mathbf{x}_2}, \mathbf{x}_1}^\top]^\top \cdot \mathbf{G}^{-1}(\mathbf{u})\| \leq s_1(\mathbf{H}_{\mathbb{1}_{\mathbf{x}_2}, \mathbf{x}_1}) \|\mathbf{G}^{-1}(\mathbf{u})\| \leq T$ following Lemma 4.1 and given the fact that we perform at most γ deletions. Therefore, $\|w'_{\mathbf{x}_1}\| \leq \|w_{\mathbf{x}_1}\| + T \leq \beta$.

Finally, the correctness of MemWitCreate follows by inspection using the arguments developed above and the fact that the randomness of SampleLeft is fixed by the PRF. $\square$

Remark 4.1. The runtime of MemWitCreate is linear in the number of items that were deleted. However, for an accumulated element $\mathbf{x}$, one may think that we could circumvent that overhead by sampling a “fresh” low-norm vector $\tilde{\mathbf{s}}_x$ with respect to the most current value of the accumulator, but proceeding this way could

reveal a trapdoor for the matrix $[\mathbf{A} \mid \mathbf{B} - \mathbf{x}^\top \otimes \mathbf{G}]$. More specifically, suppose the accumulator value is $\mathcal{A} = \mathbf{c}$ and we have a low-norm vector $\mathbf{s}_\mathbf{x}$ such that $[\mathbf{A} \mid \mathbf{B} - \mathbf{x}^\top \otimes \mathbf{G}] \cdot \mathbf{s}_\mathbf{x} = \mathbf{c}$. In addition, suppose we perform $\kappa < \gamma$ deletions, updating $\mathbf{c}$ to $\tilde{\mathbf{c}}$ in the process. Then, we sample a “fresh” low-norm vector $\tilde{\mathbf{s}}_\mathbf{x}$ such that $[\mathbf{A} \mid \mathbf{B} - \mathbf{x}^\top \otimes \mathbf{G}] \cdot \tilde{\mathbf{s}}_\mathbf{x} = \tilde{\mathbf{c}}$. Using Corollary 2.2.1, we can compute $\mathbf{s}'_\mathbf{x} = \mathbf{s}_\mathbf{x} + \sum_{i=0}^{\kappa-1} \mathbf{h}'_i$ such that $[\mathbf{A} \mid \mathbf{B} - \mathbf{x}^\top \otimes \mathbf{G}] \cdot \mathbf{s}'_\mathbf{x} = \tilde{\mathbf{c}}$, where $\mathbf{h}'_i \in R^{m+\ell m'}$ is a vector associated to the i -th deleted element. Therefore, $[\mathbf{A} \mid \mathbf{B} - \mathbf{x}^\top \otimes \mathbf{G}] \cdot [\tilde{\mathbf{s}}_\mathbf{x} - \mathbf{s}'_\mathbf{x}] = \mathbf{0}$, and $\tilde{\mathbf{s}}_\mathbf{x} - \mathbf{s}'_\mathbf{x} \neq \mathbf{0}$ with overwhelming probability. Hence, by giving out several “fresh” membership witnesses for $\mathbf{x}$, we could compute a trapdoor for the matrix $[\mathbf{A} \mid \mathbf{B} - \mathbf{x}^\top \otimes \mathbf{G}]$ [GPV08, Proposition 5.8]. For this reason, we opted to ensure that MemWitCreate outputs the *same* membership witness that was returned during the execution of Add by using a PRF to *fix* the randomness. Note that the PRF can be dropped if one is willing to store each membership witness that is returned after an Add query. In Remark 4.3, we describe how Construction 4.1 can be adapted to make MemWitCreate run in constant time while avoiding the potential leakage of a trapdoor. In [dCP23] and [dCP23]+[WW23a], MemWitCreate is linear in the number of items in the accumulator and cannot be made constant time.

Increasing the number of deletions. A trivial way to increase the bound on the number of deletions in Construction 4.1 is to make the parameters superpolynomial in λ . Alternatively, using a Merkle tree (or a similar data structure), we can increase the bound on the number of deletions in our PRFA from γ to $k \cdot \gamma$, with $k \in \mathbb{N}$, without degrading the quality of the parameters. In brief, instead of sampling one vector $\mathbf{c}$ uniformly at random, the accumulator manager will sample k vectors $[\mathbf{c}_1, \dots, \mathbf{c}_k]$ and then hash them into a Merkle tree by putting $\mathbf{c}_i$ in leaf position i in the tree. Now, the root of the Merkle tree is our new accumulator value. Observe that the resulting accumulator is still communication efficient: we can add an element $\mathbf{x}$ to the new accumulator without changing its value by just adding $\mathbf{x}$ to the sub-accumulator associated with one of the leaf matrices $\mathbf{c}_i$. In addition, we can delete an element $\mathbf{y}$ from this new accumulator by removing $\mathbf{y}$ from its corresponding leaf associated with the vector $\mathbf{c}_j$; as a result, the leaf in position j will need to be replaced with the vector $\mathbf{c}'_j$ and the Merkle tree will need to be recomputed to reflect that; updating the Merkle tree requires $\log k$ computation and communication. Although this transformation allows us to increase the number of deletions we can perform, it also increases the size of a membership witness to incorporate the authenticating path in the Merkle tree. It also complicates using the resulting accumulator as part of the commit-and-proof paradigm.

Reducing the size of witnesses. A membership witness $w_\mathbf{x} = \mathbf{s}_\mathbf{x} \in R^{m+\ell m'}$ for an element $\mathbf{x} \in \mathcal{M}$ requires $\tilde{O}(d(m + \ell m'))$ bits of storage (cf. Theorem 4.1). However, by relying on the ℓ -succinct M-SIS assumption instead of the M-SIS assumption, this can be reduced to $\tilde{O}(d(m + m'))$ in exchange for increasing the size of the public parameters by a factor of $O(\ell)$ and the norm bound on the witnesses by a factor of $O(\ell^2 \varsigma d(m + m'))$, where $\varsigma > 2d \cdot q^{n/m+2/(dm)}$. To achieve this reduction in the size of witnesses, Construction 4.1 can be adapted as follows:

- During the execution of Gen, instead of sampling $\mathbf{B} \leftarrow R_q^{n \times \ell m'}$, sample $\mathbf{W} \leftarrow R_q^{n \ell \times m'}$, $\mathbf{B}_1 \leftarrow R_q^{n \times m'}$. Then, using $\mathbf{T}_\mathbf{A}$, sample $\mathbf{T} \in R_q^{(\ell m + m') \times \ell m'}$ such that $[\mathbf{I}_\ell \otimes \mathbf{A} \mid \mathbf{W}] \cdot \mathbf{T} = \mathbf{I}_\ell \otimes \mathbf{G}$. Parse $[\mathbf{T}_0^\top \mid \mathbf{T}_1^\top]^\top := \mathbf{T}$ with $\mathbf{T}_0 \in R_q^{\ell m \times \ell m'}$ and $\mathbf{T}_1 \in R_q^{m' \times \ell m'}$. Set $\text{pp} := (\mathbf{A}, \mathbf{B}_1, \mathbf{W}, \mathbf{T}_0, \mathbf{T}_1, \mathbf{u})$.
- During the execution of Add on an input $\mathbf{x}$, sample $\tilde{\mathbf{s}}_\mathbf{x} \in R^{m+\ell m'}$ such that $[\mathbf{A} \mid -\mathbf{B}_1 \mathbf{T}_1 - \mathbf{x}^\top \otimes \mathbf{G}] \cdot \tilde{\mathbf{s}}_\mathbf{x} = \mathbf{c}$. Then, compute $\mathbf{s}_\mathbf{x} \leftarrow \begin{bmatrix} \mathbf{I}_m & -(\mathbf{x}^\top \otimes \mathbf{I}_m) \mathbf{T}_0 \\ \mathbf{0}_{m' \times m} & -\mathbf{T}_1 \end{bmatrix} \cdot \tilde{\mathbf{s}}_\mathbf{x}$ and return $w_\mathbf{x} := \mathbf{s}_\mathbf{x}$.

- During the execution of MemWitUp on an input $\mathbf{x}$ after an element $\mathbf{y}$ was removed, compute

$$w'_x \leftarrow w_x + \begin{bmatrix} -(\mathbf{x}^\top \otimes \mathbf{I}_m) \mathbf{T}_0 \\ -\mathbf{T}_1 \end{bmatrix} \cdot \mathbf{H}_{\mathbb{1}_y, \mathbf{x}} \cdot \mathbf{G}^{-1}(\mathbf{u}).$$

- During the execution of Verify on an input $(\mathcal{A}, \mathbf{x}, w_x)$, parse $\mathbf{c} := \mathcal{A}$, $\mathbf{s}_x := w_x$. Then, check if $\|\mathbf{s}_x\|$ is small and if $[\mathbf{A} \mid \mathbf{B}_1 + (\mathbf{x}^\top \otimes \mathbf{I}_n) \mathbf{W}] \cdot \mathbf{s}_x = \mathbf{c}$.

Following Remark 2.3, it is clear that by switching to the ℓ -succinct M-SIS assumption, Construction 4.1 is still correct and its compactness is improved.

4.1 Security Analysis of Construction 4.1

Remark 4.2 (Replacement-free condition). It is crucial that elements previously added to the accumulator described in Construction 4.1 are not re-added once they are removed. Otherwise, for an element $\mathbf{x} \in \mathcal{M}$, we can recover a trapdoor for the matrix $\mathbf{A}_x = [\mathbf{A} \mid \mathbf{B} - \mathbf{x}^\top \otimes \mathbf{G}]$ as follows: Suppose at epoch t , we add $\mathbf{x}$, i.e., we sample a low-norm vector $\mathbf{s}_x \in R^{m+\ell m'}$ such that $\mathbf{A}_x \cdot \mathbf{s}_x = \mathbf{c}_t$. Now, at epoch $t+1$, we delete $\mathbf{x}$ by computing $\mathbf{c}_{t+1} \leftarrow \mathbf{c}_t + \mathbf{B}_{\mathbb{1}_x} \cdot \mathbf{G}^{-1}(\mathbf{u})$, and then, we re-add $\mathbf{x}$ by sampling a new low-norm vector $\tilde{\mathbf{s}}_x \in R^{m+\ell m'}$ such that $\mathbf{A}_x \cdot \tilde{\mathbf{s}}_x = \mathbf{c}_{t+1}$ (epochs are counted according to the number of deletions). Let $\mathbf{h}_{\mathbb{1}_x, \mathbf{x}} \leftarrow \text{EvalFx}(\mathbb{1}_x, \mathbf{B}, \mathbf{x}) \cdot \mathbf{G}^{-1}(\mathbf{u})$. It follows that $\mathbf{A}_x \cdot (\mathbf{s}_x + [\mathbf{0}_m \mid \mathbf{h}_{\mathbb{1}_x, \mathbf{x}}]^\top - \tilde{\mathbf{s}}_x) = -\mathbf{u}$. If at epoch $t+2$ we do the same thing, i.e., remove $\mathbf{x}$ and then add it again by sampling a new low-norm vector $\hat{\mathbf{s}}_x$, it follows that $\mathbf{A}_x \cdot (\tilde{\mathbf{s}}_x + [\mathbf{0}_m \mid \mathbf{h}_{\mathbb{1}_x, \mathbf{x}}]^\top - \hat{\mathbf{s}}_x) = -\mathbf{u}$. Therefore, by linearly combining $\mathbf{s}_x, \tilde{\mathbf{s}}_x, \hat{\mathbf{s}}_x$, and $\mathbf{h}_{\mathbb{1}_x, \mathbf{x}}$, we can recover a point in the right-kernel of the matrix $\mathbf{A}_x$. Hence, if the replacement-free condition is violated polynomially many-time, we can recover a trapdoor for $\mathbf{A}_x$ that can allow to forge membership witnesses [GPV08, Proposition 5.8]. Nevertheless, this can be fixed by either appending a unique tag to $\mathbf{x}$ everytime it is added to the accumulator or by randomizing the matrix $\mathbf{A}_x$ (cf. Remark 4.3).

Theorem 4.3 (Selective Collision-Freeness). *For any $\lambda \in \mathbb{N}$, let $q = q(\lambda)$ be an integer that satisfies Lemma 2.4, $n = n(\lambda)$, $m' = n \lceil \log_b q \rceil$, $m = O(m')$ be lattice parameters. Let $b \geq 2$, $\alpha \geq 2$, $\varepsilon = \text{negl}(\lambda)$, and $\varsigma > 2d \cdot q^{n/m+2/(dm)}$. Let $\theta_1 = \sqrt{2d/3}(\sqrt{m} + \sqrt{m'} + \log \sqrt{d})$ and $\theta_2 = \sqrt{2d/3}(\sqrt{m} + \sqrt{\ell m'} + \log \sqrt{d})$. Let $\ell = \ell(\lambda)$ be the input length, $\gamma = \gamma(\lambda)$ be a bound on the number of deletions, $T = \gamma(b-1)\sqrt{m'd}\sqrt{((b-1)dm'\sqrt{\ell-1})^2 + 1}$ be an upper bound on ℓ_2 -norm of the vector used to update witnesses, $\mathfrak{s} = \max(T, b\sqrt{1+\theta_1^2} \cdot \log \sqrt{nd}, b\varsigma\sqrt{d}(\sqrt{m} + \sqrt{m'} + \log \sqrt{d}) \log \sqrt{nd})$ be a Gaussian width parameter, $\beta_0 = \mathfrak{s}\sqrt{d(m+\ell m')} + T$ be a norm bound on the witnesses, and $\mathfrak{r} = \mathfrak{r}(\lambda)$ be a bound on the randomness used by SampleLeft and SampleRight. Let*

$$\beta_1 = (\mathfrak{s} + 1)\sqrt{md} + \theta_2(\mathfrak{s}\sqrt{d\ell m'} + T) + \beta_0\sqrt{1 + \theta_2^2}$$

Suppose F is a secure PRF, the assumptions $\text{M-LWE}_{m-n, m, q, U(S_1^\infty)}$, $\text{M-LWE}_{m-n, m, q, U(\mathcal{B}_1)}$, and $\text{M-SIS}_{n, m, q, \beta_1}$ hold. Then Construction 4.1 is a selectively collision-free PRFA (cf. Definitions 3.3 and 3.4).

We postpone the proof of Theorem 4.3 to Appendix C. Note that the M-LWE assumption can be removed from Theorem 4.3 in exchange for larger parameters and by relying on the Regularity Lemma [LPR13].

Remark 4.3. As we have previously shown (cf. Remark 4.2), if the replacement-free condition on Construction 4.1 is violated, then one can recover a trapdoor that can allow them to forge membership witnesses for their element. This issue stems from the fact that by issuing a “fresh” witness $\mathbf{s}_x$ for an element $\mathbf{x} \in \{0, 1\}^\ell$, we reveal additional information with respect to the matrix $[\mathbf{A} \mid \mathbf{B} - \mathbf{x}^\top \otimes \mathbf{G}]$ that can help recover a trapdoor. To avoid this issue, we can *randomize* the matrix $[\mathbf{A} \mid \mathbf{B} - \mathbf{x}^\top \otimes \mathbf{G}]$ everytime we issue a witness for $\mathbf{x}$. More specifically, we borrow the idea of Libert et al. [LLM⁺16] and

publish additional matrices $\mathbf{A}_0, \dots, \mathbf{A}_{\ell_2} \in R_q^{n \times m'}$ as part of the public parameters. Now, during the execution of Add or MemWitCreate, we sample a fresh randomness $\mathbf{r} \in \{0, 1\}^{\ell_2}$, compute the matrix $\mathbf{A}_{\mathbf{r}} = \mathbf{A}_0 + \sum_{i=1}^{\ell_2} \mathbf{r}[i-1] \mathbf{A}_i$ and sample $\mathbf{s}_{\mathbf{x}}$ such that $[\mathbf{A} \mid \mathbf{B} - \mathbf{x}^\top \otimes \mathbf{G} \mid \mathbf{A}_{\mathbf{r}}] \cdot \mathbf{s}_{\mathbf{x}} = \mathbf{c}$. Now, the membership witness is $w_{\mathbf{x}} = (\mathbf{r}, \mathbf{s}_{\mathbf{x}})$. It is easy to see that this transformation still allows us to update $\mathbf{s}_{\mathbf{x}}$ and $\mathbf{c}$ such that we can perform deletion. In addition, since for two pairs $(\mathbf{r}', \mathbf{s}'_{\mathbf{x}})$ and $(\tilde{\mathbf{r}}, \tilde{\mathbf{s}}_{\mathbf{x}})$, $\mathbf{r}' \neq \tilde{\mathbf{r}}$ with overwhelming probability, it will be nearly impossible to obtain a trapdoor to forge membership witnesses for $\mathbf{x}$ if the replacement-free condition is not respected. Furthermore, this modification allows us to circumvent the issue with MemWitCreate presented in Remark 4.1 and enables us to make it *stateless* and constant-time. However, besides increasing the size of our public parameters and witnesses, this transformation also increases the complexity of the relation that needs to be satisfied for the commit-and-prove argument, since one needs to prove knowledge of a low-norm $\mathbf{r}$ such that $\hat{\mathbf{A}}$ in $[\mathbf{A} \mid \mathbf{B} - \mathbf{x}^\top \otimes \mathbf{G} \mid \hat{\mathbf{A}}]$ satisfies $\hat{\mathbf{A}} = \mathbf{A}_0 + \sum_{i=1}^{\ell_2} \mathbf{r}[i-1] \mathbf{A}_i$. Although this modification prevents *leaking* trapdoors, we are only able to prove it secure in the replacement-free regime. We provide a full description of this transformation in Appendix E.

4.2 Obtaining an Adaptively Secure Positive Dynamic Accumulator

Baldimsti et al. [BCD⁺17, Theorem 1] showed how any non-adaptively secure positive dynamic accumulator Π_{PACC} can be turned into an adaptively secure Π'_{PACC} by combining it with an adaptively secure digital signature Σ . In brief, to add an element x , we sample an element r_x uniformly at random from the domain of Π_{PACC} and generate its membership witness $\tilde{w}_{r_x}$ with respect to Π_{PACC} . Then, we bind x to r_x by using Σ to generate a signature σ_x on the pair (x, r_x) . Finally, the membership witness for x with respect to Π'_{PACC} is the triple $w_x = (r_x, \sigma_x, \tilde{w}_{r_x})$. To delete x from Π'_{PACC} , we only need to remove r_x from Π_{PACC} . Clearly, Π'_{PACC} is adaptively secure since forging a membership witness for an element x' results in either breaking the security of Σ or forging a membership witness with respect to Π_{PACC} for a random element. In addition, if Π_{PACC} is communication efficient, then Π'_{PACC} is also communication efficient.

In Construction 4.2, we show how Baldimsti et al.'s compiler can be used to transform any selectively secure PRFA into an adaptively secure positive dynamic accumulator. For any element x , to ensure that the replacement free condition of the underlying PRFA is not violated, we sample a fresh random element r_x everytime x is added to the accumulator.

Construction 4.2. Let $\Sigma = (\text{Gen}, \text{Sign}, \text{Verify})$ be an EUF-CMA signature with message space $\mathcal{V} \times \mathcal{M}$, $\Pi_{\text{PRFA}} = (\text{Gen}, \text{Add}, \text{Delete}, \text{MemWitUp}, \text{MemVer}, \text{MemWitCreate})$ be a selective collision-free communication efficient PRFA with input domain $\mathcal{M}$, and, for any $\lambda \in \mathbb{N}$, let $F : \{0, 1\}^\lambda \times \mathcal{V} \times \mathbb{N} \rightarrow \mathcal{M}$ be a PRF. We construct an adaptively collision-free positive dynamic accumulator for the domain $\mathcal{V}$ as follows:

- $\text{Gen}(1^\lambda, \text{aux}_{\Pi_{\text{PRFA}}})$: Given λ and $\text{aux}_{\Pi_{\text{PRFA}}}$, the auxilliary information for the underlying PRFA, do the following:
 1. Execute $(\text{pk}_\Sigma, \text{sk}_\Sigma) \leftarrow \Sigma.\text{Gen}(1^\lambda)$.
 2. Execute $(\tilde{\text{pp}}, \tilde{\text{sk}}, \tilde{\mathcal{A}}_0, \tilde{\text{st}}) \leftarrow \Pi_{\text{PRFA}}.\text{Gen}(1^\lambda, \text{aux}_{\Pi_{\text{PRFA}}})$.
 3. Sample $k \leftarrow \mathbb{S} \{0, 1\}^\lambda$.
 4. Initialize an empty dictionary $\text{DX} : \mathcal{V} \rightarrow \{\text{Add}, \text{Del}, \perp\} \times \mathbb{N}$.
 5. Set $\text{pp} := (\text{pk}_\Sigma, \tilde{\text{pp}})$, $\text{sk} := (\text{sk}_\Sigma, \tilde{\text{sk}})$, $\mathcal{A}_0 := \tilde{\mathcal{A}}_0$, and $\text{st} = (k, \tilde{\text{st}}, \text{DX})$.
 6. Return $(\text{pp}, \text{sk}, \mathcal{A}_0, \text{st})$.
- $\text{Add}(\text{pp}, \text{sk}, \text{st}, \mathcal{A}, x)$: Given $(\text{pk}_\Sigma, \tilde{\text{pp}}) := \text{pp}$, $(\text{sk}_\Sigma, \tilde{\text{sk}}) := \text{sk}$, $(k, \tilde{\text{st}}, \text{DX}) := \text{st}$, $\tilde{\mathcal{A}} := \mathcal{A}$, and $x \in \mathcal{V}$, do the following:

1. Parse $(\text{Op}, i) := \text{DX}[x]$, and if $\text{Op} = \text{Add}$, abort. $\blacktriangleright$ *x was already added*
 2. Compute $r_x \leftarrow F_k(x, i)$, then $\text{ans} \leftarrow \Pi_{\text{PRFA}}.\text{Add}(\tilde{\text{pp}}, \tilde{\text{sk}}, \tilde{\text{st}}, \tilde{\mathcal{A}}, r_x)$.
 3. While $\text{ans} = \perp$, do : $\blacktriangleright$ *if ans = $\perp$, then r_x was added to Π_{PRFA} at one point*
 - Set $i := i + 1$.
 - Compute $r_x \leftarrow F_k(x, i)$, then $\text{ans} \leftarrow \Pi_{\text{PRFA}}.\text{Add}(\tilde{\text{pp}}, \tilde{\text{sk}}, \tilde{\text{st}}, \tilde{\mathcal{A}}, r_x)$
 4. Parse $(\tilde{\mathcal{A}}', \tilde{w}_{r_x}, \tilde{\text{st}}') := \text{ans}$. $\blacktriangleright$ *we omitted upmsg since Π_{PRFA} is communication efficient*
 5. Compute $\sigma_x \leftarrow \Sigma.\text{Sign}(\text{sk}_\Sigma, (x, r_x))$.
 6. Set $\text{DX}[x] := (\text{Add}, i)$ and $\text{st}' := (k, \tilde{\text{st}}', \text{DX})$.
 7. Set $w_x := (r_x, \sigma_x, \tilde{w}_{r_x})$, and $\mathcal{A}' := \tilde{\mathcal{A}}'$.
 8. Return $(\mathcal{A}', w_x, \text{st}')$.
- $\text{Delete}(\text{pp}, \text{sk}, \text{st}, \mathcal{A}, x, w_x)$: Given $(\text{pk}_\Sigma, \tilde{\text{pp}}) := \text{pp}$, $(\text{sk}_\Sigma, \tilde{\text{sk}}) := \text{sk}$, $(k, \tilde{\text{st}}, \text{DX}) := \text{st}$, $\tilde{\mathcal{A}} := \mathcal{A}$, $x \in \mathcal{V}$, and $(r_x, \sigma_x, \tilde{w}_{r_x}) := w_x$, do the following:
 1. Parse $(\text{Op}, i) := \text{DX}[x]$, and if $\text{Op} \neq \text{Add}$, abort. $\blacktriangleright$ *x is not in the accumulator*
 2. Compute $r'_x \leftarrow F_k(x, i)$, and if $r'_x \neq r_x$, abort.
 3. Compute $(\tilde{\mathcal{A}}', \text{upmsg}, \tilde{\text{st}}') \leftarrow \Pi_{\text{PRFA}}.\text{Delete}(\tilde{\text{pp}}, \tilde{\text{sk}}, \tilde{\text{st}}', r_x, \tilde{w}_{r_x})$.
 4. Set $\text{DX}[x] := (\text{Del}, i + 1)$, $\mathcal{A}' := \tilde{\mathcal{A}}'$, and $\text{st}' := (k, \tilde{\text{st}}', \text{DX})$.
 5. Return $(\mathcal{A}', \text{upmsg}, \text{st}')$.
 - $\text{MemWitUp}(\text{pp}, x, w_x, \text{upmsg})$: Given $(\text{pk}_\Sigma, \tilde{\text{pp}}) := \text{pp}$, $x \in \mathcal{V}$, $(r_x, \sigma_x, \tilde{w}_{r_x}) := w_x$ and $(\text{Op}, y) := \text{upmsg}$, do the following:
 1. If $\text{Op} = \text{Add}$: do nothing.
 2. Else, if $\text{Op} = \text{Del}$:
 - Compute $\tilde{w}'_{r_x} \leftarrow \Pi_{\text{PRFA}}.\text{MemWitUp}(\tilde{\text{pp}}, r_x, \tilde{w}_{r_x}, \text{upmsg})$.
 - Return $(r_x, \sigma_x, \tilde{w}'_{r_x})$.
 - $\text{MemVer}(\text{pp}, \mathcal{A}, x, w_x)$: Given $(\text{pk}_\Sigma, \tilde{\text{pp}}) := \text{pp}$, $\tilde{\mathcal{A}} := \mathcal{A}$, $x \in \mathcal{V}$, and $(r_x, \sigma_x, \tilde{w}_{r_x}) := w_x$, return $(\Sigma.\text{Verify}(\text{pp}_\Sigma, \sigma_x, (x, r_x)) \wedge \Pi_{\text{PRFA}}.\text{MemVer}(\tilde{\text{pp}}, \tilde{\mathcal{A}}, r_x, \tilde{w}_{r_x}))$.
 - $\text{MemWitCreate}(\text{pp}, \text{sk}, \text{st}, \mathcal{A}, x)$: Given $(\text{pk}_\Sigma, \tilde{\text{pp}}) := \text{pp}$, $(\text{sk}_\Sigma, \tilde{\text{sk}}) := \text{sk}$, $(k, \tilde{\text{st}}, \text{DX}) := \text{st}$, $\tilde{\mathcal{A}} := \mathcal{A}$, and $x \in \mathcal{V}$, do the following:
 1. Parse $(\text{Op}, i) := \text{DX}[x]$, and if $\text{Op} \neq \text{Add}$, abort. $\blacktriangleright$ *x is not in the accumulator*
 2. Compute $r_x \leftarrow F_k(x, i)$.
 3. Compute $\tilde{w}_{r_x} \leftarrow \Pi_{\text{PRFA}}.\text{MemWitCreate}(\tilde{\text{pp}}, \tilde{\text{sk}}, \tilde{\text{st}}, \tilde{\mathcal{A}}, r_x)$.
 4. Compute $\sigma_x \leftarrow \Sigma.\text{Sign}(\text{sk}_\Sigma, (x, r_x))$.
 5. Return $(r_x, \sigma_x, \tilde{w}_{r_x})$.

Assuming that $|\mathcal{V}| = O(|\mathcal{M}|)$ and the size of a signature, σ_m , on a message $m \in \mathcal{V} \times \mathcal{M}$ produced by Σ is upper bounded by $|\sigma_m| = \text{poly}(\lambda, |m|)$, it follows that correctness and compactness of Construction 4.2 directly follows from the correctness of both Σ and Π_{PRFA} and the compactness of Π_{PRFA} . In addition, given that Π_{PRFA} is communication efficient, it follows that Construction 4.2 is also communication efficient.

Theorem 4.4. *Suppose F is a secure PRF, Σ is an EUF-CMA signature, and Π_{PRFA} is a selectively collision-free PRFA. Then, Construction 4.2 is an adaptively collision-free positive dynamic accumulator (cf. Definition 3.3).*

Proof. Let $\mathcal{A}$ be a ppt adversary that outputs a forgery (x^*, w_{x^*}) against Construction 4.2, with $w_{x^*} = (r_{x^*}, \sigma_{x^*}, \tilde{w}_{r_{x^*}})$, after q addition queries. Let $Q_{\text{Add}} = \{(x, r) \in \mathcal{V} \times \mathcal{M} : x \leftarrow \mathcal{A}, r \leftarrow \$\mathcal{M}\}$, with $\#Q_{\text{Add}} = q$, be a set generated by $\mathcal{A}$'s add queries. By construction, the probability that there exist $(x_i, r_i), (x_j, r_j) \in Q_{\text{Add}}$ such that $r_i = r_j$ and $i \neq j$ is exactly zero. Hence, the replacement-free condition of Π_{PRFA} is never breached. Let E_1 be the event that $\mathcal{A}$ outputs its forgery with $(x^*, r_{x^*}) \notin Q_{\text{Add}}$ and E_2 be the event that $\mathcal{A}$ outputs its forgery with $(x^*, r_{x^*}) \in Q_{\text{Add}}$. By a simple hybrid argument, it follows that $\Pr[\mathcal{A} \text{ wins}] \leq \Pr[E_1] + \Pr[E_2] + \epsilon_{\text{PRF}}$, where ϵ_{PRF} is the advantage that a ppt distinguisher D distinguishes F from a truly random function, which is negligible by assumption. For event E_1 , given that $(x^*, r_{x^*}) \notin Q_{\text{Add}}$, it follows that $\mathcal{A}$ provided a valid signature σ_{x^*} on the pair (x^*, r_{x^*}) that was not signed by the challenger. However, by the security of Σ it follows that $\Pr[E_1] = \text{negl}(\lambda)$. Now, for event E_2 , since $(x^*, r_{x^*}) \in Q_{\text{Add}}$ it follows that $\mathcal{A}$ provided a valid membership witness $\tilde{w}_{r_{x^*}}$ for a random element r_{x^*} that was added once and then deleted. However, by the selective collision-freeness of Π_{PRFA} , it follows that $\Pr[E_2] = \frac{1}{q} \text{negl}(\lambda)$. $\square$

5 Applications to Anonymous Credentials with Revocation

In this section, we show how our positive replacement-free accumulator can be applied in the existing lattice-based anonymous credentials (and most likely the future works following the similar template) to support revocation.

5.1 Signatures with efficient protocols from lattices

The state-of-the-art lattice-based anonymous credential systems [BLNS23b], [JRS23], [LCL⁺23], [AGJ⁺24] and blind signatures [dPK22], [BLNS23a] follow the widely-used “signatures with efficient protocols” template by Camenisch and Lysyanskaya [CL02b]. In the lattice setting, the underlying signature schemes stem from the GPV framework [GPV08]. Namely, the secret key is a trapdoor $\mathbf{T}_U$ for a public matrix $\mathbf{U} \in R_q^{\hat{n} \times \hat{m}}$, which allows the signer to sample short preimages of $\mathbf{U}$ to any targets. Then, to sign a short message $\mathbf{m}$, the signer samples a short preimage $\mathbf{s} \in R_q^{\hat{m}}$ which satisfies

$$\mathbf{U}\mathbf{s} = \mathbf{D}\mathbf{m}, \quad (5.1)$$

where $\mathbf{D}$ is a public uniformly random matrix². One can think of the right-hand side $\mathbf{D}\mathbf{m}$ as an Ajtai commitment [Ajt96] of the message $\mathbf{m}$. We sketch how this signature scheme is used in the anonymous credential setting below. In the following, we assume for simplicity that the attributes and the randomness polynomials come from the set of ring elements with binary coefficients $\mathcal{B}_1$. Extending to larger coefficients or different distributions (e.g. discrete Gaussian) is straightforward.

Credential generation. The user $\mathcal{U}$ possesses a vector of attributes $\text{attr} \in \mathcal{B}_1^w$ of length w . Then, $\mathcal{U}$ commits to the attributes attr by generating a randomness vector $\mathbf{r} \leftarrow \$\mathcal{B}_1^l$ and computing the Ajtai commitment [Ajt96]:

$$\mathbf{t} := \mathbf{B}_1 \text{attr} + \mathbf{B}_2 \mathbf{r} \quad (5.2)$$

where $\mathbf{B}_1, \mathbf{B}_2$ are public, uniformly random matrices. Next, the user generates a non-interactive zero-knowledge proof of knowledge (NIZK) π_0 of $(\text{attr}, \mathbf{r})$ such that $(\text{attr}, \mathbf{r}) \in \mathcal{B}_1^{w+l}$ (range proof) and $\mathbf{B}_1 \text{attr} + \mathbf{B}_2 \mathbf{r} = \mathbf{t}$ (linear relation). For the security argument, the proof π_0 should satisfy *straight-line* extractability.

²Some constructions additionally require sampling a random tag $\tau \leftarrow \$\mathcal{T}$ for security, and thus the matrix $\mathbf{U}$ ultimately depends on τ . For presentation purposes, we ignore existence of the tag. One can easily extend our approach to the tag-based signatures by additionally giving a proof of knowledge of a valid tag $\tau \in \mathcal{T}$.

This has been instantiated by applying the LNP framework together with: (i) either the encryption-to-the-sky paradigm [LCL⁺23], or (ii) the Katsumata transform [Kat21], [dPK22], [BLNS23b] to prove security in the quantum setting. Finally, the user sends $(\mathbf{t}, \pi_0)$ to the issuer.

The issuer $\mathcal{I}$ first checks validity of the proof π_0 . Then, using the trapdoor $\mathbf{T}_U$ for the public matrix $\mathbf{U}$, the issuer returns to the user a short preimage $\mathbf{s}$ which satisfies

$$\mathbf{U}\mathbf{s} = \mathbf{t}. \quad (5.3)$$

We highlight that $\mathbf{s}$ becomes a plain signature described in Equation 5.1 for

$$\mathbf{D} := [\mathbf{B}_1 \mid \mathbf{B}_2] \text{ and } \mathbf{m} := \begin{bmatrix} \text{attr} \\ \mathbf{r} \end{bmatrix}.$$

Finally, we discuss how the final credential is constructed. Suppose we can parse $\text{attr}^\top := [\text{attr}_{\text{priv}}^\top \mid \text{attr}_{\text{pub}}^\top]$, where $\text{attr}_{\text{priv}}$ (resp. attr_{pub}) are the attributes the user wants to hide (resp. reveal). Accordingly, we can split the matrix $\mathbf{B}_1 := [\mathbf{B}_{\text{priv}} \mid \mathbf{B}_{\text{pub}}]$ and via Equations 5.2 and 5.3 we obtain

$$\mathbf{U}\mathbf{s} = \mathbf{t} = [\mathbf{B}_{\text{priv}} \mid \mathbf{B}_2] \begin{bmatrix} \text{attr}_{\text{priv}} \\ \mathbf{r} \end{bmatrix} + \mathbf{B}_{\text{pub}}\text{attr}_{\text{pub}}.$$

Then, the anonymous credential produced by the user becomes a NIZK π_1 of short vectors $\text{attr}_{\text{priv}}, \mathbf{r}, \mathbf{s}$ such that

$$[-\mathbf{B}_{\text{priv}} \mid -\mathbf{B}_2 \mid \mathbf{U}] \begin{bmatrix} \text{attr}_{\text{priv}} \\ \mathbf{r} \\ \mathbf{s} \end{bmatrix} = \mathbf{B}_{\text{pub}}\text{attr}_{\text{pub}}.$$

If one specifically requires the witness vector to contain only binary polynomials in $\mathcal{B}_1$, we can binary-decompose $\mathbf{s} \in R_q^{\hat{m}}$ into a longer vector $\hat{\mathbf{s}} := \mathbf{G}_s^{-1}(\mathbf{s})$ which contains binary polynomials in $\mathcal{B}_1$ and $\mathbf{G}_s\hat{\mathbf{s}} = \mathbf{s}$ for a gadget matrix $\mathbf{G}_s$. Then, the proof π_1 gets transformed into proving knowledge of a witness vector $(\text{attr}_{\text{priv}}, \mathbf{r}, \hat{\mathbf{s}})$ over $\mathcal{B}_1$ such that

$$[-\mathbf{B}_{\text{priv}} \mid -\mathbf{B}_2 \mid \mathbf{U}\mathbf{G}_s] \begin{bmatrix} \text{attr}_{\text{priv}} \\ \mathbf{r} \\ \hat{\mathbf{s}} \end{bmatrix} = \mathbf{B}_{\text{pub}}\text{attr}_{\text{pub}}.$$

Strategies to prove security. To prove that the scheme satisfies anonymity or unlinkability, one makes use of the fact that the terms revealed by a user during the interaction with the issuer, or when showing the credential, are: (i) a hiding commitment $\mathbf{t}$, (ii) zero-knowledge proof π_0 and (iii) zero-knowledge proof π_1 .

On the other hand, one-more unforgeability is shown via an efficient reduction to the unforgeability of the underlying signature scheme. That is, on the i -th signing query, where a reduction is given the proof $\pi_{0,i}$ from the adversary against one-more unforgeability, it extracts the corresponding witness $(\text{attr}_i, \mathbf{r}_i)$ in a straight-line fashion and queries the signing oracle to obtain a valid signature s_i . Then, the reduction outputs s_i to the adversary. Finally, after κ signing queries, the adversary outputs $\kappa+1$ valid attribute-credential pairs $(\text{attr}_j^*, \pi_{1,j}^*)_{j \in [\kappa+1]}$ ³. The reduction forges a new signature by extracting the witness from the proof $\pi_{1,j}^*$, where j is an index such that $\text{attr}_j^* \notin \{\text{attr}_1, \dots, \text{attr}_\kappa\}$. As noted in prior works, e.g. [BLNS23b], [dPK22], the NIZK proof π_1 does not need to be straight-line extractable.

³For presentation, we assumed that no partial attributes are revealed. One can similarly handle the case for selective disclosure.

5.2 Supporting Credential Revocation

We now describe how to upgrade the anonymous credential framework above to support credential revocation using our positive dynamic accumulator (cf. Construction 4.1) and the commit-and-prove argument in Section D.

In addition to the attribute vector $\text{attr} \in \mathcal{B}_1^w$, we can assume the user $\mathcal{U}$ also knows their revocation attribute $\mathbf{x} \in \mathcal{M} = \{0, 1\}^\ell$. The user samples a randomness vector $\mathbf{r} \leftarrow \mathcal{B}_1^l$ and computes the Ajtai commitment

$$\mathbf{t} := \mathbf{B}_1 \begin{bmatrix} \mathbf{x} \\ \text{attr} \end{bmatrix} + \mathbf{B}_2 \mathbf{r}.$$

Moreover, the user generates a NIZK π_0 of $(\text{attr}, \mathbf{r})$ such that $(\text{attr}, \mathbf{r}) \in \mathcal{B}_1^{w+l}$ and $\mathbf{t}$ is well-formed. We note that in π_0 , the revocation attribute $\mathbf{x}$ is treated as public. The user outputs a pair $(\mathbf{t}, \pi_0)$.

The issuer, which knows the revocation attribute $\mathbf{x}$ of $\mathcal{U}$, first verifies validity of the proof π_0 . Then, as before, it produces a short preimage $\mathbf{s}$, such that $\mathbf{U}\mathbf{s} = \mathbf{t}$, using the trapdoor $\mathbf{T}_U$. Now, for revocation purposes, the issuer runs the algorithm Add from Construction 4.1 to obtain a witness matrix $\mathbf{s}_\mathbf{x}$. The issuer outputs a pair $(\mathbf{s}_\mathbf{x}, \mathbf{s})$ to the user.

To generate a final anonymous credential, let us parse $\text{attr}^\top := [\text{attr}_{\text{priv}}^\top \mid \text{attr}_{\text{pub}}^\top]$ in a way that:

$$\mathbf{t} = [\mathbf{B}_{\text{priv}} \mid \mathbf{B}_2] \begin{bmatrix} \mathbf{x} \\ \text{attr}_{\text{priv}} \\ \mathbf{r} \end{bmatrix} + \mathbf{B}_{\text{pub}} \text{attr}_{\text{pub}}.$$

Then, by binary-decomposing the vector $\mathbf{s}$ into $\hat{\mathbf{s}}$ as before, the user produces a proof π_1 of knowledge of $(\text{attr}_{\text{priv}}, \mathbf{x}, \mathbf{r}, \hat{\mathbf{s}}, \mathbf{s}_\mathbf{x})$ such that:

$$\begin{aligned} \mathbf{x} \in \{0, 1\}^\ell, \quad (\text{attr}_{\text{priv}}, \mathbf{r}, \hat{\mathbf{s}}) &\in \mathcal{B}_1^{\hat{l}}, \quad \|\mathbf{s}_\mathbf{x}\| \leq \beta \\ [-\mathbf{B}_{\text{priv}} \mid -\mathbf{B}_2 \mid \mathbf{U}\mathbf{G}_s] \begin{bmatrix} \mathbf{x} \\ \text{attr}_{\text{priv}} \\ \mathbf{r} \\ \hat{\mathbf{s}} \end{bmatrix} &= \mathbf{B}_{\text{pub}} \text{attr}_{\text{pub}} \\ [\mathbf{A} \mid \mathbf{B} - \mathbf{x}^\top \otimes \mathbf{G}] \cdot \mathbf{s}_\mathbf{x} &= \mathbf{c} \end{aligned}$$

for some length $\hat{l}$. To conclude our construction, we highlight that this exact statement is covered in our commit-and-prove protocol in Section D.

5.3 Concrete Parameter Instantiation

We consider the anonymous credential system implemented in the LaZer library [LSS24, Section 6.2]. Namely, let $p = 12289$ and $d = 64$. Then, π_1 essentially becomes the proof of knowledge of $\mathbf{r} \in \mathcal{R}_p^{16}$, $\mathbf{m} \in \mathcal{R}_p^{16}$, $\mathbf{s} \in \mathcal{R}_p^{16}$ such that the following holds: (1) $\mathbf{U}_r \mathbf{r} + \mathbf{U}_m \mathbf{m} + \mathbf{U}_s \mathbf{s} = \mathbf{t}$, (2) $\mathbf{m} \in \mathcal{B}_1^{16}$, (3) $\|\mathbf{r}\| \leq 109$, and (4) $\|\mathbf{s}\| \leq \sqrt{34034726}$, where $\mathbf{U}_r, \mathbf{U}_m, \mathbf{U}_s \in \mathcal{R}_p^{8 \times 16}$ and $\mathbf{t}$ are public. Let us add a revocation attribute

vector as described before and set $\ell := 32$ for concreteness. Then, the statement to prove becomes:

$$\begin{aligned} (1) \quad & \mathbf{U}_r \mathbf{r} + \mathbf{U}_m \begin{bmatrix} \mathbf{m} \\ \mathbf{x} \end{bmatrix} + \mathbf{U}_s \mathbf{s} = \mathbf{t} \pmod{p} \\ (2) \quad & [\mathbf{A} \mid \mathbf{B} - \mathbf{x}^\top \otimes \mathbf{G}] \cdot \mathbf{s}_x = \mathbf{c} \pmod{q} \\ (3) \quad & \mathbf{m} \in \mathcal{B}_1^{16}, \|\mathbf{r}\| \leq 109, \|\mathbf{s}\| \leq \sqrt{34034726}, \\ (4) \quad & \|\mathbf{s}_x\| \leq \beta, \text{ and } \mathbf{x} \in \{0, 1\}^{32} \end{aligned}$$

where now where $\mathbf{U}_r, \mathbf{U}_s \in \mathcal{R}_p^{8 \times 16}$ and $\mathbf{U}_m \in \mathcal{R}_p^{8 \times 48}$.

Since the modulus p is not friendly with our positive accumulator and commit-and-prove constructions, we prove the first statement by “lifting-over-integers” [LNP22a]. Namely, instead of proving the linear equation modulo p , we first lift it to R , and thus there exists some “carry” vector $\mathbf{v} \in R^8$ such that:

$$\mathbf{U}_r \mathbf{r} + \mathbf{U}_m \begin{bmatrix} \mathbf{m} \\ \mathbf{x} \end{bmatrix} + \mathbf{U}_s \mathbf{s} - \mathbf{t} = p \cdot \mathbf{v} \quad (5.4)$$

over integers. By denoting $P = (p - 1)/2$, we note that

$$\begin{aligned} \|\mathbf{v} \cdot \mathbf{v}\|_\infty &\leq \|\mathbf{U}_r \mathbf{r}\|_\infty + \left\| \mathbf{U}_m \begin{bmatrix} \mathbf{m} \\ \mathbf{x} \end{bmatrix} \right\|_\infty + \|\mathbf{U}_s \mathbf{s}\| + \|\mathbf{t}\|_\infty \\ &\leq P(109\sqrt{16} + 48 \cdot d + \sqrt{34034726} \cdot \sqrt{16}). \end{aligned}$$

Thus, $\|\mathbf{v}\|_\infty \leq \frac{P(109\sqrt{16} + 48 \cdot d + \sqrt{34034726} \cdot \sqrt{16})}{p} < 13421$. Hence, now we append $\mathbf{v}$ to the witness and prove Equation 5.4 over R_q . If we pick q large enough and prove other norm constraints, we are able to deduce that Equation 5.4 holds over integers. Finally, by binary-decomposing vectors $\mathbf{r}, \mathbf{s}$ and $\mathbf{v}$ and knowing that $\lceil \log(109) \rceil = 7$, $\lceil \log(\sqrt{34034726}) \rceil = 13$ and $\lceil \log(12397) \rceil = 14$ we can alternatively (with slightly looser bounds) prove knowledge of a vector $\mathbf{r} \in \mathcal{R}_q^{16 \cdot 7 + 16 + 16 \cdot 13 + 16 \cdot 14}$, the revocation attribute $\mathbf{x} \in R_q^\ell$ and its witness vector $\mathbf{s}_x$ such that

$$\begin{cases} \mathbf{x} \in \{0, 1\}^\ell \wedge \|\mathbf{s}_x\| \leq \beta \wedge \mathbf{r} \in \mathcal{B}_1^{560} \\ \mathbf{A}_1 \mathbf{x} + \mathbf{A}_2 \mathbf{r} = \mathbf{t} \\ [\mathbf{A} \mid \mathbf{B} - \mathbf{x}^\top \otimes \mathbf{G}] \cdot \mathbf{s}_x = \mathbf{c} \end{cases} \quad (5.5)$$

for some public $\mathbf{A}_1 \in \mathcal{R}_q^{8 \times 32}$, $\mathbf{A}_2 \in R_q^{8 \times 560}$ and $\mathbf{t} \in \mathcal{R}_q^8$. Our strategy to prove this statement is by composing the LNP commit-and-prove zero-knowledge framework by Lyubashevsky et al. [LNP22a] together with the LaBRADOR succinct proof system by Beullens and Seiler [BS23]. Consequently, we achieve honest-verifier zero-knowledge property thanks to the LNP layer, while obtaining succinct proof sizes due to LaBRADOR (see Appendix D).

Instantiating the positive replacement-free accumulator. We set parameters to match the 128-bit security level. We pick a prime modulus $q \approx 2^{100}$ and a ring dimension $d = 4096^4$. We assume the length of the revocation attribute is $\ell = 32$ and the maximal number of deletions γ equals 2^{32} . For concrete efficiency, we also naturally adapted the results from [MP12] to accommodate for different bases than 2 because having large m' significantly affects the accumulator and the membership witness sizes. Here, we set the base to be

⁴A cautious reader may notice that this is not the ring dimension defined before. We will come back to this issue.

Scheme	d	q	#Add	#Del	n	m	$\lceil \log_b q \rceil$	β	$ w_x $	$ \text{upmsg} _{\text{Add}}$	$ \text{upmsg} _{\text{Del}}$	$ \mathcal{A} $	$ \text{pp} $
[dCP23] (M-SIS)	2^{12}	$\approx 2^{90}$	2^{32}	2^{32}	1	20	10	$\approx 2^{75.46}$	12MB	4 B	4 B	45KB	14.2MB
[dCP23]+[WW23a] (ℓ -Succinct M-SIS)	2^{12}	$\approx 2^{150}$	2^{32}	2^{32}	1	64	30	$\approx 2^{118.4}$	5.5MB	4 B	4 B	75KB	77.3MB
Our work (M-SIS)	2^{12}	$\approx 2^{100}$	—	2^{32}	1	20	10	$\approx 2^{87.7}$	14.72MB	—	4 B	50KB	16.7MB
Our work (ℓ -Succinct M-SIS)	2^{12}	$\approx 2^{162}$	—	2^{32}	1	90	63	$\approx 2^{124}$	9.33MB	—	4 B	81KB	171.7MB

Table 2: Parameters for $\lambda = 128$ and $\ell = 32$. #Add is the max number of addition, and #Del is the max number of deletions. n and m are the dimensions of the matrix $\mathbf{A}$ for the (ℓ -Succinct) M-SIS assumption. β is the ℓ_2 norm bound on witnesses. $|\text{upmsg}|_{\text{Op}}$ is the size of the update information produced after executing one $\text{Op} \in \{\text{Add}, \text{Delete}\}$.

$\approx q^{1/10}$. Other key parameters are summarized in Table 2. For comparison, we also presented parameters for De Castro and Peikert’s accumulator [dCP23], as well as its ℓ -succinct M-SIS variant proposed by Wee and Wu [WW23a], evaluated under the same setting (128-bit security and $\ell = 32$). In our Construction, the membership witness’s norm grows linearly with the number of deletions, so we set β to handle 2^{32} deletions. In contrast, in the De Castro-Peikert and We-Wu schemes, the norm grows linearly with the number of additions, so we set β to handle 2^{32} additions. Focusing on the M-SIS variants, we observe that our construction is almost comparable to De Castro-Peikert scheme in terms of computational resources, while offering free addition. However, for all constructions, we need choose the modulus q such that $q > \beta$, and this is one of the leading causes for the current impracticality of these constructions. Following Lemma 2.4, we can choose q to be a product of small primes that are Number Theoretic Transform friendly, and this would allow to improve efficiency via parallelization. But, even in that case, we do not foresee the efficiency gain to be significant for deployment.

Commit-and-prove sizes. As mentioned above, the ring dimension of the accumulator and the anonymous credentials do not match. Luckily, as shown in [LNPS21], statements over $\mathbb{Z}_q[X]/(X^{4096} + 1)$ can be translated into equivalent ones over a smaller ring $\mathbb{Z}_q[X]/(X^{64} + 1)$. We apply this strategy to when providing the LNP and LaBRADOR proofs. For the LNP layer, we obtain the partial proof of size $\approx 25\text{KB}$. As for the LaBRADOR and the experiments shown in [BS23, LSS24], we believe the rest of the proof can be compressed to $\approx 100\text{KB}$. Hence, the total proof size would very likely be under 125KB.

Acknowledgments

We thank Scott Griffy and Rutchathon (Champ) Chairattana-Apirom for insightful discussions on this project. Victor Youdom Kemmoe and Anna Lysyanskaya were supported by NSF Grants 2312241, 2154170, and 2247305 as well as the Ethereum Foundation.

References

- [AAB⁺24] Marius A. Aardal, Diego F. Aranha, Katharina Boudgoust, Sebastian Kolby, and Akira Takahashi. Aggregating falcon signatures with labrador. In *CRYPTO (1)*. Springer, 2024.
- [AB09] Sanjeev Arora and Boaz Barak. *Computational Complexity: A Modern Approach*. Cambridge University Press, 2009.

- [ABB10] Shweta Agrawal, Dan Boneh, and Xavier Boyen. Efficient lattice (H)IBE in the standard model. In EUROCRYPT. Springer, 2010.
- [AGJ⁺24] Sven Argo, Tim Güneysu, Corentin Jeudy, Georg Land, Adeline Roux-Langlois, and Olivier Sanders. Practical post-quantum signatures for privacy. In CCS. ACM, 2024.
- [Ajt96] Miklós Ajtai. Generating hard instances of lattice problems (extended abstract). In STOC. ACM, 1996.
- [ATSM09] Man Ho Au, Patrick P. Tsang, Willy Susilo, and Yi Mu. Dynamic universal accumulators for DDH groups and their application to attribute-based anonymous credential systems. In CT-RSA. Springer, 2009.
- [BBC⁺24] Carsten Baum, Olivier Blazy, Jan Camenisch, Jaap-Henk Hoepman, Eysa Lee, Anja Lehmann, Anna Lysyanskaya, René Mayrhofer, Hart Montgomery, Ngoc Khanh Nguyen, Bart Praneel, abhi shelat, Daniel Slamanig, Stefano Tessaro, Søren Eller Thomsen, and Carmela Troncoso. Cryptographers’ feedback on the eu digital identity’s ARF. <https://github.com/user-attachments/files/15904122/cryptographers-feedback.pdf>, 2024.
- [BCD⁺17] Foteini Baldimtsi, Jan Camenisch, Maria Dubovitskaya, Anna Lysyanskaya, Leonid Reyzin, Kai Samelin, and Sophia Yakoubov. Accumulators with applications to anonymity-preserving revocation. In EuroS&P. IEEE, 2017.
- [BCTW16] Zvika Brakerski, David Cash, Rotem Tsabary, and Hoeteck Wee. Targeted homomorphic attribute-based encryption. In TCC (B2), 2016.
- [BdM93] Josh Benaloh and Michael de Mare. One-way accumulators: A decentralized alternative to digital sinatures. In EUROCRYPT. Springer, 1993.
- [BGG⁺14] Dan Boneh, Craig Gentry, Sergey Gorbunov, Shai Halevi, Valeria Nikolaenko, Gil Segev, Vinod Vaikuntanathan, and Dhinakaran Vinayagamurthy. Fully key-homomorphic encryption, arithmetic circuit ABE and compact garbled circuits. In EUROCRYPT. Springer, 2014.
- [BJRW20] Katharina Boudgoust, Corentin Jeudy, Adeline Roux-Langlois, and Weiqiang Wen. Towards classical hardness of module-lwe: The linear rank case. In ASIACRYPT (2). Springer, 2020.
- [BLNS23a] Ward Beullens, Vadim Lyubashevsky, Ngoc Khanh Nguyen, and Gregor Seiler. Lattice-based blind signatures: Short, efficient, and round-optimal. In CCS. ACM, 2023.
- [BLNS23b] Jonathan Bootle, Vadim Lyubashevsky, Ngoc Khanh Nguyen, and Alessandro Sorniotti. A framework for practical anonymous credentials from lattices. In CRYPTO (2). Springer, 2023.
- [BLR⁺18] Shi Bai, Tancrède Lepoint, Adeline Roux-Langlois, Amin Sakzad, Damien Stehlé, and Ron Steinfeld. Improved security proofs in lattice-based cryptography: Using the rényi divergence rather than the statistical distance. J. Cryptol., 31(2), 2018.
- [BP97] Niko Barić and Birgit Pfitzmann. Collision-free accumulators and fail-stop signature schemes without trees. In EUROCRYPT. Springer, 1997.
- [BS23] Ward Beullens and Gregor Seiler. Labrador: Compact proofs for R1CS from module-sis. In CRYPTO (5). Springer, 2023.

- [CHKP10] David Cash, Dennis Hofheinz, Eike Kiltz, and Chris Peikert. Bonsai trees, or how to delegate a lattice basis. In EUROCRYPT. Springer, 2010.
- [CHW25] Jeffrey Champion, Yao-Ching Hsieh, and David J. Wu. Registered ABE and adaptively-secure broadcast encryption from succinct LWE. Cryptology ePrint Archive, Paper 2025/044, 2025.
- [CKL⁺15] Jan Camenisch, Stephan Krenn, Anja Lehmann, Gert Læssøe Mikkelsen, Gregory Neven, and Michael Østergaard Pedersen. Formal treatment of privacy-enhancing credential systems. In SAC. Springer, 2015.
- [CKS09] Jan Camenisch, Markulf Kohlweiss, and Claudio Soriente. An accumulator based on bilinear maps and efficient revocation for anonymous credentials. In Public Key Cryptography. Springer, 2009.
- [CL02a] Jan Camenisch and Anna Lysyanskaya. Dynamic accumulators and application to efficient revocation of anonymous credentials. In CRYPTO. Springer, 2002.
- [CL02b] Jan Camenisch and Anna Lysyanskaya. A signature scheme with efficient protocols. In SCN. Springer, 2002.
- [dCP23] Leo de Castro and Chris Peikert. Functional commitments for all functions, with transparent setup and from SIS. In EUROCRYPT (3). Springer, 2023.
- [DM14] Léo Ducas and Daniele Micciancio. Improved short lattice signatures in the standard model. Cryptology ePrint Archive, Paper 2014/495, 2014.
- [dPK22] Rafaël del Pino and Shuichi Katsumata. A new framework for more efficient round-optimal lattice-based (partially) blind signature via trapdoor sampling. In CRYPTO (2). Springer, 2022.
- [ESZ22] Muhammed F. Esgin, Ron Steinfeld, and Raymond K. Zhao. Matric⁺: More efficient post-quantum private blockchain payments. In SP. IEEE, 2022.
- [FS86] Amos Fiat and Adi Shamir. How to prove yourself: Practical solutions to identification and signature problems. In CRYPTO. Springer, 1986.
- [GHL22] Craig Gentry, Shai Halevi, and Vadim Lyubashevsky. Practical non-interactive publicly verifiable secret sharing with thousands of parties. In EUROCRYPT (1). Springer, 2022.
- [GPV08] Craig Gentry, Chris Peikert, and Vinod Vaikuntanathan. Trapdoors for hard lattices and new cryptographic constructions. In STOC. ACM, 2008.
- [GSW13] Craig Gentry, Amit Sahai, and Brent Waters. Homomorphic encryption from learning with errors: Conceptually-simpler, asymptotically-faster, attribute-based. In CRYPTO (1). Springer, 2013.
- [GVW15] Sergey Gorbunov, Vinod Vaikuntanathan, and Daniel Wichs. Leveled fully homomorphic signatures from standard lattices. In STOC. ACM, 2015.
- [JML24] Samuel Jaques, Hart Montgomery, and Michael Lodder. ALLOSAUR: accumulator with low-latency oblivious sublinear anonymous credential updates with revocations. In AsiaCCS. ACM, 2024.
- [JRS23] Corentin Jeudy, Adeline Roux-Langlois, and Olivier Sanders. Lattice signature with efficient protocols, application to anonymous credentials. In CRYPTO (2). Springer, 2023.

- [Kat21] Shuichi Katsumata. A new simple technique to bootstrap various lattice zero-knowledge proofs to QROM secure nizks. In CRYPTO (2). Springer, 2021.
- [KB21] Ioanna Karantaidou and Foteini Baldimtsi. Efficient constructions of pairing based accumulators. In CSF. IEEE, 2021.
- [KL24] Victor Youdom Kemmoe and Anna Lysyanskaya. Rsa-based dynamic accumulator without hashing into primes. In CCS. ACM, 2024.
- [KLSS23] Duhyeong Kim, Dongwon Lee, Jinyeong Seo, and Yongsoo Song. Toward practical lattice-based proof of knowledge from hint-mlwe. In CRYPTO (5). Springer, 2023.
- [LCL⁺23] Qiqi Lai, Chongshen Chen, Feng-Hao Liu, Anna Lysyanskaya, and Zhedong Wang. Lattice-based commit-transferrable signatures and applications to anonymous credentials. Cryptology ePrint Archive, Paper 2023/766, 2023.
- [LLM⁺16] Benoît Libert, San Ling, Fabrice Mouhartem, Khoa Nguyen, and Huaxiong Wang. Signature schemes with efficient protocols and dynamic group signatures from lattice assumptions. In ASIACRYPT (2), 2016.
- [LLNW23] Benoît Libert, San Ling, Khoa Nguyen, and Huaxiong Wang. Zero-knowledge arguments for lattice-based accumulators: Logarithmic-size ring signatures and group signatures without trapdoors. J. Cryptol., 36(3), 2023.
- [LLX07] Jiangtao Li, Ninghui Li, and Rui Xue. Universal accumulators with efficient nonmembership proofs. In ACNS. Springer, 2007.
- [LN22] Vadim Lyubashevsky and Ngoc Khanh Nguyen. BLOOM: bimodal lattice one-out-of-many proofs and applications. In ASIACRYPT (4). Springer, 2022.
- [LNP22a] Vadim Lyubashevsky, Ngoc Khanh Nguyen, and Maxime Plançon. Lattice-based zero-knowledge proofs and applications: Shorter, simpler, and more general. In CRYPTO (2). Springer, 2022.
- [LNP22b] Vadim Lyubashevsky, Ngoc Khanh Nguyen, and Maxime Plançon. Lattice-based zero-knowledge proofs and applications: Shorter, simpler, and more general. IACR Cryptol. ePrint Arch., page 284, 2022.
- [LNPS21] Vadim Lyubashevsky, Ngoc Khanh Nguyen, Maxime Plançon, and Gregor Seiler. Shorter lattice-based group signatures via "almost free" encryption and other optimizations. In ASIACRYPT (4). Springer, 2021.
- [LPR13] Vadim Lyubashevsky, Chris Peikert, and Oded Regev. A toolkit for ring-lwe cryptography. In EUROCRYPT. Springer, 2013.
- [LS15] Adeline Langlois and Damien Stehlé. Worst-case to average-case reductions for module lattices. Des. Codes Cryptogr., 75(3), 2015.
- [LSS24] Vadim Lyubashevsky, Gregor Seiler, and Patrick Steuer. The lazer library: Lattice-based zero knowledge and succinct proofs for quantum-safe privacy. In CCS. ACM, 2024.
- [Lyu12] Vadim Lyubashevsky. Lattice signatures without trapdoors. In EUROCRYPT. Springer, 2012.

- [MP12] Daniele Micciancio and Chris Peikert. Trapdoors for lattices: Simpler, tighter, faster, smaller. In EUROCRYPT. Springer, 2012.
- [MR07] Daniele Micciancio and Oded Regev. Worst-case to average-case reductions based on gaussian measures. SIAM J. Comput., 37(1), 2007.
- [Neu99] Jürgen Neukirch. Algebraic Integers. Springer Berlin Heidelberg, 1999.
- [Ngu05] Lan Nguyen. Accumulators from bilinear pairings and applications. In CT-RSA. Springer, 2005.
- [Pei07] Chris Peikert. Limits on the hardness of lattice problems in ℓ_p norms. In Twenty-Second Annual IEEE Conference on Computational Complexity (CCC’07), 2007.
- [PPS21] Chris Peikert, Zachary Pepin, and Chad Sharp. Vector and functional commitments from lattices. In TCC (3). Springer, 2021.
- [PSTY13] Charalampos Papamanthou, Elaine Shi, Roberto Tamassia, and Ke Yi. Streaming authenticated data structures. In EUROCRYPT. Springer, 2013.
- [RY16] Leonid Reyzin and Sophia Yakubov. Efficient asynchronous accumulators for distributed PKI. In SCN, pages 292–309. Springer, 2016.
- [Ste96] Jacques Stern. A new paradigm for public key identification. IEEE Trans. Inf. Theory, 42(6), 1996.
- [TT15] Katsuyuki Takashima and Atsushi Takayasu. Tighter security for efficient lattice cryptography via the rényi divergence of optimized orders. In ProvSec. Springer, 2015.
- [Wee24] Hoeteck Wee. Circuit abe with poly (depth, λ)-sized ciphertexts and keys from lattices. In CRYPTO (3). Springer, 2024.
- [WW23a] Hoeteck Wee and David J. Wu. Lattice-based functional commitments: Fast verification and cryptanalysis. In ASIACRYPT (5), pages 201–235. Springer, 2023.
- [WW23b] Hoeteck Wee and David J. Wu. Succinct vector, polynomial, and functional commitments from lattices. In EUROCRYPT (3). Springer, 2023.
- [YAY⁺18] Zuoxia Yu, Man Ho Au, Rupeng Yang, Junzuo Lai, and Qiuliang Xu. Lattice-based universal accumulator with nonmembership arguments. In ACISP. Springer, 2018.
- [ZYH24] Yong Zhao, Shaojun Yang, and Xinyi Huang. Lattice-based dynamic universal accumulator: Design and application. Computer Standards & Interfaces, 89, 2024.

A Extra Preliminaries

In this section, we introduce additional preliminaries.

Lemma A.1 ([WW23b], Lemma 2.10). *Let n, q be lattice parameters and take $m = O(n \log q)$. For any matrix $\mathbf{A} \in R_q^{n \times m}$ such that the columns of $\mathbf{A}$ generates R_q^n and matrix $\mathbf{B} \in \mathcal{R}_q^{n \times k}$ where $k = \text{poly}(n, \log q)$, let $\mathbf{C} = [\mathbf{A} \mid \mathbf{B}] \in \mathcal{R}_q^{n \times (m+k)}$. It follows that for any target vector $\mathbf{t} \in R_q^n$ and Gaussian*

width parameter $\mathfrak{s} \geq \eta_\varepsilon(\Lambda^\perp(\mathbf{A}))$ for some $\varepsilon(n) = \text{negl}(n)$, the following distributions are statistically close:

$$\left\{ \mathbf{v} : \mathbf{v} \leftarrow D_{\Lambda_t^\perp(\mathbf{C}), \mathfrak{s}} \right\} \stackrel{s}{\approx} \left\{ \begin{pmatrix} \mathbf{v}_1 \\ \mathbf{v}_2 \end{pmatrix} : \mathbf{v}_2 \leftarrow D_{R^k, \mathfrak{s}}, \mathbf{v}_1 \leftarrow D_{\Lambda_{\mathbf{t} - \mathbf{B}\mathbf{v}_2}^\perp(\mathbf{A}), \mathfrak{s}} \right\}$$

Lemma A.2. *Let $\mathbf{R} \leftarrow_{\$} (S_1^\infty)^{n \times m}$ be an $n \times m$ matrix whose components are polynomials in R with coefficients in $\{-1, 0, 1\}$ sampled uniformly at random. Then, $s_1(\mathbf{R}) \leq \sqrt{2d/3}(\sqrt{n} + \sqrt{m} + \log \sqrt{d})$ with overwhelming probability.*

Proof. Ducas and Micciancio [DM14, Fact 6] proved that for any matrix $\mathbf{A} \in R^{n \times m}$ whose components are polynomials in R with coefficients sampled independently from a mean zero subgaussian distribution with Gaussian width parameter $\mathfrak{s}$, it follows that $s_1(\mathbf{A}) \leq \mathfrak{s}\sqrt{d}(\sqrt{n} + \sqrt{m} + \log \sqrt{d})$ with overwhelming probability. Therefore, all we need to show is that for any matrix $\mathbf{R} \leftarrow_{\$} (S_1^\infty)^{n \times m}$, the coefficients of the polynomials that compose $\mathbf{R}$ are independent mean zero subgaussian random variables with Gaussian width parameter $\sqrt{2/3}$. Let $\mathbf{R}_{i,j}$ be one of the component of $\mathbf{R}$ and let $\mathbf{R}_{i,j}[k]$ be one of the coefficients of $\mathbf{R}_{i,j}$. By definition, $\mathbf{R}_{i,j}[k] \sim U(\{0, \pm 1\})$ and $\mathbb{E}(\mathbf{R}_{i,j}[k]) = 0$. In addition, for any $t \in \mathbb{R}$, we have

$$\mathbb{E} \left(e^{t\mathbf{R}_{i,j}[k]} \right) = \frac{1}{3} (e^{-t} + e^t) \leq e^{\frac{(2/3)t^2}{2}}$$

This completes the proof. $\square$

B Rényi divergence

Definition B.1 ([BLR⁺18], adapted). *Let P and Q be two discrete probability distributions such that $\text{Supp}(P) \subseteq \text{Supp}(Q)$. For any $\alpha \in [1, \infty]$, the Rényi divergence of order α of P from Q is defined by*

$$\mathfrak{R}_\alpha(P\|Q) = \begin{cases} \exp \left(\sum_{x \in \text{Supp}(P)} P(x) \log \frac{P(x)}{Q(x)} \right) & , \text{ if } \alpha = 1 \\ \left(\sum_{x \in \text{Supp}(P)} \frac{P(x)^\alpha}{Q(x)^{\alpha-1}} \right)^{1/(\alpha-1)} & , \text{ if } \alpha \in (1, \infty) \\ \max_{x \in \text{Supp}(P)} \frac{P(x)}{Q(x)} & , \text{ if } \alpha = \infty \end{cases}$$

In addition, it satisfies the following properties:

- **Data Processing Inequality:** $\mathfrak{R}_\alpha(P^{\mathbf{f}}\|Q^{\mathbf{f}}) \leq \mathfrak{R}_\alpha(P\|Q)$ for any function $\mathbf{f}$, where $P^{\mathbf{f}}$ (resp. $Q^{\mathbf{f}}$) denotes the distribution of $\mathbf{f}(x)$ induced by sampling $y \sim P$ (resp. $y \sim Q$).
- **Probability Preservation:** Let $E \subseteq \text{Supp}(Q)$ be an arbitrary event. For any $\alpha \in (1, \infty)$, we have $Q(E) \geq P(E)^{\frac{\alpha}{\alpha-1}} / \mathfrak{R}_\alpha(P\|Q)$. In addition, $Q(E) \geq P(E) / \mathfrak{R}_\infty(P\|Q)$.

Lemma B.1 ([TT15], adapted). *Let $\Lambda \subset \mathbb{R}^n$ be an n -dimensional lattice. For any $\mathbf{c}, \mathbf{c}' \in \mathbb{R}^n$, if $\mathbf{c}, \mathbf{c}' \in \Lambda$, let $\varepsilon = 0$. Else, let $\varepsilon \in (0, 1)$. For any Gaussian width parameter $\mathfrak{s} \geq \eta_\varepsilon(\Lambda)$, we have*

$$\mathfrak{R}_\alpha(D_{\Lambda, \mathfrak{s}, \mathbf{c}} \| D_{\Lambda, \mathfrak{s}, \mathbf{c}'}) \leq \left(\frac{1 + \varepsilon}{1 - \varepsilon} \right)^{\alpha/(\alpha-1)} \cdot \exp \left(\alpha \pi \frac{\|\mathbf{c} - \mathbf{c}'\|^2}{\mathfrak{s}^2} \right)$$

C Deferred Proofs

C.1 Proof of Theorem 4.3

Theorem 4.3[Selective Collision-Freeness] For any $\lambda \in \mathbb{N}$, let $q = q(\lambda)$ be an integer that satisfies Lemma 2.4, $n = n(\lambda)$, $m' = n \lceil \log_b q \rceil$, $m = O(m')$ be lattice parameters. Let $b \geq 2$, $\alpha \geq 2$, $\varepsilon = \text{negl}(\lambda)$, and $\varsigma > 2d \cdot q^{n/m+2/(dm)}$. Let $\theta_1 = \sqrt{2d/3}(\sqrt{m} + \sqrt{m'} + \log \sqrt{d})$ and $\theta_2 = \sqrt{2d/3}(\sqrt{m} + \sqrt{\ell m'} + \log \sqrt{d})$. Let $\ell = \ell(\lambda)$ be the input length, $\gamma = \gamma(\lambda)$ be a bound on the number of deletions, $T = \gamma(b-1)\sqrt{m'd}\sqrt{((b-1)dm'\sqrt{\ell}-1)^2+1}$ be an upper bound on ℓ_2 -norm of the vector used to update witnesses, $\mathfrak{s} = \max(T, b\sqrt{1+\theta_1^2} \cdot \log \sqrt{nd}, b\varsigma\sqrt{d}(\sqrt{m} + \sqrt{m'} + \log \sqrt{d}) \log \sqrt{nd})$ be a Gaussian width parameter, $\beta_0 = \mathfrak{s}\sqrt{d(m+\ell m')} + T$ be a norm bound on the witnesses, and $\mathfrak{r} = \mathfrak{r}(\lambda)$ be a bound on the randomness used by SampleLeft and SampleRight. Let

$$\beta_1 = (\mathfrak{s} + 1)\sqrt{md} + \theta_2(\mathfrak{s}\sqrt{d\ell m'} + T) + \beta_0\sqrt{1+\theta_2^2}$$

Suppose F is a secure PRF, the assumptions $\text{M-LWE}_{m-n,m,q,U(S_1^\infty)}$, $\text{M-LWE}_{m-n,m,q,U(\mathcal{B}_1)}$, and $\text{M-SIS}_{n,m,q,\beta_1}$ hold. Then Construction 4.1 is a selectively collision-free PRFA (cf. Definitions 3.3 and 3.4).

Proof. We proceed via a sequence of hybrids:

- **Hyb₀:** This is the real selective collision-freeness experiment:
 1. The adversary $\mathcal{A}$ commits to an input $\mathbf{x}^* \in \mathcal{M}$. Then, the challenger simulates the execution of $\text{Gen}(1^\lambda, \text{aux} = (1^n, 1^m, q))$ as follows: execute $(\mathbf{A}, \mathbf{T}_\mathbf{A}) \leftarrow \text{TrapGen}(1^n, 1^m, q)$, sample $\mathbf{B} \leftarrow \$ R_q^{n \times \ell m'}$, $(\mathbf{c}_0, \mathbf{u}) \leftarrow \$ R_q^{2n}$, initialize $\text{ctr} := 0$ and $\text{DX} : \mathcal{M} \rightarrow \{\text{Add}, \text{Del}, \perp\} \times \mathbb{N}$, sample a PRF key $k \leftarrow \$ \{0, 1\}^\lambda$ and, finally, set $\text{pp} := (\mathbf{A}, \mathbf{B}, \mathbf{u})$, $\text{sk} := \mathbf{T}_\mathbf{A}$, $\mathcal{A}_0 := \mathbf{c}_0$, and $\text{st} := (k, \text{ctr}, \mathbf{c}_0, \text{DX})$.
 2. For any Add, Del, or MemWitCreate queries from $\mathcal{A}$, the challenger answers by executing the respective algorithms. Also, it keeps an up-to-date value of $\mathcal{A}$.
 3. $\mathcal{A}$ outputs a witness $w_{\mathbf{x}^*}$ for $\mathbf{x}^*$.
 4. Let $(\text{Op}, i) := \text{DX}[\mathbf{x}^*]$. If $\text{Op} \neq \text{Add}$ and $[\mathbf{A} \mid \mathbf{B} - (\mathbf{x}^*)^\top \otimes \mathbf{G}] \cdot w_{\mathbf{x}^*} = \mathcal{A}$ and $\|w_{\mathbf{x}^*}\| \leq \beta_0$, the challenger outputs 1. Else, it outputs 0.
- **Hyb₁:** Similar to Hyb₀. However, the challenger replaces the PRF by a random function $f : \mathcal{M} \rightarrow \{0, 1\}^\mathfrak{r}$ sampled using the standard lazy sampling technique.
 $\text{Hyb}_1 \stackrel{\text{c}}{\approx} \text{Hyb}_0$ by the security of the PRF.
- **Hyb₂:** Similar to Hyb₁. However, during the execution of Gen , the challenger samples $\mathbf{B}' \leftarrow \$ R_q^{\ell m' \times m'}$ and sets $\mathbf{B} \leftarrow \mathbf{B}' + (\mathbf{x}^*)^\top \otimes \mathbf{G}$.
 $\text{Hyb}_2 \stackrel{\text{c}}{\approx} \text{Hyb}_1$ since $\mathbf{B}$ in Hyb₂ and $\mathbf{B}$ in Hyb₁ are sampled from the same distribution.
- **Hyb₃:** Similar to Hyb₂. However, during the execution of Gen , the challenger samples, $[\mathbf{R}_0 \mid \dots \mid \mathbf{R}_{\ell-1}]$ where $\mathbf{R}_i \leftarrow (S_1^\infty)^{m \times m'}$, then, sets $\mathbf{B}' \leftarrow \mathbf{A}[\mathbf{R}_0 \mid \dots \mid \mathbf{R}_{\ell-1}]$ and stores $[\mathbf{R}_0 \mid \dots \mid \mathbf{R}_{\ell-1}]$ in its state st .
 $\text{Hyb}_3 \stackrel{\text{c}}{\approx} \text{Hyb}_2$ by the hardness of $\text{M-LWE}_{m-n,m,q,U(S_1^\infty)}$.
- **Hyb₄:** Similar to Hyb₃. However, during the execution of Gen , the challenger samples $\mathbf{r} \leftarrow \$ \mathcal{B}_1^m$, sets $\mathbf{u} \leftarrow \mathbf{A}\mathbf{r}$, and stores $\mathbf{r}$ in its state.
 $\text{Hyb}_4 \stackrel{\text{c}}{\approx} \text{Hyb}_3$ by the hardness of $\text{M-LWE}_{m-n,m,q,U(\mathcal{B}_1)}$.
- **Hyb₅:** Similar to Hyb₄. However, if the challenger receives a valid Add query on $\mathbf{x}^*$, it proceeds as follows:

- Sample $\mathbf{s}_{\mathbf{x}^*,1} \leftarrow D_{R,\mathfrak{s}}^{\ell m'}$.
- Compute $\mathbf{s}_{\mathbf{x}^*,0} \leftarrow \text{SamplePre}(\mathbf{A}, \mathbf{T}_{\mathbf{A}}, \mathbf{c} - (\mathbf{B} - (\mathbf{x}^*)^\top \otimes \mathbf{G}) \cdot \mathbf{s}_{\mathbf{x}^*,1}, \mathfrak{s})$. ► *Note that $\mathbf{c}$ represents the current value of $\mathcal{A}$*
- Set $\mathbf{s}_{\mathbf{x}^*} \leftarrow (\mathbf{s}_{\mathbf{x}^*,0} \| \mathbf{s}_{\mathbf{x}^*,1})$
- Store $\mathbf{s}_{\mathbf{x}^*}$ in its state st to answer MemWitCreate on $\mathbf{x}^*$ by only executing step 3 and 7 of MemWitCreate algorithm (cf. Construction 4.1).
- Complete the execution of Add .

$\text{Hyb}_5 \stackrel{s}{\approx} \text{Hyb}_4$ by Lemma A.1.

- Hyb_6 : Similar to Hyb_5 . However, during the execution of Gen , the challenger samples $\mathbf{s}_0 \leftarrow D_{R,\mathfrak{s}}^m$ and $\mathbf{s}_1 \leftarrow D_{R,\mathfrak{s}}^{\ell m'}$ and sets $\mathbf{c}_0 \leftarrow \mathbf{A} \cdot \mathbf{s}_0 + (\mathbf{B} - (\mathbf{x}^*)^\top \otimes \mathbf{G}) \cdot \mathbf{s}_1$. Then, it stores $\mathbf{s}_0$ and $\mathbf{s}_1$ in its state st .
 $\text{Hyb}_6 \stackrel{s}{\approx} \text{Hyb}_5$ by Lemma 2.5 with respect to $\mathbf{A}$.
- Hyb_7 : Similar to Hyb_6 . However, during Gen , it sets $\mathbf{s}_{\mathbf{x}^*,0} = \mathbf{s}_0$ and $\mathbf{s}_{\mathbf{x}^*,1} = \mathbf{s}_1$, and then stores them in its state st . Now, during a valid Add query on $\mathbf{x}^*$, it proceeds as follows:

1. Compute $Z = \{\mathbf{y} \in \mathcal{M} : (\text{Op}, j) := \text{DX}[\mathbf{y}], \text{Op} = \text{Del}, 0 < j \leq \text{ctr}\}$. ► *The set of deleted before an add request was made for $\mathbf{x}^*$*
2. Set $\tilde{\mathbf{h}} := \mathbf{0}_{\ell m'}$.
3. For each $\mathbf{y} \in Z$, do:
 - Compute $\mathbf{h}_{\mathbb{1}_{\mathbf{y}}, \mathbf{x}^*} \leftarrow \text{EvalFx}(\mathbb{1}_{\mathbf{y}}, \mathbf{B}, \mathbf{x}^*) \cdot \mathbf{G}^{-1}(\mathbf{u})$.
 - Compute $\tilde{\mathbf{h}} \leftarrow \tilde{\mathbf{h}} + \mathbf{h}_{\mathbb{1}_{\mathbf{y}}, \mathbf{x}^*}$.
4. $\mathbf{s}_{\mathbf{x}^*} \leftarrow \begin{bmatrix} \tilde{\mathbf{s}}_{\mathbf{x}^*,0} \\ \tilde{\mathbf{s}}_{\mathbf{x}^*,1} + \tilde{\mathbf{h}} \end{bmatrix}$
5. Complete the execution of Add .

Let E_7 be the event that $\mathcal{A}$ wins in Hyb_7 and let E_6 be the event it wins in Hyb_6 . By parsing $\mathbf{s}_{\mathbf{x}^*} = [\mathbf{s}_{\mathbf{x}^*,0}^\top \mid \mathbf{s}_{\mathbf{x}^*,1}^\top]^\top$, the difference between Hyb_7 and Hyb_6 is in how $\mathbf{s}_{\mathbf{x}^*,1}$ is sampled. In Hyb_7 , $\mathbf{s}_{\mathbf{x}^*,1} = \tilde{\mathbf{s}}_{\mathbf{x}^*,1} + \tilde{\mathbf{h}}$, with $\tilde{\mathbf{s}}_{\mathbf{x}^*,1} \sim D_{R,\mathfrak{s}}^{\ell m'}$ and $\|\tilde{\mathbf{h}}\| \leq T$, whereas in Hyb_6 , $\mathbf{s}_{\mathbf{x}^*,1} \sim D_{R,\mathfrak{s}}^{\ell m'}$. Given that $\mathfrak{s} \geq T$, it follows that $\mathfrak{R}_\alpha(D_{R^{\ell m'}, \mathfrak{s}} \| D_{R^{\ell m'}, \mathfrak{s}, \tilde{\mathbf{h}}}) \leq e^{\alpha\pi}$ by Lemma B.1. Therefore, by the probability preservation property of the Rényi divergence, we have $\Pr[E_7] \geq \Pr[E_6]^{\frac{\alpha}{\alpha-1}} \cdot e^{-\alpha\pi}$, which is well defined since $\alpha \geq 2$.

- Hyb_8 : Similar to Hyb_7 . However, the challenger discards $\mathbf{T}_{\mathbf{A}}$. To answer Add or MemWitCreate queries on any $\mathbf{x} \neq \mathbf{x}^*$, it proceeds as follows:
 1. Let $i \in [\ell]$ such that $\mathbf{x}[i] \neq \mathbf{x}^*[i]$. Given that $\mathbf{B} - \mathbf{x}^\top \otimes \mathbf{G} = \mathbf{B}' + (\mathbf{x}^* - \mathbf{x})^\top \otimes \mathbf{G} = \mathbf{A}[\mathbf{R}_0 \mid \cdots \mid \mathbf{R}_{\ell-1}] + (\mathbf{x}^* - \mathbf{x})^\top \otimes \mathbf{G}$, it follows that $\mathbf{T}_i = [-\mathbf{R}_i^\top \mid \mathbf{0}_{m' \times im'} \mid \mathbf{I}_{m'} \mid \mathbf{0}_{m' \times (\ell-1-i)m'}]^\top$ is a $\mathbf{G}$ -trapdoor [MP12] for the matrix $\mathbf{A}_{\mathbf{x}} := [\mathbf{A} \mid \mathbf{B} - \mathbf{x}^\top \otimes \mathbf{G}]$. In addition, $s_1(\mathbf{T}_i) \leq \sqrt{\theta_1^2 + 1}$ by Lemma A.2.
 2. For Add queries, replace step 3 of the Add algorithm with $\mathbf{s}_{\mathbf{x}} \leftarrow \text{SampleRight}(\mathbf{A}_{\mathbf{x}}, \mathbf{T}_i, \mathbf{c}, \mathfrak{s}; r)$ and for MemWitCreate queries, replace step 6 of MemWitCreate algorithm with $\mathbf{s}_{\mathbf{x}} \leftarrow \text{SampleRight}(\mathbf{A}_{\mathbf{x}}, \mathbf{T}_i, \mathbf{c}, \mathfrak{s}; r)$.

Since $\mathfrak{s} \geq \max\{s_1(\mathbf{T}_{\mathbf{A}}), s_1(\mathbf{T}_i)\} \cdot \mathfrak{b} \cdot \log \sqrt{nd}$, Lemma 2.6 implies that $\text{Hyb}_8 \stackrel{s}{\approx} \text{Hyb}_7$.

- Hyb₉ : Similar to Hyb₈. However, during the execution of Gen, the challenger samples $\mathbf{A} \leftarrow R_q^{n \times m}$.
Hyb₈ $\approx$ Hyb₉ by Theorem 2.1. In addition, by Lemma C.1, $\Pr[E_9] = \text{negl}(\lambda)$ under M-SIS _{n, m, q, β_1} , where E_9 is the probability that $\mathcal{A}$ wins in Hyb₉.

Lemma C.1. *Under M-SIS _{n, m, q, β_1} , $\Pr[E_9] = \text{negl}(\lambda)$.*

Proof. We proceed by contraposition. In Hyb₉, suppose $\mathcal{A}$ outputs $\hat{\mathbf{s}}_{\mathbf{x}^*}$ such that $\|\hat{\mathbf{s}}_{\mathbf{x}^*}\| \leq \beta_0$ and $[\mathbf{A} \mid \mathbf{B} - (\mathbf{x}^*)^\top \otimes \mathbf{G}] \cdot \hat{\mathbf{s}}_{\mathbf{x}^*} = \mathbf{c}$, where $\mathbf{c}$ is the current value of $\mathcal{A}$. We show how a ppt algorithm $\mathcal{B}$ can recover an ℓ -succinct M-SIS solution for the matrix $\mathbf{A}$ by using $\mathcal{A}$ as follows:

Without loss of generality, we consider that $\mathcal{A}$ always issues a valid Add query for $\mathbf{x}^*$. From Hyb₃ and Hyb₄, $[\mathbf{A} \mid \mathbf{B} - (\mathbf{x}^*)^\top \otimes \mathbf{G}] = \mathbf{A}[\mathbf{I}_m \mid \mathbf{R}_0 \mid \cdots \mid \mathbf{R}_{\ell-1}]$ with $\mathbf{R}_i \leftarrow (S_1^\infty)^{m \times m'}$, and $\mathbf{u} = \mathbf{A}\mathbf{r}$, with $\mathbf{r} \sim \mathcal{B}_1^m$. Also, following Hyb₇, there exist $\mathbf{s}_{\mathbf{x}^*,0} \sim D_{R, \mathbf{s}_1}^m$, $\mathbf{s}_{\mathbf{x}^*,1} \sim D_{R, \mathbf{s}_1}^{\ell m'}$ and $\tilde{\mathbf{h}} \in R_q^{\ell m'}$, with $\|\tilde{\mathbf{h}}\| \leq T$, such that

$$\mathbf{A} \cdot [\mathbf{I} \mid \mathbf{R}_0 \mid \cdots \mid \mathbf{R}_{\ell-1}] \cdot \begin{bmatrix} \mathbf{s}_{\mathbf{x}^*,0} \\ \mathbf{s}_{\mathbf{x}^*,1} + \tilde{\mathbf{h}} \end{bmatrix} = \mathbf{c} - \mathbf{u}$$

which implies that

$$\begin{aligned} & \mathbf{A} \cdot [\mathbf{I} \mid \mathbf{R}_0 \mid \cdots \mid \mathbf{R}_{\ell-1}] \left(\begin{bmatrix} \mathbf{s}_{\mathbf{x}^*,0} \\ \mathbf{s}_{\mathbf{x}^*,1} + \tilde{\mathbf{h}} \end{bmatrix} - \hat{\mathbf{s}}_{\mathbf{x}^*} \right) = -\mathbf{u} \\ \implies & \underbrace{\mathbf{A} \left([\mathbf{I} \mid \mathbf{R}_0 \mid \cdots \mid \mathbf{R}_{\ell-1}] \left(\begin{bmatrix} \mathbf{s}_{\mathbf{x}^*,0} \\ \mathbf{s}_{\mathbf{x}^*,1} + \tilde{\mathbf{h}} \end{bmatrix} - \hat{\mathbf{s}}_{\mathbf{x}^*} \right) + \mathbf{r} \right)}_{\mathbf{v}} = \mathbf{0} \end{aligned}$$

Therefore $\mathbf{A}\mathbf{v} = \mathbf{0}$ and $\mathbf{v} \neq \mathbf{0}$ given that $\mathbf{r}$ was sampled uniformly at random from $\mathcal{B}_1^m$. In addition,

$$\begin{aligned} \|\mathbf{v}\| & \leq \left\| [\mathbf{I} \mid \mathbf{R}_0 \mid \cdots \mid \mathbf{R}_{\ell-1}] \left(\begin{bmatrix} \mathbf{s}_{\mathbf{x}^*,0} \\ \mathbf{s}_{\mathbf{x}^*,1} + \tilde{\mathbf{h}} \end{bmatrix} - \hat{\mathbf{s}}_{\mathbf{x}^*} \right) \right\| + \|\mathbf{r}\| \\ & \leq \|\mathbf{s}_{\mathbf{x}^*,0}\| + \|\mathbf{s}_{\mathbf{x}^*,1} + \tilde{\mathbf{h}}\|_{s_1}([\mathbf{R}_0 \mid \cdots \mid \mathbf{R}_{\ell-1}]) \\ & \quad + s_1([\mathbf{I} \mid \mathbf{R}_0 \mid \cdots \mid \mathbf{R}_{\ell-1}])\|\hat{\mathbf{s}}_{\mathbf{x}^*}\| + \|\mathbf{r}\| \\ & \leq s\sqrt{md} + (s\sqrt{\ell m' d} + T)\theta_2 + \beta_0\sqrt{1 + \theta_2^2} + \sqrt{md} \\ & = \beta_1 \end{aligned}$$

The last inequality follows from lemmas 2.2, 2.3, 4.1, and A.2. □

This completes the proof □

D Lattice-Based Commit-and-Prove Argument

In this section we instantiate a commit-and-prove argument for the positive dynamic accumulator in Construction 4.1. Concretely, we show how to prove knowledge of the revocation attribute $\mathbf{x} \in R_q^\ell$, the membership witness $\mathbf{s}_{\mathbf{x}} \in R_q^{m+\ell m'}$, and an additional short vector $\mathbf{r} \in R_q^\ell$, which in the context of anonymous

credentials will contain the user's attributes, such that:

$$\begin{cases} \mathbf{x} \in \{0, 1\}^\ell \wedge \|\mathbf{s}_\mathbf{x}\| \leq \beta \wedge \mathbf{r} \in \mathcal{B}_1^\ell \\ \mathbf{A}_1 \mathbf{x} + \mathbf{A}_2 \mathbf{r} = \mathbf{t} \\ [\mathbf{A} \mid \mathbf{B} - \mathbf{x}^\top \otimes \mathbf{G}] \cdot \mathbf{s}_\mathbf{x} = \mathbf{c} \end{cases} \quad (\text{D.1})$$

where $\mathbf{A}_1, \mathbf{A}_2, \mathbf{A}, \mathbf{B}, \mathbf{c}, \mathbf{t}$ are all public.

D.1 Modifying the Commit-and-Prove Relation

We first need to adapt the relation in Equation D.1 to the language supported by the LNP framework. We start by binary-decomposing the vector $\mathbf{s}_\mathbf{x} \in R_q^{m+\ell m'}$ to obtain a vector $\bar{\mathbf{s}}_\mathbf{x} \in \mathcal{B}_1^{(m+\ell m') \cdot b}$ where $b = \lceil \log \beta \rceil$. Then, the third line in Equation D.1 is equivalent to

$$[\mathbf{A} \mid \mathbf{B} - \mathbf{x}^\top \otimes \mathbf{G}] \mathbf{G}_\beta \cdot \bar{\mathbf{s}}_\mathbf{x} = \mathbf{c} \text{ where } \mathbf{G}_\beta = \mathbf{I}_{m+\ell m'} \otimes \begin{bmatrix} 1 & 2 & \dots & 2^{b-1} \end{bmatrix}.$$

Finally, we make the third modification using the result from [ESZ22, Corollary 3] generalized to $q = p_1 \cdots p_t$, where the p_i 's are distinct primes (cf. Lemma 2.4).

Lemma D.1 ([ESZ22]). *Let q be a sufficiently large integer that satisfies Lemma 2.4. Then, for any $a \in R_q$, $a \in \{0, 1\}$ if and only if $a \in \mathcal{B}_1$ and $a \cdot (a - 1) = 0$.*

Thus, proving the relations in Equation D.1 is equivalent to proving knowledge of $\mathbf{x} \in R_q^\ell, \mathbf{r} \in R_q^\ell$ and $\bar{\mathbf{s}}_\mathbf{x} \in R_q^{(m+\ell m') \cdot b}$ such that

$$\begin{cases} (\mathbf{x}, \mathbf{r}, \bar{\mathbf{s}}_\mathbf{x}) \in \mathcal{B}_1^{\ell + \ell + (m+\ell m') \cdot b} \\ \mathbf{A}_1 \mathbf{x} + \mathbf{A}_2 \mathbf{r} = \mathbf{t} \\ [\mathbf{A} \mid \mathbf{B} - \mathbf{x}^\top \otimes \mathbf{G}] \mathbf{G}_\beta \cdot \bar{\mathbf{s}}_\mathbf{x} = \mathbf{c} \\ x_i \cdot (x_i - 1) = 0 \quad \text{for } i = 1, 2, \dots, \ell. \end{cases} \quad (\text{D.2})$$

Relaxed extraction. A careful reader may notice that in the extraction one obtains a witness $\mathbf{s}_\mathbf{x}$ such that $\|\mathbf{s}_\mathbf{x}\| \leq \beta \sqrt{(m + \ell m')d}$ instead of just $\|\mathbf{s}_\mathbf{x}\| \leq \beta$. This is still fine for the security of the accumulator as long as other parameters are scaled appropriately (similarly as in the lattice-based identification schemes [Lyu12]). The reason we decided not to go with exact norm proofs is that the crucial step in there is proving that $\langle \vec{s}_\mathbf{x}, \vec{s}_\mathbf{x} \rangle \bmod q = \beta$, where $\vec{s}_\mathbf{x}$ is the coefficient vector of $\mathbf{s}_\mathbf{x}$ ⁵. To this end, [LNP22a] shows that $\vec{s}_\mathbf{x}$ is somewhat short and $\langle \vec{s}_\mathbf{x}, \vec{s}_\mathbf{x} \rangle = \beta \bmod q$. Then, to argue that the equation holds over integers, one has to make sure that no modulo overflow occurs, and in particular $\beta^2 \ll q$. Unfortunately, in our instantiation (Table 2), β is already big, and hence one would need a huge modulus q to accommodate for proving exact norm bounds, thus likely being more inefficient.

D.2 LNP Layer

D.2.1 Background

Notation. For a polynomial $x \in R_q$, we define $\vec{x}$ (resp. $\tilde{x}$) to be the coefficient vector of x (resp. the constant coefficient of x). The definition extends naturally to a polynomial vector. Let $\sigma : R \rightarrow R$ be the ring automorphism defined by $\sigma(X) := X^{-1}$. For a polynomial vector (and similarly for a matrix) $\mathbf{y} =$

⁵We are actually interested in the inequality. However, by padding with additional coefficients via sum-of-four-squares theorem, one makes it an equality.

d	κ	ν	k	$ \mathcal{C} $
64	8	140	32	2^{129}
128	2	59	32	2^{147}

Figure 2: Example parameters to instantiate $\mathcal{C}$ for a modulus q from [BLNS23b].

$(y_1, \dots, y_n) \in R^n$, define $\sigma(\mathbf{y}) := (\sigma(y_1), \dots, \sigma(y_n))$. Further, let $\tau \in \mathbb{N}$ be such that $1/q^\tau = \text{negl}(\lambda)$, i.e. a parameter used for soundness amplification.

Inner products and ring relations. We are interested in the function σ due to the following property. Namely, for any $a, b \in R$, the constant coefficient of $a \cdot \sigma(b)$ is equal to $\langle \vec{a}, \vec{b} \rangle$. Then, using the fact that a vector $\vec{b}$ has binary coefficients if and only if $\langle \vec{b}, \vec{b} - \vec{1} \rangle = 0$ over integers, we conclude that a polynomial $b \in \mathcal{B}_1$ if and only if the constant coefficient of

$$b \cdot \left(b - \sum_{i=0}^{d-1} X^i \right) \text{ is equal to 0.}$$

Therefore, we translated proving binary coefficients into proving that the constant term of a certain ring element equals zero.

ABDLOP Commitment. We recall the ABDLOP commitment defined in the LNP framework. Let $\bar{n}, \bar{m}_1, \bar{m}_2, \bar{\ell} \in \mathbb{N}$. The commitment key consists of uniformly random matrices $\text{crs} := (\bar{\mathbf{A}}_1, \bar{\mathbf{A}}_2, \bar{\mathbf{B}}) \in R_q^{\bar{n} \times \bar{m}_1} \times R_q^{\bar{n} \times \bar{m}_2} \times R_q^{\bar{\ell} \times \bar{m}_2}$. Now, to commit to a message vector $\mathbf{s}_1 \in R_q^{\bar{m}_1}$ with small coefficients as well as a “full-fledged” polynomial vector $\mathbf{m} \in R_q^{\bar{\ell}}$, we sample a randomness vector $\mathbf{s}_2 \leftarrow \mathcal{B}_1^{\bar{m}_2}$ and compute:

$$\begin{bmatrix} \mathbf{t}_A \\ \mathbf{t}_B \end{bmatrix} := \begin{bmatrix} \bar{\mathbf{A}}_1 \\ \mathbf{0} \end{bmatrix} \mathbf{s}_1 + \begin{bmatrix} \bar{\mathbf{A}}_2 \\ \bar{\mathbf{B}} \end{bmatrix} \mathbf{s}_2 + \begin{bmatrix} \mathbf{0} \\ \mathbf{m} \end{bmatrix}.$$

When $\bar{\ell} = 0$ (resp. $\bar{m}_1 = 0$) then this construction ends up being the Ajtai (resp. BDLOP) commitment scheme. In particular, the commitment size does not depend on the length $\bar{m}_1$ of $\mathbf{s}_1$ (but it does on $\bar{\ell}$). The strategy is to commit to long vectors with small coefficients in the “Ajtai” part $\mathbf{s}_1$ and commit to a few *garbage* polynomials used for the proofs in the “BDLOP” part $\mathbf{m}$ which will be appended on the go during the execution of the protocol.

Similarly as for the Ajtai commitment above, binding holds under the Module-SIS assumption on the matrix $[\bar{\mathbf{A}}_1 \parallel \bar{\mathbf{A}}_2]$ while hiding is shown via arguing that $\begin{bmatrix} \bar{\mathbf{A}}_2 \\ \bar{\mathbf{B}} \end{bmatrix} \mathbf{s}_2$ is indistinguishable from random for suitably chosen parameters.

Challenge space. Let us fix $\kappa, \nu > 0$ and a power-of-two k and define the following set $\mathcal{C}$ as:

$$\mathcal{C} := \left\{ c \in R_q : \|c\|_\infty \leq \kappa \wedge \sigma(c) = c \wedge \sqrt[2k]{\|c^{2k}\|_1} \leq \nu \right\}. \quad (\text{D.3})$$

This will be one of the key challenge spaces used by the verifier. Roughly speaking, the condition $\sigma_{-1}(c) = c$ is needed to prove quadratic equations in the committed messages which might additionally involve automorphisms. On the other hand, the last condition allows us to use [LNP22b, Lemma 2.15] and deduce that if $\|\mathbf{r}\| \leq \beta$ and $c \in \mathcal{C}$ then $\|\mathbf{c}\mathbf{r}\| \leq \nu\beta$. We will choose the constant ν such that (experimentally) the probability for $c \leftarrow S_\kappa$ to satisfy $\sqrt[2k]{\|c^{2k}\|_1} \leq \nu$ is at least 99%. In Figure 2 we show example parameters (v, ν, k) and the lower-bound on the size of $\mathcal{C}$.

For security of our protocols, we need the invertibility property of the set $\mathcal{C}$, i.e. the difference of any two distinct elements of $\mathcal{C}$ is invertible over R_q . Hence, we apply [LNP22b, Lemma 2.6] and thus we only need the condition $\kappa < q/2$. Secondly, to achieve negligible soundness error, we will need $|\mathcal{C}|$ to be exponentially large.

Approximate Range Proofs. We use the results developed in [GHL22, LNP22b] to upper-bound the operator norm of a matrix $\mathfrak{R} \leftarrow \text{Bin}_\xi^{256 \times m}$. Here, Bin denotes the binomial distribution with a positive integer parameter ξ , which is the distribution $\sum_{i=1}^\xi (a_i - b_i)$, where $a_i, b_i \leftarrow \{0, 1\}$. The variance of this distribution is $\xi/2$ and it holds that $\text{Bin}_{\xi_1} \pm \text{Bin}_{\xi_2} = \text{Bin}_{\xi_1 + \xi_2}$.

The bound on the operator norm of $\mathfrak{R}$ is used for the *approximate range proof* part, where we want to show that a vector $\mathbf{w}$ has relatively small coefficients w.r.t. to the proof system modulus q . Namely, we use the following (heuristic) results:

$$\begin{aligned} \Pr_{\mathfrak{R} \leftarrow \text{Bin}_\xi^{256 \times m}} [\|\mathfrak{R}\vec{w}\|^2 < \xi \cdot \|\vec{w}\|^2 \cdot 13] &\gtrsim 2^{-256}, \\ \Pr_{\mathfrak{R} \leftarrow \text{Bin}_\xi^{256 \times m}} [\|\mathfrak{R}\vec{w}\|^2 > \xi \cdot \|\vec{w}\|^2 \cdot 337] &\gtrsim 2^{-128}. \end{aligned}$$

for any $\vec{w} \in \mathbb{Z}^m$. In the soundness argument, we use the following result from [LNP22b, Lemma 2.9]:

Lemma D.2. Fix $m, q \in \mathbb{N}$ and a bound $\beta \leq q/(8\sqrt{2} \cdot \sqrt{13} \cdot m)$, and let $\vec{w} \in [\pm q/2]^m$ with $\|\vec{w}\| \geq \beta$, and let $\vec{y}$ be an arbitrary vector in $[\pm q/2]^m$. Then

$$\Pr_{\mathfrak{R} \leftarrow \text{Bin}_1^{256 \times m}} [\|\mathfrak{R}\vec{w} + \vec{y} \bmod q\| < \beta \cdot \sqrt{13/2}] < 2^{-128}.$$

Informally, the role of the approximate range proof is as follows. Given a challenge $\mathfrak{R}$ from the verifier, the prover will output $\vec{z} = \mathfrak{R}\vec{w} + \vec{y}$, where the role of $\vec{y}$ is to mask $\mathfrak{R}\vec{w}$ for zero-knowledge. The verifier can manually check that $\vec{z}$ has small coefficients. Then, in the knowledge soundness argument, an extractor manages to extract a candidate witness $\mathbf{w}$ with the coefficient vector $\vec{w}$. If coefficients of $\vec{w}$ were large, then with high probability $\vec{z} = \mathfrak{R}\vec{w} + \vec{y}$ would also have been large by Lemma D.2. But one of the verification checks was that $\vec{z}$ is indeed short. Hence, we conclude that $\vec{w}$ is short as well.

D.2.2 LNP Protocol

We are ready to provide description of the LNP protocol for proving relations in Equation D.2. For presentation, write the protocol in an interactive form, which can be made non-interactive using the Fiat-Shamir transform [FS86].

First round. The prover starts by creating an ABDLOP commitment. To this end, let $(\bar{\mathbf{A}}_1, \bar{\mathbf{A}}_2, \bar{\mathbf{B}}_y, \bar{\mathbf{B}}_p, \bar{\mathbf{B}}_{\text{quad}})$ be uniformly random matrices. Let $\bar{m}_1 := \ell + l + (m + \ell m') \cdot b$ and define $\mathbf{t}_A := \bar{\mathbf{A}}_1 \mathbf{s}_1 + \bar{\mathbf{A}}_2 \mathbf{s}_2$ where

$$\mathbf{s}_1 = \begin{bmatrix} \mathbf{x} \\ \mathbf{r} \\ \bar{\mathbf{s}}_{\mathbf{x}} \end{bmatrix} \in R_q^{\bar{m}_1} \quad \text{and} \quad \mathbf{s}_2 \leftarrow \mathcal{B}_1^{\bar{m}_2}.$$

For zero-knowledge purposes, the prover samples $\mathbf{y}_1 \leftarrow D_{\mathbf{s}_1}^{\bar{m}_1}, \mathbf{y}_2 \leftarrow D_{\mathbf{s}_2}^{\bar{m}_2}$ and $\mathbf{y}_3 \leftarrow D_{\mathbf{s}_3}^{256/d}$ and computes

$$\mathbf{w} := \bar{\mathbf{A}}_1 \mathbf{y}_1 + \bar{\mathbf{A}}_2 \mathbf{y}_2 \in R_q^{\bar{n}}, \quad \mathbf{t}_y := \bar{\mathbf{B}}_y \mathbf{s}_2 + \mathbf{y}_3 \in R_q^{256/d}.$$

The terms $\mathbf{y}_1, \mathbf{y}_2$ will be related to $\mathbf{s}_1$ and $\mathbf{s}_2$ respectively, while $\mathbf{y}_3$ will be used for the approximate range proof. The prover then sends $(\mathbf{t}_A, \mathbf{w}, \mathbf{t}_y)$. In return, the verifier sends a challenge $\mathfrak{R} \leftarrow \text{Bin}_1^{256 \times \bar{m}_1}$.

Third round. The prover computes $\vec{z}_3 := \mathfrak{R} \vec{s}_1 + \vec{y}_3 \in \mathbb{Z}^{256}$ and performs a standard rejection sampling procedure before outputting $\vec{z}_3$.

We explain the motivation for the next steps. Firstly, we need to prove that $\vec{z}_3$ is well-formed. To this end, for $j = 1, \dots, 256$, define $\mathbf{r}_j \in R_q^{\bar{m}_1}$ be the polynomial vector with the coefficient vector being the j -th row of the matrix $\mathfrak{R}$. Also, define $\mathbf{e}_j \in R_q^{256/d}$ to be the polynomial vector with coefficient vector $(0, \dots, 0, 1, 0, \dots, 0)$ with the 1-entry in the j -th position. Then, using the automorphism property, we want to show that for all $j = 1, \dots, 256$, the constant coefficient of

$$\sigma(\mathbf{r}_j)^\top \mathbf{s}_1 + \mathbf{e}_j^\top \mathbf{y}_3 \quad \text{equals to } z_{3,j} \quad (\text{D.4})$$

So far, the approximate range proof allows to prove that the vector $\mathbf{s}_1$ has somewhat small coefficients. However, we wanted to prove that $\mathbf{s}_1 := (s_{1,1}, \dots, s_{1,\bar{m}_1}) \in \mathcal{B}_1^{\bar{m}_1}$. To this end, we will prove $\langle \vec{s}_1, \vec{s}_1 - \vec{1} \rangle = 0 \pmod{q}$. Note that for a large enough modulus q , this implies $\langle \vec{s}_1, \vec{s}_1 - \vec{1} \rangle = 0$ over integers due to no modulo wrap-around. Finally, using the σ automorphism, this is equivalent to proving that the constant coefficient of

$$\sum_{i=1}^{\bar{m}_1} s_{1,i} \cdot \sigma \left(s_{1,i} - \sum_{j=0}^{d-1} X^j \right) \quad \text{equals 0.} \quad (\text{D.5})$$

Revealing the sums in Equations D.4 and D.5 in the clear could potentially reveal some information about $\mathbf{s}_1$. Hence, we will mask all the other coefficients of the sums apart from the constant one by sampling $\mathbf{p} := (p_1, \dots, p_\tau) \leftarrow \{u \in R_q : \tilde{u} = 0\}$ and computing the corresponding commitment:

$$\mathbf{t}_p := \bar{\mathbf{B}}_p \mathbf{s}_2 + \mathbf{p} \in R_q^\tau.$$

Eventually, conditioned on the rejection sampling procedure not aborting, the prover outputs $(\vec{z}_3, \mathbf{t}_p)$. The verifier responds by outputting $(\alpha_{i,j}) \leftarrow \mathbb{Z}_q^{\tau \times 257}$.

Fifth round. The prover outputs the masked openings $(h_1, \dots, h_\tau) \in R_q^\tau$ of the sums in Equations D.4 and D.5 defined as:

$$h_i := p_i + \sum_{j=1}^{256} \alpha_{i,j} \cdot (\sigma(\mathbf{r}_j)^\top \mathbf{s}_1 + \mathbf{e}_j^\top \mathbf{y}_3) + \alpha_{i,257} \cdot \sum_{i=1}^{\bar{m}_1} s_{1,i} \cdot \sigma \left(s_{1,i} - \sum_{j=0}^{d-1} X^j \right).$$

Note that since each α_i is an integer and $\tilde{p}_i = 0$, the constant coefficient of h_i is equal to $\sum_{j=1}^{256} \alpha_{i,j} z_{3,j}$ – this can be manually checked by the verifier.

We provide motivation for the next steps. The prover now has to prove a bunch of quadratic relations (some of them involve the automorphism σ):

$$\mathbf{A}_1 \mathbf{x} + \mathbf{A}_2 \mathbf{r} = \mathbf{t}$$

$$[\mathbf{A} \mid \mathbf{B} - \mathbf{x}^\top \otimes \mathbf{G}] \mathbf{G}_\beta \cdot \bar{\mathbf{s}}_{\mathbf{x}} = \mathbf{c}$$

$$x_i \cdot (x_i - 1) = 0 \quad \text{for } i = 1, 2, \dots, \ell.$$

$$\forall i \in [\tau], h_i = p_i + \sum_{j=1}^{256} \alpha_{i,j} \cdot (\sigma(\mathbf{r}_j)^\top \mathbf{s}_1 + \mathbf{e}_j^\top \mathbf{y}_3) + \alpha_{i,257} \cdot \sum_{i=1}^{\bar{m}_1} s_{1,i} \cdot \sigma \left(s_{1,i} - \sum_{j=0}^{d-1} X^j \right)$$

where the terms in blue are secret and committed. To this end, the verifier outputs challenges $\gamma_1 \leftarrow R_q^t, \gamma_2 \leftarrow R_q^{m' \cdot n}, \gamma_3 \leftarrow R_q^\ell, \gamma_4 \leftarrow R_q^\tau$ which will reduce the task to proving a single quadratic equation.

Seventh round. The prover now has to prove:

$$\gamma_1^\top \mathbf{t} + \gamma_2^\top \mathbf{c} + \sum_{i=1}^{\tau} \gamma_{4,i} h_i = U_1 + U_2 + U_3 + U_4 \quad (\text{D.6})$$

where

$$\begin{aligned} U_1 &:= \gamma_1^\top (\mathbf{A}_1 \mathbf{x} + \mathbf{A}_2 \mathbf{r}) \\ U_2 &:= \gamma_2^\top [\mathbf{A} \mid \mathbf{B} - \mathbf{x}^\top \otimes \mathbf{G}] \mathbf{G}_\beta \cdot \bar{\mathbf{s}}_{\mathbf{x}} \\ U_3 &:= \sum_{i=1}^{\ell} \gamma_{3,i} \cdot x_i \cdot (x_i - 1) \\ U_4 &:= \sum_{i=1}^{\tau} \gamma_{4,i} \left(p_i + \sum_{j=1}^{256} \alpha_{i,j} \cdot \left(\sigma(\mathbf{r}_j)^\top \mathbf{s}_1 + \mathbf{e}_j^\top \mathbf{y}_3 \right) + \alpha_{i,257} \cdot \sum_{i=1}^{\bar{m}_1} s_{1,i} \cdot \sigma(s_{1,i} - \sum_{j=0}^{d-1} X^j) \right). \end{aligned}$$

To provide intuition on proving the single quadratic equation, let us fix some $c \in \mathcal{C}$ and denote $\mathbf{z}_i := \mathbf{y}_i + c\mathbf{s}_i$ for $i = 1, 2$ (which will ultimately be sent to the verifier). Then, let us parse $\mathbf{z}_1$ and $\mathbf{y}_1$ as:

$$\underbrace{\begin{bmatrix} \mathbf{z}_x \\ \mathbf{z}_r \\ \mathbf{z}_{sx} \end{bmatrix}}_{\mathbf{z}_1} = \underbrace{\begin{bmatrix} \mathbf{y}_x \\ \mathbf{y}_r \\ \mathbf{y}_{sx} \end{bmatrix}}_{\mathbf{y}_1} + c \cdot \underbrace{\begin{bmatrix} \mathbf{x} \\ \mathbf{r} \\ \bar{\mathbf{s}}_{\mathbf{x}} \end{bmatrix}}_{\mathbf{s}_1}.$$

Consider the term U_1 . Now, note that

$$c\gamma_1^\top (\mathbf{A}_1 \mathbf{z}_x + \mathbf{A}_2 \mathbf{z}_r) = c^2 \cdot U_1 + c \cdot \gamma_1^\top (\mathbf{A}_1 \mathbf{y}_x + \mathbf{A}_2 \mathbf{y}_r). \quad (\text{D.7})$$

Similarly,

$$\begin{aligned} &\gamma_2^\top [c\mathbf{A} \mid c\mathbf{B} - \mathbf{z}_x^\top \otimes \mathbf{G}] \mathbf{G}_\beta \cdot \mathbf{z}_{sx} \\ &= c^2 \cdot U_2 + c \cdot \gamma_2^\top \cdot ([\mathbf{A} \mid \mathbf{B} - \mathbf{x}^\top \otimes \mathbf{G}] \mathbf{G}_\beta \cdot \mathbf{y}_{sx} - [\mathbf{0} \mid \mathbf{y}_x^\top \otimes \mathbf{G}] \mathbf{G}_\beta \cdot \mathbf{s}_{\mathbf{x}}) \\ &+ [\mathbf{0} \mid -\mathbf{y}_x^\top \otimes \mathbf{G}] \mathbf{G}_\beta \cdot \mathbf{y}_{sx}. \end{aligned} \quad (\text{D.8})$$

As for U_3 , we have:

$$\sum_{i=1}^{\ell} \gamma_{3,i} \cdot z_{x,i} \cdot (z_{x,i} - c) = c^2 \cdot U_3 + c \cdot \left(\sum_{i=1}^{\ell} \gamma_{3,i} \cdot y_{x,i} \cdot (2x_i - 1) \right) + \left(\sum_{i=1}^{\ell} \gamma_{3,i} \cdot y_{x,i}^2 \right). \quad (\text{D.9})$$

Finally, as for U_4 , let us denote $\mathbf{e}'_i \in R_q^\tau$ to be the unit vector with 1-entry exactly on the i -th position for $i = 1, 2, \dots, \tau$ and $\text{PW} := (\sum_{j=0}^{d-1} X^j, \dots, \sum_{j=0}^{d-1} X^j) \in R_q^{\bar{m}_1}$. Then, we can rewrite U_4 as

$$U_4 = \sum_{i=1}^{\tau} \gamma_{4,i} \left(\mathbf{e}_i'^\top \mathbf{p} + \sum_{j=1}^{256} \alpha_{i,j} \cdot (\sigma(\mathbf{r}_j)^\top \mathbf{s}_1 + \mathbf{e}_j^\top \mathbf{y}_3) + \alpha_{i,257} \cdot \mathbf{s}_1^\top \sigma(\mathbf{s}_1 - \text{PW}) \right).$$

Then, using the property of the ABDLOP commitment that

$$\begin{bmatrix} c\mathbf{t}_y - \mathbf{B}_y \mathbf{z}_2 \\ c\mathbf{t}_p - \mathbf{B}_p \mathbf{z}_2 \end{bmatrix} = \begin{bmatrix} -\mathbf{B}_y \mathbf{y}_2 \\ -\mathbf{B}_p \mathbf{y}_2 \end{bmatrix} + c \cdot \begin{bmatrix} \mathbf{y}_3 \\ \mathbf{p} \end{bmatrix} \quad (\text{D.10})$$

and the fact that $\sigma(c) = c$, we have

$$\begin{aligned}
& \sum_{i=1}^{\tau} \gamma_{4,i} \left(c \cdot \mathbf{e}_i^{\top} (c \mathbf{t}_p - \mathbf{B}_p \mathbf{z}_2) + c \cdot \sum_{j=1}^{256} \alpha_{i,j} \cdot \left(\sigma(\mathbf{r}_j)^{\top} \mathbf{z}_1 + \mathbf{e}_j^{\top} (c \mathbf{t}_y - \mathbf{B}_y \mathbf{z}_2) \right) + \alpha_{i,257} \cdot \mathbf{z}_1^{\top} \sigma(\mathbf{z}_1 - c \cdot \text{PW}) \right) \\
& = c^2 \cdot U_4 \\
& + c \cdot \left(\sum_{i=1}^{\tau} \gamma_{4,i} \cdot (-\mathbf{B}_p \mathbf{y}_2) + \sum_{j=1}^{256} \alpha_{i,j} \cdot \left(\sigma(\mathbf{r}_j)^{\top} \mathbf{y}_1 - \mathbf{e}_j^{\top} (\mathbf{B}_y \mathbf{y}_2) \right) + \alpha_{i,257} \cdot (\mathbf{y}_1^{\top} \sigma(\mathbf{s}_1 - \text{PW}) + \mathbf{s}_1^{\top} \sigma(\mathbf{y}_1)) \right) \\
& + \mathbf{y}_1^{\top} \sigma(\mathbf{y}_1).
\end{aligned} \tag{D.11}$$

Crucially, if $\mathbf{z}_1, \mathbf{z}_2$ are sent, then the left-hand side of the Equations D.7, D.8, D.9 and D.11 can be explicitly computed by the verifier. Moreover, each of them can be written as quadratic functions in c where the quadratic coefficient is U_i for $i = 1, 2, 3, 4$. Consider the summation of Equations D.7, D.8, D.9 and D.11. Then, by Equation D.6, the right-hand side is of the form

$$c^2 \cdot \left(\gamma_1^{\top} \mathbf{t} + \gamma_2^{\top} \mathbf{c} + \sum_{i=1}^{\tau} \gamma_{4,i} h_i \right) + c \cdot g_1 + g_0$$

where

$$\begin{aligned}
g_1 := & \gamma_1^{\top} (\mathbf{A}_1 \mathbf{y}_x + \mathbf{A}_2 \mathbf{y}_r) + \gamma_2^{\top} \cdot ([\mathbf{A} \mid \mathbf{B} - \mathbf{x}^{\top} \otimes \mathbf{G}] \mathbf{G}_{\beta} \cdot \mathbf{y}_{sx} - [\mathbf{0} \mid \mathbf{y}_x^{\top} \otimes \mathbf{G}] \mathbf{G}_{\beta} \cdot \mathbf{s}_{\mathbf{x}}) \\
& + \left(\sum_{i=1}^{\ell} \gamma_{3,i} \cdot y_{x,i} \cdot (2x_i - 1) \right) + \left(\sum_{i=1}^{\tau} \gamma_{4,i} \cdot (-\mathbf{B}_p \mathbf{y}_2) + \sum_{j=1}^{256} \alpha_{i,j} \cdot (\sigma(\mathbf{r}_j)^{\top} \mathbf{y}_1 - \mathbf{e}_j^{\top} (\mathbf{B}_y \mathbf{y}_2)) \right)
\end{aligned}$$

and

$$g_0 := [\mathbf{0} \mid -\mathbf{y}_x^{\top} \otimes \mathbf{G}] \mathbf{G}_{\beta} \cdot \mathbf{y}_{sx} + \left(\sum_{i=1}^{\ell} \gamma_{3,i} \cdot y_{x,i}^2 \right) + \mathbf{y}_1^{\top} \sigma(\mathbf{y}_1)$$

To this end, the prover commits to g_1 as

$$t_{\text{quad}} := \bar{\mathbf{B}}_{\text{quad}} \mathbf{s}_2 + g_1.$$

Hence, the sum of Equations D.7, D.8, D.9 and D.11 can be rewritten as:

$$Z = c^2 \cdot \left(\gamma_1^{\top} \mathbf{t} + \gamma_2^{\top} \mathbf{c} + \sum_{i=1}^{\tau} \gamma_{4,i} h_i \right) + (c t_{\text{quad}} - \bar{\mathbf{B}}_{\text{quad}} \mathbf{z}_2) + \bar{\mathbf{B}}_{\text{quad}} \mathbf{y}_2 + g_0$$

where Z is public if $\mathbf{z}_1, \mathbf{z}_2$ are known. Having this strategy in mind, the prover outputs $(t_{\text{quad}}, v := \bar{\mathbf{B}}_{\text{quad}} \mathbf{y}_2 + g_0)$ and the verifier responds with a final challenge $c \leftarrow \mathcal{C}$.

Ninth round and verification equations. The prover computes $\mathbf{z}_i = \mathbf{y}_i + c \cdot \mathbf{s}_i$ for $i = 1, 2$ and applies rejection sampling for both $\mathbf{z}_1$ and $\mathbf{z}_2$. If no abort occurs, the prover outputs $\mathbf{z}_1, \mathbf{z}_2$. Finally, the verifier checks the following constraints:

$$\begin{aligned}
& \|\mathbf{z}_1\| \leq \beta_1, \quad \|\mathbf{z}_2\| \leq \beta_2, \quad \|\vec{z}_3\| \leq \beta_3 \\
& \bar{\mathbf{A}}_1 \mathbf{z}_1 + \bar{\mathbf{A}}_2 \mathbf{z}_2 = \mathbf{w} + c \mathbf{t}_A \\
& Z = c^2 \cdot \left(\gamma_1^{\top} \mathbf{t} + \gamma_2^{\top} \mathbf{c} + \sum_{i=1}^{\tau} \gamma_{4,i} h_i \right) + (c t_{\text{quad}} - \bar{\mathbf{B}}_{\text{quad}} \mathbf{z}_2) + v \\
& \tilde{h}_i = \sum_{j=1}^{256} \alpha_{i,j} z_{3,j} \quad \text{for } i = 1, \dots, \tau
\end{aligned} \tag{D.12}$$

where Z is explicitly defined as

$$Z := c\gamma_1^\top (\mathbf{A}_1 \mathbf{z}_x + \mathbf{A}_2 \mathbf{z}_r) + \gamma_2^\top [c\mathbf{A} \mid c\mathbf{B} - \mathbf{z}_x^\top \otimes \mathbf{G}] \mathbf{G}_\beta \cdot \mathbf{z}_{sx} + \sum_{i=1}^{\ell} \gamma_{3,i} \cdot z_{x,i} \cdot (z_{x,i} - c) \\ + \sum_{i=1}^{\tau} \gamma_{4,i} \left(c \cdot \mathbf{e}_i'^\top (\mathbf{c} \mathbf{t}_p - \mathbf{B}_p \mathbf{z}_2) + c \cdot \sum_{j=1}^{256} \alpha_{i,j} \cdot (\sigma(\mathbf{r}_j)^\top \mathbf{z}_1 + \mathbf{e}_j^\top (\mathbf{c} \mathbf{t}_y - \mathbf{B}_y \mathbf{z}_2)) \right. \\ \left. + \alpha_{i,257} \cdot \mathbf{z}_1^\top \sigma(\mathbf{z}_1 - c \cdot \text{PW}) \right).$$

D.2.3 Parameter Setting for Security

We briefly describe how to set parameters to satisfy completeness, (honest-verifier) zero-knowledge and knowledge consistency. Due to the proofs being described in prior works, e.g. [BLNS23b, LNP22b], we skip the formal treatment for presentation.

Completeness. First, we set the standard deviations $\mathfrak{s}_1, \mathfrak{s}_2, \mathfrak{s}_3$. For the rejection sampling purposes (c.f. [Lyu12, Theorem 4.6]), we set $\mathfrak{s}_i = 12 \cdot T_i$ for $i = 1, 2$, where T_i is an upper-bound on $\|\mathbf{c} \mathbf{s}_i\|$. Using the definition of the challenge space $\mathcal{C}$, we have $\|\mathbf{c} \mathbf{s}_i\| \leq \nu \cdot \|\mathbf{s}_i\|$. Hence, we can set $T_1 := \nu \cdot \sqrt{\bar{m}_1 \cdot d}$ and $T_2 := \nu \cdot \sqrt{\bar{m}_2 \cdot d}$. Similarly, we compute the bound (with an overwhelming probability) $\|\mathfrak{A} \vec{s}_1\| \leq \sqrt{337} \cdot \|\vec{s}_1\| \leq \sqrt{337} \cdot \bar{m}_1 \cdot d$ and set $\mathfrak{s}_3 = 12 \cdot \sqrt{337} \cdot \bar{m}_1 \cdot d$. Consequently, we set the norm bounds $\beta_1 := \mathfrak{s}_1 \sqrt{2\bar{m}_1 d}$, $\beta_2 := \mathfrak{s}_2 \sqrt{\bar{m}_2 \cdot d}$ and $\beta_3 = \mathfrak{s}_3 \sqrt{2} \cdot 256$.

Honest-verifier zero-knowledge. Apart from rejection sampling, we need to make sure that the ABDLOP commitment is hiding. This means showing that the following is indistinguishable from random:

$$\begin{bmatrix} \bar{\mathbf{A}}_2 \\ \bar{\mathbf{B}}_y \\ \bar{\mathbf{B}}_g \\ \bar{\mathbf{B}}_{\text{quad}} \end{bmatrix} \mathbf{s}_2.$$

For simplicity, we will rely on the Leftover Hash Lemma over rings [BJRW20, Lemma 7]. To this end, we set $\bar{m}_2 \geq (\bar{n} + 256/d + \tau + 1) \log q + \omega(\log d)$.

Knowledge soundness. First, we need to make sure that the ABDLOP commitment satisfies so-called relaxed binding. To this end, Module-SIS problem for the matrix $[\bar{\mathbf{A}}_1 \mid \bar{\mathbf{A}}_2]$ should be hard w.r.t. the norm bound $4\nu \sqrt{\beta_1^2 + \beta_2^2}$ [BLNS23b, Lemma 5.2]. Next, to make use of Lemma D.2, we need $q \geq \frac{\beta_3}{\sqrt{13/2}} \cdot (8\sqrt{2} \cdot \sqrt{13} \cdot \bar{m}_1 \cdot d) \geq 16\beta_3 \cdot \bar{m}_1 \cdot d$. Then, via the approximate range proof, we can argue that the extracted $\mathbf{s}_1$ satisfies $\|\vec{s}_1\| \leq \beta_3 \cdot \sqrt{2/13}$. Finally, we need to make sure $\langle \vec{s}_1, \vec{s}_1 - \vec{1} \rangle = 0$ over integers by showing no modulo overflow occurs. Here, we can use a bound $\|\vec{s}_1\|_1 \leq \|\vec{s}_1\|_2 \cdot \sqrt{\bar{m}_1 \cdot d}$. Then, let us set $q > \beta_3^2 \cdot 2/13 + \beta_3 \cdot \sqrt{2\bar{m}_1 \cdot d}/13$ which satisfies:

$$\begin{aligned} |\langle \vec{s}_1, \vec{s}_1 - \vec{1} \rangle| &\leq \|\vec{s}_1\|^2 + \|\vec{s}_1\|_1 \\ &\leq \|\vec{s}_1\|^2 + \|\vec{s}_1\|_2 \cdot \sqrt{\bar{m}_1 \cdot d} < q. \end{aligned}$$

Finally, for invertibility of any difference of distinct challenges in $\mathcal{C}$ and to argue $\mathbf{x} \in \{0, 1\}^\ell$, we need $q > \max(8, \kappa/2)$. The knowledge error becomes $\approx 2^{-128} + q^{-\tau} + q^{-d/2} + 2/|\mathcal{C}|$.

D.2.4 Possible Optimizations.

We already highlighted that for efficiency, one may pick lower value for $\bar{m}_2$ by relying on the Module-LWE assumption. Lyubashevsky and Nguyen [LN22] showed how to make use of bimodal rejection sampling to further reduce the standard deviations s_1 and s_3 at the cost of committing to an additional sign $\{-1, 1\}$. Moreover, one can remove the rejection sampling procedure for s_2 using the Hint-LWE approach by Kim et al. [KLSS23] which should considerably improve the protocol runtime. Further, we can apply the trick from [LNP22b, Section 4.4] which reduces the number of garbage polynomials in $\mathbf{p}$ from τ to $\tau/2$.

Finally, using the aforementioned methodology we can reduce the size of $\mathbf{z}_x$ by a factor of two. Namely, suppose that ℓ is even. Then, instead of committing to $\mathbf{x} = (x_1, \dots, x_\ell) \in \{0, 1\}^\ell$ and spending a whole ring element just for a bit in $\{0, 1\}$, we commit to $\mathbf{x}' = (x'_1, \dots, x'_{\ell/2})$ where

$$x'_i := x_{2i-1} + x_{2i}X^{d/2} \in R_q \quad \text{for } i = 1, 2, \dots, \ell/2.$$

Then, observe that for all $i = 1, \dots, \ell/2$ we have:

$$(x'_i + \sigma(x'_i))/2 = x_{2i-1} \quad \text{and} \quad (x'_i - \sigma(x'_i))/(2X^{d/2}) = x_{2i}.$$

In particular, we can write

$$\mathbf{x} = \left(\begin{bmatrix} 2^{-1} \\ (2X^{d/2})^{-1} \end{bmatrix} \otimes \mathbf{I}_{\ell/2} \right) \mathbf{x}' + \left(\begin{bmatrix} 2^{-1} \\ -(2X^{d/2})^{-1} \end{bmatrix} \otimes \mathbf{I}_{\ell/2} \right) \sigma(\mathbf{x}').$$

Hence, the equation

$$[\mathbf{A} \mid \mathbf{B} - \mathbf{x}^\top \otimes \mathbf{G}] \mathbf{G}_\beta \cdot \bar{\mathbf{s}}_x = \mathbf{c}$$

becomes a quadratic relation (with automorphism σ) in $\bar{\mathbf{s}}_x$ and $\mathbf{x}'$. This means that now we can commit to $\mathbf{x}'$ instead of $\mathbf{x}$. Then, one has to prove the quadratic equation above, together with the prove wellformedness of each $x'_i := x_{2i-1} + x_{2i}X^{d/2}$. This can be done by proving simultaneously that (i) $x'_i \in \mathcal{B}_1$, (ii) $\langle \vec{x}'_i, \vec{1} \rangle = 2$, (iii) constant coefficient of x'_i is 1, and (iv) constant coefficient of $X^{-d/2} \cdot x_i$ is 1 – all of which are supported by the LNP framework.

D.3 LaBRADOR Layer

We note that the most expensive part of the LNP protocol above is the last message sent by the prover: $(\mathbf{z}_1, \mathbf{z}_2)$. To circumvent this efficiency bottleneck, we use the LaBRADOR succinct proof system to prove knowledge of $\mathbf{z}_1 \in R_q^{\bar{m}_1}, \mathbf{z}_2 \in R_q^{\bar{m}_2}$ which satisfy the verification equations in Equation D.12.

Translating verification checks into the principal relation. First, we recall the principal relation, the native language for LaBRADOR defined by Beullens and Serier in [BS23]. The relation is characterised by the rank $n' \geq 1$, multiplicity $r' \geq 1$ and the norm bound $\beta' > 0$. The statement is a triple $(\mathcal{F}, \mathcal{F}', \beta')$, where $\mathcal{F}$ and $\mathcal{F}'$ are families of functions $f : R_q^{n'} \times \dots \times R_q^{n'} \rightarrow R_q$ of the form

$$f(\bar{\mathbf{s}}_1, \dots, \bar{\mathbf{s}}_{r'}) = \sum_{i,j=1}^{r'} a_{i,j} \langle \bar{\mathbf{s}}_i, \bar{\mathbf{s}}_j \rangle + \sum_{i=1}^{r'} \langle \phi_i, \bar{\mathbf{s}}_i \rangle - b$$

for $a_{i,j}, b \in R_q$ and $\phi_i \in R_q^{n'}$. Then, a valid witness is a sequence of vectors $\bar{\mathbf{s}}_1, \dots, \bar{\mathbf{s}}_{r'} \in R_q^n$ which satisfy:

$$\begin{aligned} f(\bar{\mathbf{s}}_1, \dots, \bar{\mathbf{s}}_{r'}) &= 0 \quad \forall f \in \mathcal{F} \\ \text{constant coeff. of } (f'(\bar{\mathbf{s}}_1, \dots, \bar{\mathbf{s}}_{r'})) &= 0 \quad \forall f' \in \mathcal{F}' \\ \sum_{i=1}^{r'} \|\bar{\mathbf{s}}_i\|^2 &\leq \beta'^2. \end{aligned}$$

In our setting, we will keep $\mathbf{z}_r, \mathbf{z}_{sx}, \mathbf{z}_2$ as part of the witness while we send $\mathbf{z}_x \in R_q^\ell$ in the clear (we will provide the reasoning later). Then, looking at the Equation D.12, we note that the verification checks are almost of the form of principal relation apart from the following two constraints. The first one is that the principal relation does not natively support quadratic relations with automorphisms (see the third equation which includes the term $\mathbf{z}_r^\top \sigma(\mathbf{z}_r) + \mathbf{z}_{sx}^\top \sigma(\mathbf{z}_{sx})$). To circumvent this issue, we additionally append new vectors $\mathbf{z}'_r := \sigma(\mathbf{z}_r), \mathbf{z}'_{sx} = \sigma(\mathbf{z}_{sx})$ to the witness and prove its well-formedness. Since the map σ is simply a signed permutation on the coefficients of the polynomial vector, there exists a public matrix $M \in \mathbb{Z}_q^{ld \times ld}$ such that $\vec{z}'_r = M \vec{z}_r \pmod{q}$. Hence, proving $\mathbf{z}'_r = \sigma(\mathbf{z}_r)$ is equivalent to proving for each $i = 1, \dots, ld$, the constant coefficient of

$$\sigma(\mathbf{e}_i^\dagger)^\top \mathbf{z}'_r - \sigma(\mathbf{m}_i)^\top \mathbf{z}_r \quad \text{is equal to } 0 \quad (\text{D.13})$$

where $\mathbf{m}_i$ (resp. $\mathbf{e}_i^\dagger$) is the polynomial vector with the coefficient vector being the i -th row of M (resp. the unit vector $(0, \dots, 0, 1, 0, \dots, 0)$ with the 1-entry in the i -th position). Similar reasoning can be applied to $\mathbf{z}_{sx}$.

The second issue is that the original principal relation only considers a single norm bound on the whole witness vector, rather than multiple bounds. To this end, we focus on a relaxed relation $\|(\mathbf{z}_r, \mathbf{z}'_r, \mathbf{z}_{sx}, \mathbf{z}'_{sx}, \mathbf{z}_2)\| \leq \sqrt{2\beta_1 + \beta_2}$. To sum up, by setting the witness vector as $(\mathbf{z}_r, \mathbf{z}'_r, \mathbf{z}_{sx}, \mathbf{z}'_{sx}, \mathbf{z}_2) \in R_q^{2(l+(m+\ell m) \cdot b) + \bar{m}_2}$, we can rewrite the second check of Equation D.12 as

$$\mathbf{A}' \begin{bmatrix} \mathbf{z}_r \\ \mathbf{z}'_r \\ \mathbf{z}_{sx} \\ \mathbf{z}'_{sx} \\ \mathbf{z}_2 \end{bmatrix} = \mathbf{t}'$$

for some public $\mathbf{A}'$ and $\mathbf{t}'$. The third equation, on the other hand, can be rewritten as:

$$\mathbf{u}'^\top \begin{bmatrix} \mathbf{z}_r \\ \mathbf{z}'_r \\ \mathbf{z}_{sx} \\ \mathbf{z}'_{sx} \\ \mathbf{z}_2 \end{bmatrix} + \langle \mathbf{z}_r, \mathbf{z}'_r \rangle + \langle \mathbf{z}_{sx}, \mathbf{z}'_{sx} \rangle = t''$$

for some public $\mathbf{u}'$ and t'' . Combined with the relations in Equation D.13 for $\mathbf{z}_r$ and $\mathbf{z}_{sx}$, we showed that our checks can be formulated using the language of the principal relation, where the rank of the relation n' should simultaneously divide $l, (m + \ell m) \cdot b$ and $\bar{m}_2$.

The reason why we decided to reveal $\mathbf{z}_x$ in the clear is the fact that in the third verification check of Equation D.12, there is a “quadratic” term in $\mathbf{z}_x$ and $\mathbf{z}_{sx}$, i.e. $\gamma_2^\top [c\mathbf{A} \mid c\mathbf{B} - \mathbf{z}_x^\top \otimes \mathbf{G}] \mathbf{G}_\beta \cdot \mathbf{z}_{sx}$, to which we have not found a straightforward way to translate to the principal relation (without going through a generic transformation via Rank-1 Constraint System).

Non-interactive zero-knowledge proofs. For anonymous credential applications, we need the proof system to be non-interactive. To this end, we apply the Fiat-Shamir transform. We refer to the security of both LNP and LaBRADOR in the random oracle model in [LNP22b] and [AAB⁺24] respectively.

E Randomized PRFA

In this section, we present a *randomized* version of Construction 4.1. It prevents the potential recovery of a trapdoor in the event that the replacement-free condition is violated by randomizing the verification matrix associated to an element $\mathbf{x}$ everytime a membership witness is issued for $\mathbf{x}$. However, the size of the public parameters and membership witnesses are larger by a factor of ℓ_2 , where ℓ_2 represents the bit length of the randomness.

Construction E.1. Let $(\text{TrapGen}, \text{SampleLeft})$ be algorithms defined in Theorem 2.1 and Lemma 2.6, and $(\text{Eval}, \text{EvalFx})$ be algorithms defined in Theorem 2.2. For any $\lambda \in \mathbb{N}$, let $n = n(\lambda)$, $m = m(\lambda)$, $d = d(\lambda)$, and $q = q(\lambda)$ be lattice parameters, and let $m' = n \lceil \log q \rceil$. Let $\mathfrak{s} = \mathfrak{s}(\lambda)$ be a Gaussian width parameter, $\gamma = \gamma(\lambda)$ be a bound on the number of deletions, $\ell_1 = \ell_1(\lambda)$ be an input length, and $\ell_2 = \ell_2(\lambda)$ be the randomness length, and $\beta = \beta(\lambda)$ be a norm bound. We construct a randomized PRFA for the domain $\mathcal{M} = \{0, 1\}^\ell$ as follows:

- $\text{Gen}(1^\lambda, \text{aux} = (1^n, 1^m, q))$: Given λ and $\text{aux} = (1^n, 1^m, q)$, do the following:
 1. Compute $(\mathbf{A}, \mathbf{T}_\mathbf{A}) \leftarrow \text{TrapGen}(1^n, 1^m, q)$.
 2. Sample $[\mathbf{A}_0 \mid \dots \mid \mathbf{A}_{\ell_2}] \leftarrow \$ R_q^{n \times (\ell_2 + 1)m'}$ and $\mathbf{B} \leftarrow \$ R_q^{n \times \ell_1 m'}$.
 3. Sample $\mathbf{c}_0 \leftarrow \$ R_q^n$, $\mathbf{u} \leftarrow \$ R_q^n$ and $k \leftarrow \{0, 1\}^\lambda$.
 4. Initialize an empty dictionary $\text{DX} : \mathcal{M} \rightarrow \{\text{Add}, \text{Del}, \perp\}$ and a counter $\text{ctr} := 0$.
 5. Set $\text{pp} := (\mathbf{A}, \mathbf{B}, \mathbf{A}_0, \dots, \mathbf{A}_{\ell_2}, \mathbf{u})$, $\text{sk} := \mathbf{T}_\mathbf{A}$, $\mathcal{A}_0 := \mathbf{c}_0$, and $\text{st} := (\text{ctr}, \text{DX})$.
 6. Return $(\text{pp}, \text{sk}, \mathcal{A}_0, \text{st})$.
- $\text{Add}(\text{pp}, \text{sk}, \text{st}, \mathcal{A}, \mathbf{x})$: Given $(\mathbf{A}, \mathbf{B}, \mathbf{A}_0, \dots, \mathbf{A}_{\ell_2}, \mathbf{u}) := \text{pp}$, $\mathbf{T}_\mathbf{A} := \text{sk}$, $(\text{ctr}, \text{DX}) := \text{st}$, $\mathbf{c} := \mathcal{A}$, $\mathbf{x} \in \mathcal{M}$, do the following:
 1. Parse $\text{Op} := \text{DX}[\mathbf{x}]$, and if $\text{Op} \neq \perp$, abort. ▶ $\mathbf{x}$ was already added or deleted
 2. Sample $\mathbf{r}_\mathbf{x} \leftarrow \$ \{0, 1\}^{\ell_2}$.
 3. Compute $\mathbf{A}_{\mathbf{r}_\mathbf{x}} := \mathbf{A}_0 + \sum_{i=1}^{\ell_2} \mathbf{r}_\mathbf{x}[i-1] \mathbf{A}_i$.
 4. Compute $\mathbf{s}_\mathbf{x} \leftarrow \text{SampleLeft}([\mathbf{A} \mid \mathbf{B} - \mathbf{x}^\top \otimes \mathbf{G} \mid \mathbf{A}_{\mathbf{r}_\mathbf{x}}], \mathbf{T}_\mathbf{A}, \mathbf{c}, \mathfrak{s})$.
 5. Set $\text{DX}[\mathbf{x}] := \text{Add}$, $w_\mathbf{x} := (\mathbf{r}_\mathbf{x}, \mathbf{s}_\mathbf{x})$, and $\text{upmsg} = (\text{Add}, \perp)$.
 6. Set $\text{st}' := (\text{ctr}, \text{DX})$.
 7. Return $(\mathcal{A}, w_\mathbf{x}, \text{upmsg}, \text{st}')$.
- $\text{Delete}(\text{pp}, \text{sk}, \text{st}, \mathcal{A}, \mathbf{x}, w_\mathbf{x})$: Given $(\mathbf{A}, \mathbf{B}, \mathbf{A}_0, \dots, \mathbf{A}_{\ell_2}, \mathbf{u}) := \text{pp}$, $\mathbf{T}_\mathbf{A} := \text{sk}$, $(\text{ctr}, \text{DX}) := \text{st}$, $\mathbf{C} := \mathcal{A}$, $\mathbf{x} \in \mathcal{M}$, and $(\mathbf{r}_\mathbf{x}, \mathbf{s}_\mathbf{x}) := w_\mathbf{x}$, do the following:
 1. If $\text{ctr} > \gamma$, abort. ▶ We have exceeded the allowed number of deletions
 2. Parse $\text{Op} := \text{DX}[\mathbf{x}]$, and if $\text{Op} \neq \text{Add}$, abort. ▶ $\mathbf{x}$ was already deleted or never added
 3. Compute $\mathbf{B}_{\perp_\mathbf{x}} \leftarrow \text{EvalF}(\mathbb{I}_\mathbf{x}, \mathbf{B})$, and then $\mathbf{c}' \leftarrow \mathbf{c} + \mathbf{B}_{\perp_\mathbf{x}} \cdot \mathbf{G}^{-1}(\mathbf{u})$.
 4. Set $\text{ctr}' \leftarrow \text{ctr} + 1$ and $\mathcal{A}' := \mathbf{c}'$.
 5. Set $\text{DX}[\mathbf{x}] := \text{Del}$, $\text{upmsg} := (\text{Del}, \mathbf{x})$, and $\text{st}' := (\text{ctr}', \text{DX})$.
 6. Return $(\mathcal{A}', \text{upmsg}, \text{st}')$.
- $\text{MemWitUp}(\text{pp}, \mathbf{x}, w_\mathbf{x}, \text{upmsg})$: Given $(\mathbf{A}, \mathbf{B}, \mathbf{A}_0, \dots, \mathbf{A}_{\ell_2}, \mathbf{u}) := \text{pp}$, $\mathbf{x} \in \mathcal{M}$, $(\mathbf{r}_\mathbf{x}, \mathbf{s}_\mathbf{x}) := w_\mathbf{x}$, $(\text{Op}, \mathbf{y}) := \text{upmsg}$, do the following:
 1. If $\text{Op} = \text{Add}$: do nothing.
 2. Else, if $\text{Op} = \text{Del}$:

- Compute $\mathbf{H}_{\mathbb{1}_y, \mathbf{x}} \leftarrow \text{EvalFx}(\mathbb{1}_y, \mathbf{B}, \mathbf{x})$, and then $\mathbf{s}'_{\mathbf{x}} \leftarrow \mathbf{s}_{\mathbf{x}} + [\mathbf{0}_{m' \times m} \mid \mathbf{H}_{\mathbb{1}_y, \mathbf{x}}^\top \mid \mathbf{0}_{m' \times m}]^\top \cdot \mathbf{G}^{-1}(\mathbf{u})$.
 - Return $(\mathbf{r}_{\mathbf{x}}, \mathbf{s}'_{\mathbf{x}})$ as $w'_{\mathbf{x}}$.
- **MemVer**(pp, $\mathcal{A}$, $\mathbf{x}$, $w_{\mathbf{x}}$): Given $(\mathbf{A}, \mathbf{B}, \mathbf{A}_0, \dots, \mathbf{A}_{\ell_2}, \mathbf{u}) := \text{pp}$, $\mathbf{c} := \mathcal{A}$, $\mathbf{x} \in \mathcal{M}$, and $(\mathbf{r}_{\mathbf{x}}, \mathbf{s}_{\mathbf{x}}) := w_{\mathbf{x}}$, do the following:
 1. Compute $\mathbf{A}_{\mathbf{r}_{\mathbf{x}}} := \mathbf{A}_0 + \sum_{i=1}^{\ell_2} \mathbf{r}_{\mathbf{x}}[i-1] \mathbf{A}_i$.
 2. If $[\mathbf{A} \mid \mathbf{B} - \mathbf{x}^\top \otimes \mathbf{G} \mid \mathbf{A}_{\mathbf{r}_{\mathbf{x}}}] \cdot \mathbf{s}_{\mathbf{x}} = \mathbf{c}$ and $\|\mathbf{s}_{\mathbf{x}}\| \leq \beta$, return 1.
 3. Otherwise, return 0.
 - **MemWitCreate**(pp, sk, st, $\mathcal{A}$, $\mathbf{x}$): Given $(\mathbf{A}, \mathbf{B}, \mathbf{A}_0, \dots, \mathbf{A}_{\ell_2}, \mathbf{u}) := \text{pp}$, $\mathbf{T}_{\mathbf{A}} := \text{sk}$, $(\text{ctr}, \text{DX}) := \text{st}$, $\mathbf{c} := \mathcal{A}$, $\mathbf{x} \in \mathcal{M}$, do the following:
 1. Parse $\text{Op} := \text{DX}[\mathbf{x}]$, and if $\text{Op} \neq \text{Add}$, abort. $\blacktriangleright$ $\mathbf{x}$ is not in the accumulator
 2. Sample $\mathbf{r}_{\mathbf{x}} \leftarrow \$_{\{0,1\}^{\ell_2}}$.
 3. Compute $\mathbf{A}_{\mathbf{r}_{\mathbf{x}}} := \mathbf{A}_0 + \sum_{i=1}^{\ell_2} \mathbf{r}_{\mathbf{x}}[i-1] \mathbf{A}_i$.
 4. Compute $\mathbf{s}_{\mathbf{x}} \leftarrow \text{SampleLeft}([\mathbf{A} \mid \mathbf{B} - \mathbf{x}^\top \otimes \mathbf{G} \mid \mathbf{A}_{\mathbf{r}_{\mathbf{x}}}], \mathbf{T}_{\mathbf{A}}, \mathbf{c}, \mathfrak{s})$.
 5. Return $(\mathbf{r}_{\mathbf{x}}, \mathbf{s}_{\mathbf{x}})$ as $w_{\mathbf{x}}$.

The same arguments used to prove correctness and compactness of Construction 4.1 can be applied to this construction by assuming that $\ell_2 = \text{poly}(\lambda)$. However, the process of proving selective collision-freeness is different: for an adversary $\mathcal{A}$, we consider whether the randomness $\mathbf{r}_{\mathbf{x}^*}$ in its forged membership witness $w_{\mathbf{x}^*} = (\mathbf{r}_{\mathbf{x}^*}, \mathbf{s}_{\mathbf{x}^*})$ is unique or was previously issued among one of $\mathbf{x}^*$'s membership witnesses it received before issuing a Delete request for $\mathbf{x}^*$. In brief, if $\mathcal{A}$ outputs a forgery such that $\mathbf{r}_{\mathbf{x}^*}$ is unique, we can use the prefix guessing strategy used by Libert et al [LLM⁺16] to guess the longest common prefix between $\mathbf{r}_{\mathbf{x}^*}$ and other randomnesses that were issued during Add and MemWitCreate queries for $\mathbf{x}^*$. Then, we can use that information to solve an M-SIS instance. If $\mathbf{r}_{\mathbf{x}^*}$ was reused, we can use a proof strategy similar to the one used to prove Theorem 4.3.

Proposition E.1 (Selective Collision-Freeness: Case 1). *For any $\lambda \in \mathbb{N}$, let $q = q(\lambda)$ be a integer that satisfies Lemma 2.4, $n = n(\lambda)$, $m' = n \lceil \log q \rceil$, $m = O(m')$ be lattice parameters. Let $\mathfrak{b} \geq 2, \alpha \geq 2, \varepsilon = \text{negl}(\lambda)$ and $\varsigma > 2d \cdot q^{n/m+2/(dm)}$. Let $\theta_1 = \sqrt{2d/3}(\sqrt{m} + \sqrt{m'} + \log \sqrt{d})$ and $\theta_2 = \sqrt{2d/3}(\sqrt{m} + \sqrt{\ell m'} + \log \sqrt{d})$. Let $\ell_1 = \ell_1(\lambda)$ be the input length and $\ell_2 = \ell_2(\lambda)$ be the randomness length, $\gamma = \gamma(\lambda)$ be a bound on the number of deletions, $T = \gamma(\mathfrak{b}-1)\sqrt{m'd}\sqrt{((\mathfrak{b}-1)dm'\sqrt{\ell}-1)^2 + 1}$ be an upper bound on ℓ_2 -norm of the vector used to update witnesses, $\mathfrak{s} = \max((\ell_2+1)\mathfrak{b}\theta_2 \log \sqrt{nd}, \mathfrak{b}\varsigma\sqrt{d}(\sqrt{m} + \sqrt{m'} + \log \sqrt{d}) \log \sqrt{nd})$ a Gaussian width parameter, and $\beta_0 = \mathfrak{s}\sqrt{d(m+m'(\ell+1))} + T$ be a norm bound on the witnesses. Let*

$$\beta_1 = 2\sqrt{md} + \theta_2 T + \beta_0 \sqrt{1 + \theta_2^2 + (\ell_2 + 1)^2 \theta_1^2}$$

Suppose the assumptions $\text{M-LWE}_{m-n, m, q, U(S_1^\infty)}$, $\text{M-LWE}_{m-n, m, q, U(\mathcal{B}_1)}$, and $\text{M-SIS}_{n, m, q, \beta_1}$ hold. Then Construction E.1 is a selectively collision-free PRFA (cf. Definitions 3.3 and 3.4) against a ppt adversary $\mathcal{A}$ that does not reuse randomness associated to a membership witness and performs at most one Add query and $Q - 1$ MemWitCreate queries on its target $\mathbf{x}^*$.

Proof. We proceed via a sequence of hybrids:

- **Hyb₀**: This is the real selective collision-freeness experiment (similar to Hyb₀ in the proof of Theorem 4.3, but once $\mathcal{A}$ outputs its forgery $w_{\mathbf{x}^*} = (\mathbf{r}_{\mathbf{x}^*}, \hat{\mathbf{s}}_{\mathbf{x}^*})$, we abort if $\mathbf{x}^*$ was previously issued during Add or MemWitCreate query on $\mathbf{x}^*$).

- **Hyb₁**: Similar to **Hyb₀**. However, during the execution of **Gen**, the challenger samples $\mathbf{B}' \leftarrow \$ R_q^{\ell m' \times m'}$ and sets $\mathbf{B} \leftarrow \mathbf{B}' + \mathbf{x}^* \otimes \mathbf{G}$. This modification does not change the distribution of $\mathbf{B}$. Therefore, $\text{Hyb}_0 \stackrel{\approx}{\approx} \text{Hyb}_1$.

- **Hyb₂**: Similar to **Hyb₁**. However, during the execution of **Gen**, the challenger samples $[\mathbf{R}_0 \mid \cdots \mid \mathbf{R}_{\ell_1-1}]$ where $\mathbf{R}_i \leftarrow \$ (S_1^\infty)^{m \times m'}$, then, sets $\mathbf{B}' \leftarrow \mathbf{A}[\mathbf{R}_0 \mid \cdots \mid \mathbf{R}_{\ell_1-1}]$ and stores $[\mathbf{R}_0 \mid \cdots \mid \mathbf{R}_{\ell_1-1}]$ in its state **st**.

$\text{Hyb}_1 \stackrel{\approx}{\approx} \text{Hyb}_2$ following from the $\text{M-LWE}_{m-n,m,q,U(S_1^\infty)}$ assumption.

- **Hyb₃**: Similar to **Hyb₄**. However, during the execution of **Gen**, the challenger samples $(\mathbf{r}_1, \mathbf{r}_2) \leftarrow \$ \mathcal{B}_1^{2m}$, sets $\mathbf{c}_0 = \mathbf{A}\mathbf{r}_1$ and $\mathbf{u} = \mathbf{A}\mathbf{r}_2$, and finally stores $(\mathbf{r}_1, \mathbf{r}_2)$ in its state **st**.

$\text{Hyb}_2 \stackrel{\approx}{\approx} \text{Hyb}_3$ following from the $\text{M-LWE}_{m-n,m,q,U(\mathcal{B}_1)}$ assumption.

- **Hyb₄**: Similar to **Hyb₃**. However, during the execution of **Gen**, the challenger samples $i^* \leftarrow \$ [Q]$ and $t^* \leftarrow \$ [\ell_2]$. Then, it sets $\mathbf{e} := \mathbf{r}_{i^*}[0, \dots, t^*-2] \mid \mathbf{r}_{i^*}[t-1]$ ($\mathbf{e}$ is the guest for the longest prefix). Next, it samples $[\mathbf{Q}_0 \mid \cdots \mid \mathbf{Q}_{\ell_2}]$ where $\mathbf{Q}_i \leftarrow \$ (S_1^\infty)^{m \times m'}$ and sets $(\mathbf{A}_0, \dots, \mathbf{A}_{\ell_2})$ as follows:

$$\begin{cases} \mathbf{A}_0 = \mathbf{A} \cdot \mathbf{Q}_0 + \sum_{i=0}^{t^*-1} \mathbf{e}[i] \mathbf{G} \\ \mathbf{A}_j = \mathbf{A} \cdot \mathbf{Q}_j + (-1)^{\mathbf{e}[j-1]} \mathbf{G} & \text{for } 1 \leq j \leq t^* \\ \mathbf{A}_j = \mathbf{A} \cdot \mathbf{Q}_j & \text{for } t^* < j \leq \ell_2 \end{cases}$$

$\text{Hyb}_3 \stackrel{\approx}{\approx} \text{Hyb}_4$ follows from the $\text{M-LWE}_{m-n,m,q,U(S_1^\infty)}$ assumption.

- **Hyb₅**: Similar to **Hyb₄**. However, during the execution of **Gen**, the challenger samples $\mathbf{A} \leftarrow \$ R_q^{n \times m}$.

Given that $\mathfrak{s} = \max\{(\ell_2 + 1)b\theta_2 \log \sqrt{nd}, b\varsigma \sqrt{d}(\sqrt{m} + \sqrt{m'} + \log \sqrt{d}) \log \sqrt{nd}\}$, it follows $\text{Hyb}_4 \stackrel{\approx}{\approx} \text{Hyb}_5$ by Theorem 2.1 and Lemma 2.6.

For any **Add** or **MemWitCreate** query on $\mathbf{x} \neq \mathbf{x}^*$, the challenger/reduction can use $\mathbf{R}_i$ such that $\mathbf{x}[i] \neq \mathbf{x}^*[i]$ to answer. For any valid **Add** or **MemWitCreate** on $\mathbf{x}^*$, the reduction can use $[\mathbf{Q}_0 \mid \cdots \mid \mathbf{Q}_{\ell_2}]$ to recover a $\mathbf{G}$ -trapdoor for the matrix $[\mathbf{A} \mid \mathbf{B}' \mid \mathbf{A}_{\mathbf{r}_i}]$ with $\mathbf{r}_i \in \{\mathbf{r}_0, \dots, \mathbf{r}_{\ell_2-1}\}$ to answer the query. Without loss of generality, we assumed that $\mathcal{A}$ performed an **add** query $\mathbf{x}^*$. Hence, by Corollary 2.2.1, there exists a matrix $\tilde{\mathbf{H}} \in R^{\ell_1 m' \times m'}$ such that once $\mathcal{A}$ outputs its forgery $w_{\mathbf{x}^*} = (\mathbf{r}_{\mathbf{x}^*}, \hat{\mathbf{s}}_{\mathbf{x}^*})$, we have:

$$\begin{aligned} & \mathbf{A} \left[\mathbf{I}_m \mid \mathbf{R}_0 \mid \cdots \mid \mathbf{R}_{\ell_1-1} \mid \mathbf{Q}_0 + \sum_{i=1}^{\ell_2} \mathbf{r}_{\mathbf{x}^*}[i-1] \mathbf{Q}_i \right] \cdot \left(\hat{\mathbf{s}}_{\mathbf{x}^*} - \begin{bmatrix} \mathbf{r}_1 \\ \tilde{\mathbf{H}} \cdot \mathbf{G}^{-1}(\mathbf{u}) \\ \mathbf{0}_{\ell m'} \end{bmatrix} \right) = \mathbf{u} \\ \Rightarrow & \underbrace{\mathbf{A} \left(\left[\mathbf{I}_m \mid \mathbf{R}_0 \mid \cdots \mid \mathbf{R}_{\ell_1-1} \mid \mathbf{Q}_0 + \sum_{i=1}^{\ell_2} \mathbf{r}_{\mathbf{x}^*}[i-1] \mathbf{Q}_i \right] \cdot \left(\hat{\mathbf{s}}_{\mathbf{x}^*} - \begin{bmatrix} \mathbf{r}_1 \\ \tilde{\mathbf{H}} \cdot \mathbf{G}^{-1}(\mathbf{u}) \\ \mathbf{0}_{\ell m'} \end{bmatrix} \right) \right)}_{\mathbf{v}} - \mathbf{r}_2 = \mathbf{0} \end{aligned}$$

and $\mathbf{v} \neq \mathbf{0}$ with overwhelming probability given that $\mathbf{r}_2 \sim U(\mathcal{B}_1^m)$. In addition,

$$\begin{aligned}
\|\mathbf{v}\| &\leq \|\mathbf{r}_1\| + \|\mathbf{r}_2\| + s_1([\mathbf{R}_0 \mid \cdots \mid \mathbf{R}_{\ell_1-1}]) \left\| \tilde{\mathbf{H}} \cdot \mathbf{G}^{-1}(\mathbf{u}) \right\| \\
&\quad + \|\hat{\mathbf{s}}_{\mathbf{x}^*}\| s_1 \left(\left[\mathbf{I}_m \mid \mathbf{R}_0 \mid \cdots \mid \mathbf{R}_{\ell_1-1} \mid \mathbf{Q}_0 + \sum_{i=1}^{\ell_2} \mathbf{r}_{\mathbf{x}^*}[i-1] \mathbf{Q}_i \right] \right) \\
&\leq 2\sqrt{md} + \theta_2 T + \beta_0 \sqrt{1 + s_1([\mathbf{R}_0 \mid \cdots \mid \mathbf{R}_{\ell_1-1}])^2 + s_1 \left(\mathbf{Q}_0 + \sum_{i=1}^{\ell_2} \mathbf{r}_{\mathbf{x}^*}[i-1] \mathbf{Q}_i \right)^2} \\
&\leq 2\sqrt{md} + \theta_2 T + \beta_0 \sqrt{1 + \theta_2^2 + (\ell_2 + 1)^2 \theta_1^2}
\end{aligned}$$

where the last inequality follows from lemmas 2.3, 4.1, and A.2. Therefore, except with probability $1/(Q \cdot \ell_2)$, we can use $\mathcal{A}$ to find a solution to an M-SIS $_{n,m,q,\beta_1}$ instance. $\square$

Proposition E.2 (Selective Collision-Freeness: Case 2). *For any $\lambda \in \mathbb{N}$, let $q = q(\lambda)$ be a integer that satisfies Lemma 2.4, $n = n(\lambda)$, $m' = n \lceil \log q \rceil$, $m = O(m')$ be lattice parameters. Let $\mathbf{b} \geq 2, \alpha \geq 2, \varepsilon = \text{negl}(\lambda)$ and $\varsigma > 2d \cdot q^{n/m+2/(dm)}$. Let $\theta_1 = \sqrt{2d/3}(\sqrt{m} + \sqrt{m'} + \log \sqrt{d})$ and $\theta_2 = \sqrt{2d/3}(\sqrt{m} + \sqrt{\ell m'} + \log \sqrt{d})$. Let $\ell_1 = \ell_1(\lambda)$ be the input length and $\ell_2 = \ell_2(\lambda)$ be the randomness length, $\gamma = \gamma(\lambda)$ be a bound on the number of deletions, $T = \gamma(\mathbf{b} - 1)\sqrt{m'd}\sqrt{((\mathbf{b} - 1)dm'\sqrt{\ell} - 1)^2 + 1}$ be an upper bound on ℓ_2 -norm of the vector used to update witnesses, $\mathfrak{s} = \max(T, (\ell_2 + 1)\mathbf{b}\theta_2 \log \sqrt{nd}, \mathbf{b}\varsigma\sqrt{d}(\sqrt{m} + \sqrt{m'} + \log \sqrt{d}) \log \sqrt{nd})$ a Gaussian width parameter, and $\beta_0 = \mathfrak{s}\sqrt{d(m + m'(\ell + 1))} + T$ be a norm bound on the witnesses. Let*

$$\beta_1 = (\mathfrak{s} + 1)\sqrt{md} + \theta_2 T + \mathfrak{s}\sqrt{dm'(\ell_1 + 1)}\sqrt{\theta_2^2 + (\ell_2 + 1)^2 \theta_1^2} + \beta_0 \sqrt{1 + \theta_2^2 + (\ell_2 + 1)^2 \theta_1^2}$$

Suppose the assumptions M-LWE $_{m-n,m,q,U(S_1^\infty)}$, M-LWE $_{m-n,m,q,U(\mathcal{B}_1)}$, and M-SIS $_{n,m,q,\beta_1}$ hold. Then, Construction E.1 is a selectively collision free PRFA (cf. Definitions 3.3 and 3.4) against a ppt adversary $\mathcal{A}$ that reuses a randomness associated to a membership witness and performs at most one Add query and $Q - 1$ MemWitCreate queries on its target.

Proof. We proceed via a sequence of hybrids:

- Hyb₀: This is the real selective collision-freeness experiment (similar to Hyb₀ in the proof of Theorem 4.3, but once $\mathcal{A}$ outputs its forgery $w_{\mathbf{x}^*} = (\mathbf{r}_{\mathbf{x}^*}, \hat{\mathbf{s}}_{\mathbf{x}^*})$, we abort if $\mathbf{x}^*$ never issued during and Add or MemWitCreate query on $\mathbf{x}^*$).
 - Hyb₁: Similar to Hyb₀. However, during the execution of Gen, the challenger samples $\mathbf{B}' \leftarrow \$ R_q^{\ell m' \times m'}$ and sets $\mathbf{B} \leftarrow \mathbf{B}' + \mathbf{x}^* \otimes \mathbf{G}$. This modification does not change the distribution of $\mathbf{B}$. Therefore, $\text{Hyb}_0 \stackrel{c}{\approx} \text{Hyb}_1$.
 - Hyb₂: Similar to Hyb₁. However, during the execution of Gen, the challenger samples $[\mathbf{R}_0 \mid \cdots \mid \mathbf{R}_{\ell_1-1}]$ where $\mathbf{R}_i \leftarrow \$ (S_1^\infty)^{m \times m'}$, then, sets $\mathbf{B}' \leftarrow \mathbf{A}[\mathbf{R}_0 \mid \cdots \mid \mathbf{R}_{\ell_1-1}]$ and stores $[\mathbf{R}_0 \mid \cdots \mid \mathbf{R}_{\ell_1-1}]$ in its state st.
- Hyb₁ $\stackrel{c}{\approx}$ Hyb₂ following from the M-LWE $_{m-n,m,q,U(S_1^\infty)}$ assumption.
- Hyb₃: Similar to Hyb₂. However, during the execution of Gen, the challenger samples $(\mathbf{r}) \leftarrow \$ \mathcal{B}_1^{2m}$, sets $\mathbf{u} = \mathbf{A}\mathbf{r}$, and finally stores $\mathbf{r}$ in its state st.
- Hyb₂ $\stackrel{c}{\approx}$ Hyb₃ following from the M-LWE $_{m-n,m,q,U(\mathcal{B}_1)}$ assumption.

- Hyb₄: Similar to Hyb₃. However, during the execution of Gen, the challenger samples $\mathbf{r}_0, \dots, \mathbf{r}_{Q-1}$ such that $\mathbf{r}_i \leftarrow \{0, 1\}^{\ell_2}$ and stores them in its state st. Everytime $\mathcal{A}$ issues a valid Add or MemWitCreate query on $\mathbf{x}^*$, the challenger samples without replacement one of the $\mathbf{r}_i$ and use it to answer $\mathcal{A}$'s query.

Given that this is just a programmatical change, it follows that $\text{Hyb}_3 \approx \text{Hyb}_4$.

- Hyb₅: Similar to Hyb₄. However, during the execution of Gen, the challenger samples $i^* \leftarrow [Q]$. Next, it samples $k_0, \dots, k_{\ell_2} \leftarrow \mathbb{Z}_q$ such that these following condition are satisfied:

- For any $\mathbf{r}_i \neq \mathbf{r}_{i^*}$, $k_0 + \sum_{i=i}^{\ell_2} k_i \cdot \mathbf{r}_i[i-1] \neq 0 \pmod q$
- For $\mathbf{r}_{i^*}$, $k_0 + \sum_{i=i}^{\ell_2} k_i \cdot \mathbf{r}_i[i-1] = 0 \pmod q$

Then, it samples $[\mathbf{Q}_0 \mid \dots \mid \mathbf{Q}_{\ell_2}] \leftarrow (S_1^\infty)^{m \times \ell_2 m'}$ and sets $(\mathbf{A}_0, \dots, \mathbf{A}_{\ell_2})$ as follows: $\mathbf{A}_i = \mathbf{A} \cdot \mathbf{Q}_i + k_i \mathbf{G}$. Finally, it stores i^* in its state st.

$\text{Hyb}_4 \approx \text{Hyb}_5$ following from the M-LWE $_{m-n, m, q, U(S_1^\infty)}$ assumption.

- Hyb₄: Similar to Hyb₃. However, to answer $\mathcal{A}$'s i^* -th query with respect to $\mathbf{x}^*$, the challenger as follows:

1. Compute $\mathbf{A}_{\mathbf{r}_{i^*}} := \mathbf{A}_0 + \sum_{i=1}^{\ell_2} \mathbf{r}_{i^*}[i-1] \mathbf{A}_i$. ► Note that $\mathbf{A}_{\mathbf{r}_{i^*}} = \mathbf{A} \left(\mathbf{Q}_0 + \sum_{i=1}^{\ell_2} \mathbf{r}_{i^*}[i-1] \mathbf{Q}_i \right)$
2. Sample $\mathbf{s}_{\mathbf{x}^*, 1} \leftarrow (D_{R, \mathbf{s}})^{\ell m' + m}$.
3. Compute $\mathbf{s}_{\mathbf{x}^*, 0} \leftarrow \text{SamplePre}(\mathbf{A}, \mathbf{T}_{\mathbf{A}}, \mathbf{c} - [\mathbf{B} - (\mathbf{x}^*)^\top \otimes \mathbf{G} \mid \mathbf{A}_{\mathbf{r}_{i^*}}] \cdot \mathbf{s}_{\mathbf{x}^*, 1}, \mathbf{s})$.
4. Set $\mathbf{s}_{\mathbf{x}^*} := [\mathbf{s}_{\mathbf{x}^*, 0}^\top \mid \mathbf{s}_{\mathbf{x}^*, 1}^\top]^\top$.
5. Return $\mathbf{s}_{\mathbf{x}^*}$.

$\text{Hyb}_3 \approx \text{Hyb}_4$ following Lemma A.1.

- From this point on, the remaining hybrids are similar to those describe in the proof of Theorem 4.3 (Hyb₆ to Hyb₉). However instead of adapting $\mathbf{s}_{\mathbf{x}^*}$ with respect to an Add query on $\mathbf{x}^*$, we adapt $\mathbf{s}_{\mathbf{x}^*}$ with respect to the i^* -th query associated to $\mathbf{x}^*$. Observe that if $i^* = 0$, then the query associated to $\mathbf{x}^*$ is an Add query. Otherwise, it is a MemWitCreate query. In addition, $[\mathbf{Q}_0 \mid \dots \mid \mathbf{Q}_{\ell_2}]$ can be used to recover a $\mathbf{G}$ trapdoor that can help the challenger/reduction answer any $i \neq i^*$ query associated to $\mathbf{x}^*$.

Now, suppose $\mathcal{A}$ outputs its forgery $w_{\mathbf{x}^*} = (\hat{\mathbf{r}}_{\mathbf{x}^*}, \hat{\mathbf{s}}_{\mathbf{x}^*})$, and the reduction correctly guess the reused randomness, i.e., $\hat{\mathbf{r}}_{\mathbf{x}^*} = \mathbf{r}_{i^*}$. Given how the matrices $\mathbf{A}_0, \dots, \mathbf{A}_{\ell_2}$ were constructed, we have

$$\mathbf{A}_{\mathbf{r}_{i^*}} := \mathbf{A}_0 + \sum_{i=1}^{\ell_2} \mathbf{r}_{i^*}[i-1] \mathbf{A}_i = \left(\mathbf{Q}_0 + \sum_{i=1}^{\ell_2} \mathbf{r}_{i^*}[i-1] \mathbf{Q}_i \right)$$

Observe that, by Corollary 2.2.1, there exists $\tilde{\mathbf{H}}$ such that

$$\begin{aligned} & \mathbf{A} \left[\mathbf{I}_m \mid \mathbf{R}_0 \mid \dots \mid \mathbf{R}_{\ell_1-1} \mid \mathbf{Q}_0 + \sum_{i=1}^{\ell_2} \mathbf{r}_{i^*}[i-1] \mathbf{Q}_i \right] \left(\hat{\mathbf{s}}_{\mathbf{x}^*} - \begin{bmatrix} \mathbf{s}_{\mathbf{x}^*, 0} \\ \mathbf{s}_{\mathbf{x}^*, 1} + \begin{pmatrix} \tilde{\mathbf{H}} \cdot \mathbf{G}^{-1}(\mathbf{u}) \\ \mathbf{0}_m \end{pmatrix} \end{bmatrix} \right) = \mathbf{u} \\ \implies & \underbrace{\mathbf{A} \left(\begin{bmatrix} \mathbf{I}_m \mid \mathbf{R}_0 \mid \dots \mid \mathbf{R}_{\ell_1-1} \mid \mathbf{Q}_0 + \sum_{i=1}^{\ell_2} \mathbf{r}_{i^*}[i-1] \mathbf{Q}_i \end{bmatrix} \left(\hat{\mathbf{s}}_{\mathbf{x}^*} - \begin{bmatrix} \mathbf{s}_{\mathbf{x}^*, 0} \\ \mathbf{s}_{\mathbf{x}^*, 1} + \begin{pmatrix} \tilde{\mathbf{H}} \cdot \mathbf{G}^{-1}(\mathbf{u}) \\ \mathbf{0}_m \end{pmatrix} \end{bmatrix} \right) \right)}_{\mathbf{v}} - \mathbf{r} = \mathbf{0} \end{aligned}$$

and $\mathbf{v} \neq \mathbf{0}$ with overwhelming probability given that $\mathbf{r} \sim U(\mathcal{B}_1^m)$. In addition,

$$\begin{aligned}
\|\mathbf{v}\| &\leq \|\mathbf{r}\| + \|\mathbf{s}_{\mathbf{x}^*,0}\| + s_1([\mathbf{R}_0 \mid \cdots \mid \mathbf{R}_{\ell_1-1}]) \left\| \tilde{\mathbf{H}} \cdot \mathbf{G}^{-1}(\mathbf{u}) \right\| \\
&\quad + s_1 \left(\left[\mathbf{R}_0 \mid \cdots \mid \mathbf{R}_{\ell_1-1} \mid \mathbf{Q}_0 + \sum_{i=1}^{\ell_2} \mathbf{r}_{i^*}[i-1]\mathbf{Q}_i \right] \right) \|\mathbf{s}_{\mathbf{x}^*,1}\| \\
&\quad s_1 \left(\left[\mathbf{I}_m \mid \mathbf{R}_0 \mid \cdots \mid \mathbf{R}_{\ell_1-1} \mid \mathbf{Q}_0 + \sum_{i=1}^{\ell_2} \mathbf{r}_{i^*}[i-1]\mathbf{Q}_i \right] \right) \|\hat{\mathbf{s}}_{\mathbf{x}^*}\| \\
&\leq \sqrt{md} + \mathfrak{s}\sqrt{md} + \theta_2 T + \mathfrak{s}\sqrt{dm'(\ell_1+1)}\sqrt{\theta_2^2 + (\ell_2+1)^2\theta_1^2} + \beta_0\sqrt{1 + \theta_2^2 + (\ell_2+1)^2\theta_1^2}
\end{aligned}$$

where the last inequality follows from lemmas 2.2, 2.3, 4.1, and A.2. Therefore, except with probability $1/Q$, we can use $\mathcal{A}$ to find a solution to an M-SIS $_{n,m,q,\beta_1}$ instance. This completes the proof. $\square$